

Trabajo Práctico N° 1: **GNU/Linux, Instalación y Conceptos Básicos, Permisos, Arranque, Usuarios. Organización Interna.**

Ejercicio 1: Características de GNU/Linux.

(a) *Mencionar y explicar las características más relevantes de GNU/Linux.*

GNU/Linux es un sistema operativo libre y de código abierto, basado en el núcleo Linux y en las herramientas del proyecto GNU. Sus características más relevantes son:

1. Software libre y código abierto: GNU/Linux se distribuye bajo licencias libres (como la GPL), lo que permite a los usuarios usar, estudiar, modificar y redistribuir el *software*. Esta libertad fomenta la colaboración y la mejora continua del sistema.
2. Multitarea y multiusuario: Permite que varios usuarios trabajen, simultáneamente, en el mismo sistema sin interferir entre sí y que múltiples procesos se ejecuten al mismo tiempo. Esto lo hace ideal para entornos de servidores y redes.
3. Portabilidad: GNU/Linux puede ejecutarse en una amplia variedad de plataformas y arquitecturas (x86, ARM, RISC-V, etc.), desde servidores y computadoras personales hasta dispositivos embebidos y teléfonos.
4. Seguridad y estabilidad: Es reconocido por su gran estabilidad, incluso en ejecuciones prolongadas, y por su robusto sistema de permisos y usuarios, que limita el alcance de posibles ataques o errores. Además, la comunidad realiza actualizaciones y parches de seguridad constantemente.
5. Estructura modular: Está compuesto por módulos independientes (núcleo, *shell*, sistema de archivos, utilidades, etc.), lo que facilita su mantenimiento y personalización.
6. Sistema de archivos jerárquico: Organiza todos los recursos (archivos, dispositivos, configuraciones) en una única estructura jerárquica, que parte del directorio raíz (/).
7. Compatibilidad y flexibilidad: Existen numerosas distribuciones (Debian, Ubuntu, Fedora, Arch, etc.) adaptadas a distintos tipos de usuarios y necesidades. Además, permite elegir entre múltiples entornos de escritorio, gestores de paquetes y herramientas.
8. Comunidad y soporte colaborativo: GNU/Linux cuenta con una gran comunidad internacional que desarrolla, documenta y brinda soporte de manera colaborativa.

(b) *Mencionar otros sistemas operativos y compararlos con GNU/Linux en cuanto a los puntos mencionados en el inciso (a).*

Otros sistemas operativos, ampliamente utilizados, son Windows (Microsoft), macOS (Apple) y Android (Google). A continuación, se comparan con GNU/Linux en cuanto a los puntos mencionados en el inciso (a):

1. Software libre y código abierto:

- Windows: Es propietario y cerrado; el código fuente no está disponible y su uso está sujeto a licencias comerciales.
 - macOS: Es privativo y exclusivo del *hardware* de Apple, aunque incorpora componentes de código abierto (como partes de BSD).
 - Android: Parcialmente abierto, ya que el núcleo es Linux, pero muchas capas y servicios de Google son propietarios.
2. **Multitarea y multiusuario:**
 - Todos los sistemas mencionados permiten multitarea (ejecución simultánea de procesos). Sin embargo, GNU/Linux fue diseñado desde sus orígenes como multiusuario real, algo que, en Windows y macOS, se implementó más tarde y con mayores restricciones.
 3. **Portabilidad:**
 - Windows: Está optimizado para arquitecturas x86/x64; su portabilidad es limitada.
 - macOS: Funciona, exclusivamente, en equipos Apple.
 - Android: Muy extendido en dispositivos móviles, pero no tan flexible fuera de ese entorno.
 4. **Seguridad y estabilidad:**
 - Windows: Más vulnerable a virus y *malware*, principalmente por su popularidad y arquitectura de permisos más laxa.
 - macOS: Tiene buena seguridad, aunque no alcanza la flexibilidad ni el control de GNU/Linux.
 - Android: Depende de las capas del fabricante y de la actualización; puede presentar vulnerabilidades si no se actualiza.
 5. **Estructura modular:**
 - GNU/Linux: Altamente modular; el usuario puede modificar o reemplazar componentes del sistema.
 - Windows y macOS: Mucho más cerrados; el usuario tiene escaso control sobre los módulos del sistema.
 - Android: Parcialmente modular, pero condicionado por Google y los fabricantes.
 6. **Sistema de archivos jerárquico:**
 - Todos utilizan una estructura jerárquica, aunque en GNU/Linux es más uniforme (todo se organiza bajo “/”). En Windows, existen múltiples unidades (C:, D:, etc.), lo que fragmenta la estructura.
 7. **Compatibilidad y flexibilidad:**
 - Windows: Ampliamente compatible con programas comerciales y *hardware*, pero poco flexible; el usuario no puede modificar ni personalizar, en profundidad, el sistema.
 - macOS: Ofrece buena compatibilidad dentro del ecosistema Apple y gran estabilidad, pero es cerrado y limitado al *hardware* de la marca.
 - Android: Posee gran compatibilidad con aplicaciones móviles, pero su flexibilidad depende del fabricante y de las capas de *software* agregadas, que restringen la personalización completa.
 8. **Comunidad y soporte:**
 - Windows y macOS: El soporte depende de las empresas propietarias.
 - Android: Tiene soporte de Google y comunidades específicas, pero menos abiertas que en GNU/Linux.

En síntesis, GNU/Linux se destaca por su libertad, seguridad, estabilidad y flexibilidad, mientras que los sistemas operativos propietarios como Windows y macOS ofrecen mayor facilidad de uso y compatibilidad comercial, pero a costa de menor control por parte del usuario.

(c) *¿Qué es GNU?*

GNU es un proyecto de *software* libre iniciado en 1983 por Richard Stallman, con el objetivo de crear un sistema operativo completamente libre, compatible con Unix, pero sin incluir ningún componente privativo. El nombre GNU es un acrónimo recursivo que significa “GNU’s Not Unix” (“GNU No es Unix”).

El proyecto forma parte del movimiento del *software* libre, que defiende las cuatro libertades fundamentales del usuario:

- Usar el programa con cualquier propósito.
- Estudiar cómo funciona y adaptarlo a las necesidades propias.
- Redistribuir copias.
- Mejorar el programa y publicar esas mejoras.

El sistema GNU proporciona las herramientas básicas de un sistema operativo (compiladores, bibliotecas, *shells*, utilidades de administración, editores de texto, etc.).

En la actualidad, la mayoría de las distribuciones conocidas como “Linux” son, en realidad, combinaciones del núcleo Linux con las herramientas del proyecto GNU, motivo por el cual su nombre correcto es GNU/Linux.

(d) *Indicar una breve historia sobre la evolución del proyecto GNU.*

El proyecto GNU fue iniciado en 1983 por Richard Stallman en el Instituto Tecnológico de Massachusetts (MIT), con la idea de desarrollar un sistema operativo completamente libre y compatible con Unix.

En 1985, Stallman fundó la *Free Software Foundation* (FSF) para apoyar el desarrollo y la difusión del *software* libre, así como para promover licencias que garantizaran la libertad de los usuarios, como la *General Public Licence* de GNU (GPL).

Durante la segunda mitad de la década de 1980, el proyecto GNU avanzó en la creación de herramientas esenciales como el compilador GCC (*GNU Compiler Collection*), el editor Emacs, el intérprete de comandos Bash y muchas utilidades del sistema. Sin embargo, el sistema GNU aún no contaba con un núcleo funcional (llamado Hurd), que se encontraba en desarrollo.

En 1991, Linus Torvalds publicó el núcleo Linux, que era libre y compatible con las herramientas GNU. La combinación de ambos permitió conformar un sistema operativo

completo: GNU/Linux, el cual se difundió, rápidamente, en universidades, empresas y entornos domésticos.

Desde entonces, el proyecto GNU continúa activo, manteniendo y desarrollando múltiples programas libres, y promoviendo los principios éticos y sociales del movimiento del *software* libre.

(e) *Explicar qué es la multitarea e indicar si GNU/Linux hace uso de ella.*

La multitarea es la capacidad de un sistema operativo para ejecutar varios procesos o programas de manera simultánea, compartiendo los recursos del procesador, la memoria y otros dispositivos.

En realidad, el procesador alterna, rápidamente, entre las distintas tareas, dando la sensación de ejecución paralela (especialmente, en sistemas con un solo núcleo). En los procesadores multinúcleo, varias tareas pueden ejecutarse, verdaderamente, al mismo tiempo.

Existen dos tipos principales de multitarea:

- Cooperativa: Cada proceso cede, voluntariamente, el control al sistema operativo.
- Preventiva: El sistema operativo decide cuándo interrumpir un proceso para dar tiempo de CPU a otro.

GNU/Linux implementa multitarea preventiva, lo que significa que el núcleo administra, de forma automática, el uso del procesador entre los distintos procesos. Esto permite que el sistema siga funcionando de manera fluida, incluso cuando muchos programas están activos al mismo tiempo.

(f) *¿Qué es POSIX?*

POSIX significa *Portable Operating System Interface* (Interfaz Portátil para Sistemas Operativos). Es un conjunto de estándares definidos por IEEE (Instituto de Ingenieros Eléctricos y Electrónicos) que especifica una interfaz común para los sistemas operativos tipo Unix.

El objetivo de POSIX es garantizar que los programas puedan compilarse y ejecutarse en distintos sistemas operativos compatibles, sin necesidad de modificar su código fuente. Para lograrlo, define normas sobre aspectos como:

- Llamadas al sistema (*system calls*).
- Estructura del sistema de archivos.
- Manejo de procesos e hilos.
- Señales y comunicación entre procesos.
- Comportamiento del *shell* y utilidades básicas.

GNU/Linux cumple, en gran medida, con el estándar POSIX, lo que permite que muchas aplicaciones desarrolladas para sistemas Unix (como BSD o macOS) puedan ejecutarse en Linux con muy pocas modificaciones.

En resumen, POSIX promueve la portabilidad y la compatibilidad entre los distintos sistemas operativos de la familia Unix y sus derivados.

Ejercicio 2: Distribuciones de GNU/Linux.

(a) *¿Qué es una distribución de GNU/Linux? Nombrar, al menos, 4 distribuciones de GNU/Linux y citar diferencias básicas entre ellas.*

Una distribución de GNU/Linux es un paquete completo de *software* que combina el núcleo Linux, las herramientas del proyecto GNU y otros programas adicionales (como gestores de paquetes, entornos de escritorio y aplicaciones) para ofrecer un sistema operativo listo para instalar y usar. Cada distribución se adapta a distintos propósitos, como servidores, escritorios, educación o seguridad informática.

Algunas distribuciones populares son:

1. Debian:
 - Enfocada en estabilidad y robustez.
 - Ideal para servidores y entornos críticos.
 - Sus paquetes son más conservadores; menos actualizaciones frecuentes.
2. Ubuntu:
 - Orientada a usuarios de escritorio y principiantes.
 - Fácil instalación y amplia comunidad de soporte.
 - Basada en Debian, con un ciclo de actualizaciones regular y soporte a largo plazo (LTS).
3. Fedora:
 - Distribución vanguardista, incluye *software* más reciente.
 - Buena para desarrolladores y pruebas de nuevas tecnologías.
 - Respaldo de *Red Hat*, aunque con un ciclo de vida más corto.
4. Arch Linux:
 - Minimalista y, altamente, personalizable.
 - Filosofía “*rolling release*” (actualizaciones continuas).
 - Requiere mayor conocimiento técnico para su instalación y mantenimiento.

En resumen, cada distribución combina los mismos elementos básicos de GNU/Linux, pero se diferencian en facilidad de uso, frecuencia de actualizaciones, estabilidad y personalización.

(b) *¿En qué se diferencia una distribución de otra?*

Aunque todas las distribuciones de GNU/Linux comparten el núcleo Linux y las herramientas GNU, se diferencian, principalmente, en los siguientes aspectos:

1. Gestor de paquetes:
 - Cada distribución utiliza su propio sistema para instalar y actualizar *software*:
 - Debian/Ubuntu usan APT y paquetes .deb.
 - Fedora/Red Hat usan DNF/YUM y paquetes .rpm.
 - Arch Linux usa Pacman.
2. Ciclo de actualizaciones y estabilidad:

- Algunas distribuciones (Debian Stable, Ubuntu LTS) priorizan estabilidad, con versiones de *software* más conservadoras.
 - Otras (Fedora, Arch Linux) priorizan lo último en *software*, con actualizaciones frecuentes.
3. Facilidad de uso e instalación:
 - Distribuciones como Ubuntu están orientadas a usuarios principiantes, con instaladores gráficos y entornos de escritorio listos para usar.
 - Arch Linux o Gentoo requieren conocimientos avanzados, con instalación y configuración manual.
 4. Entorno de escritorio y personalización:
 - Algunas distribuciones vienen con entornos gráficos predeterminados (GNOME, KDE, XFCE).
 - Otras permiten al usuario elegir o instalar el entorno que prefiera.
 5. Propósito y enfoque:
 - Algunas distribuciones se enfocan en servidores (Debian, CentOS), otras en escritorio general (Ubuntu, Linux Mint) y otras en seguridad o pruebas de penetración (Kali Linux, Parrot OS).

En resumen, las diferencias entre distribuciones se centran en gestión de paquetes, frecuencia de actualizaciones, facilidad de uso, entornos gráficos y finalidad del sistema, mientras que el núcleo Linux y las herramientas GNU siguen siendo comunes a todas.

(c) ¿Qué es Debian? Acceder al sitio <https://www.debian.org/> e indicar cuáles son los objetivos del proyecto y una breve cronología del mismo.

Debian es un sistema operativo libre y universal, compuesto por el núcleo Linux y herramientas del proyecto GNU. Es mantenido por una comunidad internacional de desarrolladores y usuarios comprometidos con el *software* libre. Su nombre proviene de la combinación de los nombres de sus creadores: Debra Lynn y Ian Murdock.

Objetivos del proyecto: Según su filosofía, Debian busca crear un sistema operativo libre, disponible para todo el mundo, sin importar su ubicación geográfica, idioma o nivel de conocimiento técnico. El proyecto se basa en principios de apertura, transparencia y colaboración comunitaria. No se enfoca en el beneficio económico, sino en el desarrollo ético y técnico del *software* libre.

Cronología del proyecto Debian:

- Agosto de 1993: Ian Murdock, estudiante de la Universidad de Purdue, inicia el proyecto Debian con el objetivo de crear una distribución de Linux completamente libre.
- 1994: Se publica la versión 0.91, que incluye un sistema de empaquetado básico y una estructura inicial de paquetes.
- 1996: Debian 1.1, conocida como “Buzz”, es lanzada, marcando la primera versión oficial del sistema.
- 1999: Se inicia el soporte para arquitecturas adicionales y se publica la versión 2.0, conocida como “Hamm”.

- 2005: Debian 3.1, conocida como “Sarge”, es lanzada después de un largo período de desarrollo.
- 2015: Se implementa el soporte a largo plazo (LTS) para versiones antiguas, extendiendo su vida útil y seguridad.
- 2025: Debian 13, conocida como “Trixie”, es lanzada, continuando con el compromiso de ofrecer un sistema operativo libre, estable y seguro.

Esta cronología destaca los hitos importantes en el desarrollo y evolución del proyecto Debian, reflejando su crecimiento y consolidación como una de las distribuciones más respetadas y utilizadas en el mundo del *software* libre.

Ejercicio 3: Estructura de GNU/Linux.

(a) *Nombrar cuáles son los 3 componentes fundamentales de GNU/Linux.*

Los tres componentes fundamentales de GNU/Linux son:

1. Núcleo (Kernel):
 - Es el corazón del sistema operativo.
 - Se encarga de gestionar el *hardware*, la memoria, los procesos y la comunicación entre dispositivos y programas.
 - Ejemplos: Linux kernel 6.x, 5.x, etc.
2. Shell o intérprete de comandos:
 - Es la interfaz entre el usuario y el núcleo.
 - Permite ejecutar comandos, *scripts* y programas.
 - Ejemplos: Bash, Zsh, Fish.
3. Sistema de archivos y utilidades GNU:
 - Conjunto de herramientas y programas esenciales para operar el sistema, como compiladores, editores de texto, utilidades de gestión de archivos y bibliotecas.
 - Incluye la estructura jerárquica de directorios, desde la raíz (/) hasta las carpetas de configuración y datos de usuario.

(b) *Mencionar y explicar la estructura básica del Sistema Operativo GNU/Linux.*

El sistema operativo GNU/Linux se organiza en varias capas jerárquicas, que permiten separar las funciones y facilitan la gestión del sistema. La estructura básica es la siguiente:

1. Núcleo (Kernel):
 - Es la capa más interna, responsable de la comunicación entre el *hardware* y el *software*.
 - Gestiona la CPU, memoria, dispositivos de entrada/salida, procesos y control de acceso.
 - Funciona como intermediario entre los programas y los recursos físicos del sistema.
2. Shell o intérprete de comandos:
 - Es la capa intermedia que permite al usuario interactuar con el sistema.
 - Puede ser texto (CLI - *Command Line Interface*) o gráfica (GUI), dependiendo de la configuración.
 - A través del *shell*, se pueden ejecutar comandos, *scripts* y programas, controlar procesos y administrar archivos.
3. Sistema de archivos y utilidades:
 - Incluye las herramientas básicas de GNU y otros programas esenciales (compiladores, editores, utilidades de red, bibliotecas, etc.)
 - Organiza los archivos en una estructura jerárquica que comienza en la raíz (/) y se ramifica en directorios como:
 - */bin*: programas esenciales.

- */etc*: archivos de configuración.
 - */home*: directorios de usuarios.
 - */usr*: programas y utilidades adicionales.
 - */var*: archivos variables (logs, bases de datos).
4. Aplicaciones y entorno de escritorio:
- Son los programas que el usuario final utiliza, como navegadores, procesadores de texto, reproductores de multimedia o entornos gráficos (GNOME, KDE, XFCE).
 - Esta capa depende de las anteriores para funcionar, ya que utiliza los servicios del *Kernel*, el *shell* y las bibliotecas del sistema.

En resumen, GNU/Linux tiene una estructura modular y jerárquica, *Kernel* → *Shell* → Sistema de archivos/utilidades → Aplicaciones, lo que permite seguridad, estabilidad y flexibilidad en la gestión del sistema.

Ejercicio 4: Kernel.

(a) *¿Cuáles son sus funciones principales?*

El *Kernel* es el núcleo del sistema operativo y su función principal es actuar como intermediario entre el *hardware* y el *software*. Sus funciones más importantes son:

1. Gestión de procesos:
 - Controla la creación, la planificación y la terminación de los procesos en ejecución.
 - Implementa la multitarea y asigna tiempo de CPU a cada proceso de manera eficiente.
2. Gestión de memoria:
 - Administra la memoria principal (RAM) y el espacio de intercambio (*swap*).
 - Se encarga de asignar memoria a los procesos y de proteger áreas de memoria para que un proceso no interfiera con otro.
3. Gestión de dispositivos:
 - Controla todos los dispositivos de *hardware* mediante los *drivers*.
 - Permite que los programas accedan a los recursos de forma estandarizada y segura.
4. Gestión del sistema de archivos:
 - Maneja la lectura, la escritura y la organización de los datos en los discos y otros medios de almacenamiento.
 - Garantiza la integridad de la información y permite acceso jerárquico a los archivos.
5. Gestión de la seguridad y permisos:
 - Controla los permisos de usuario y el acceso a recursos, evitando que procesos no autorizados realicen acciones críticas.
 - Implementa mecanismos de protección de memoria y de ejecución de código.
6. Comunicación entre procesos:
 - Facilita la interacción y la sincronización entre procesos mediante señales, tuberías, *sockets* y memoria compartida.

En resumen, el *Kernel* es el cerebro del sistema, responsable de que los procesos, la memoria, el *hardware* y los archivos funcionen de manera coordinada y segura, garantizando la estabilidad y la eficiencia del sistema operativo.

(b) *¿Cuál es la versión actual? ¿Cómo se definía el esquema de versionado del Kernel en versiones anteriores a la 2.4? ¿Qué cambió en el versionado que se impuso a partir de la versión 2.6?*

Versión actual del Kernel: Hasta el 16 de octubre de 2025, la versión estable más reciente del Kernel de Linux es la 6.17.3. Esta versión se lanzó el 15 de octubre de 2025 y es la que se utiliza en distribuciones como Ubuntu 25.10 y Fedora 43.

Esquema de versionado antes de la versión 2.4: Antes de la versión 2.4, el esquema de versionado del *Kernel* de Linux seguía una convención basada en números impares y pares:

- Números impares (por ejemplo, 2.1, 2.3, 2.5): Indicaban versiones en desarrollo, con cambios experimentales y no estables.
- Números pares (por ejemplo, 2.0, 2.2): Representaban versiones estables, listas para su uso general.

Este enfoque permitía a los desarrolladores y usuarios identificar, rápidamente, si una versión era estable o aún estaba en desarrollo.

Cambios en el esquema de versionado a partir de la versión 2.6: Con la introducción de la versión 2.6, el esquema de versionado cambió para reflejar mejor el ciclo de desarrollo y mantenimiento del *Kernel*:

- Número mayor (2): Indicaba la serie principal del *Kernel*.
- Número menor (6): Representaba la versión de desarrollo.
- Número de revisión (x): Denotaba actualizaciones menores o correcciones de errores.
- Número de parche (y): Se añadía para indicar parches específicos o actualizaciones de seguridad.

Este nuevo esquema proporcionaba una estructura más clara y coherente para el seguimiento de las versiones del *Kernel* y facilitaba la gestión de actualizaciones y mantenimiento.

(c) *¿Es posible tener más de un Kernel de GNU/Linux instalado en la misma máquina?*

Sí, es posible tener más de un *Kernel* de GNU/Linux instalado en la misma máquina. Esto se logra porque cada versión del *Kernel* se instala en su propio directorio dentro de */boot* y se registra en el GRUB (el gestor de arranque), que permite seleccionar qué *Kernel* iniciar al arrancar el sistema.

Ventajas de tener múltiples *Kernels*:

- Seguridad ante fallos: Si la versión más reciente del *Kernel* causa problemas, se puede iniciar el sistema con una versión anterior que funcione correctamente.
- Compatibilidad: Permite probar nuevas versiones del *Kernel* sin comprometer la estabilidad del sistema principal.
- Flexibilidad para desarrolladores: Facilita el desarrollo y las pruebas de módulos o *drivers* específicos para distintas versiones del *Kernel*.

En resumen, tener más de un *Kernel* instalado es común y seguro, y permite mantener la estabilidad del sistema mientras se prueban actualizaciones o nuevas características.

(d) *¿Dónde se encuentra ubicado dentro del File System?*

En GNU/Linux, el *Kernel* no es un programa como cualquier otro, es un archivo especial que se carga en memoria al arrancar el sistema. Dentro del *File System*, se encuentra ubicado en el directorio */boot*, que contiene los archivos del *Kernel* instalados en el sistema.

Ejercicio 5: Intérprete de comandos (Shell).

(a) *¿Qué es?*

El *shell* es un programa que actúa como intermediario entre el usuario y el núcleo (*Kernel*) del sistema operativo. Su función principal es interpretar los comandos que ingresa el usuario y comunicarlos al *Kernel* para que se ejecuten.

El *shell* es la interfaz de línea de comandos que permite a los usuarios interactuar con GNU/Linux de manera flexible y poderosa, tanto de forma interactiva como mediante *scripts*.

(b) *¿Cuáles son sus funciones?*

Sus funciones son:

- Interpretar comandos: Permite ejecutar programas, *scripts* o instrucciones del sistema.
- Automatización de tareas: A través de *scripts*, se pueden automatizar procesos repetitivos.
- Gestión de procesos: Permite iniciar, detener o supervisar procesos.
- Interacción con el sistema de archivos: Navegar por directorios, copiar, mover o eliminar archivos.
- Variables y programación básica: Permite usar variables, condicionales y bucles para tareas más complejas.

(c) *Mencionar, al menos, 3 intérpretes de comandos que posee GNU/Linux y compararlos entre ellos.*

GNU/Linux ofrece varios intérpretes de comandos (*shells*), cada uno con características particulares. A continuación, se mencionan 3 de los más utilizados y una breve comparación entre ellos:

1. Bash (Bourne Again Shell):
 - Es el *shell* por defecto en la mayoría de las distribuciones GNU/Linux.
 - Ventajas: compatible con *scripts* del *shell* original (sh); soporta historial de comandos, autocompletado y variables de entorno; muy estable y, ampliamente, documentado.
 - Uso típico: Ideal para administración de sistemas y *scripting* estándar.
2. Zsh (Z Shell):
 - Es una versión más avanzada y configurable que Bash.
 - Ventajas: ofrece autocompletado inteligente, sugerencias en tiempo real y temas visuales (como Oh My Zsh); permite una mayor personalización del *prompt* y funciones avanzadas de historial.

- Desventajas: puede consumir más recursos; requiere configuración inicial para aprovechar sus ventajas.
 - Uso típico: Preferido por usuarios avanzados y desarrolladores.
3. Fish (Friendly Interactive Shell):
- Diseñado para ser fácil de usar y, visualmente, más amigable.
 - Ventajas: autocompletado automático sin configuración adicional; coloreado de sintaxis por defecto; no requiere editar archivos de configuración complejos.
 - Desventajas: no es totalmente compatible con *scripts* de Bash, lo que limita su uso en entornos de producción.
 - Uso típico: Ideal para usuarios nuevos o tareas interactivas.

En resumen:

- Bash: clásico, estable y, ampliamente, usado.
- Zsh: más potente y configurable.
- Fish: moderno y simple para principiantes.

(d) ¿Dónde se ubican (path) los comandos propios y externos al Shell?

En GNU/Linux, los comandos que se ejecutan en el *shell* pueden ser de dos tipos: internos (propios del *shell*) o externos (programas ejecutables del sistema). Cada tipo se encuentra en lugares distintos del sistema de archivos.

1. Comandos internos (propios del shell):
 - Son instrucciones integradas dentro del propio intérprete (por ejemplo, *cd*, *echo*, *pwd*, *export*, *history*).
 - No son archivos ejecutables en el sistema, sino funciones incorporadas en el shell.
 - Ubicación:
 - Están dentro del intérprete de comandos (por ejemplo, dentro de */bin/bash* o */usr/bin/zsh*).
 - No tienen una ruta propia en el sistema de archivos.
2. Comandos externos:
 - Son programas ejecutables almacenados en el sistema de archivos.
 - Cuando el usuario los invoca, el *shell* los busca en los directorios definidos en la variable de entorno *\$PATH*.
 - Ubicaciones comunes:
 - */bin* → comandos básicos (por ejemplo, *ls*, *cp*, *mv*, *cat*).
 - */usr/bin* → aplicaciones de usuario (por ejemplo, *grep*, *nano*, *tar*).
 - */sbin* → comandos administrativos del sistema (por ejemplo, *reboot*, *ifconfig*).
 - */usr/sbin* → herramientas avanzadas para administración del sistema.

En resumen, los comandos internos están incorporados en el *shell*, mientras que los externos son programas independientes ubicados en los directorios del *PATH* del sistema.

(e) ¿Por qué se considera que el Shell no es parte del Kernel de GNU/Linux?

Se considera que el Shell no es parte del *Kernel* de GNU/Linux porque:

- No gestiona *hardware* ni recursos del sistema.
- Opera en modo usuario, no en modo núcleo.
- Puede ser reemplazado sin alterar el sistema operativo.
- Actúa como interfaz entre el usuario y el *Kernel*, no como parte de él.

(f) ¿Es posible definir un intérprete de comandos distinto para cada usuario? ¿Desde dónde se define? ¿Cualquier usuario puede realizar dicha tarea?

Sí, es posible definir un intérprete de comando distinto para cada usuario. En GNU/Linux, cada usuario puede tener asignado un *shell* diferente como su intérprete de comandos predeterminado. Esto significa que un usuario puede usar, por ejemplo, Bash, otro Zsh y otro Fish, según sus preferencias.

El *shell* predeterminado de cada usuario se define en el archivo */etc/passwd*.

Cada usuario puede cambiar su propio *shell*, siempre que:

- El nuevo intérprete esté instalado en el sistema.
- Esté listado en el archivo */etc/shells*, que contiene los *shells* válidos.

Sólo el usuario *root* (administrador) puede:

- Cambiar el *shell* de otros usuarios.
- Agregar nuevos intérpretes de comandos al sistema.

Ejercicio 6: El Sistema de Archivos (*File System*) en Linux.

(a) *¿Qué es?*

El sistema de archivos (*File System*) en Linux es la estructura lógica y jerárquica mediante la cual el sistema operativo organiza, almacena y administra los datos en los dispositivos de almacenamiento (como discos duros, SSD, memorias USB, etc.). En otras palabras, es la forma en que Linux ve y maneja los archivos y los directorios.

Funciones principales del *File System*:

- Organización: Estructura los datos en archivos y directorios dentro de una jerarquía.
- Gestión de acceso: Controla quién puede leer, escribir o ejecutar un archivo mediante permisos.
- Identificación: Cada archivo tiene nombre, ruta y atributos (tamaño, propietario, fecha de modificación, etc.).
- Abstracción del *hardware*: Permite al usuario trabajar con archivos sin preocuparse por cómo se almacenan, físicamente, en el disco.
- Montaje de dispositivos: Linux unifica todos los dispositivos bajo una única estructura de directorios, comenzando desde “/” (la raíz).

(b) *¿Cuál es la estructura básica de los File System en GNU/Linux? Mencionar los directorios más importantes e indicar qué tipo de información se encuentra en ellos. ¿A qué hace referencia la sigla FHS?*

El sistema de archivos (*File System*) de GNU/Linux tiene una estructura jerárquica en forma de árbol invertido, cuyo punto de partida es el directorio raíz /. A partir de él, se ramifican todos los demás directorios, que contienen archivos del sistema, configuraciones, programas y datos de los usuarios. Todo en Linux es un archivo: programas, dispositivos, directorios y procesos se representan como archivos dentro de esta estructura.

Directorios más importantes:

- /: Raíz del sistema de archivos. Contiene todos los demás directorios.
- /bin: Programas básicos del sistema (comandos esenciales como *ls*, *cp*, *mv*, *cat*).
- /boot: Archivos necesarios para el arranque del sistema, como el *Kernel* (*vmlinuz*) y el GRUB.
- /dev: Archivos que representan dispositivos del *hardware* (por ejemplo, */dev/sda* para discos).
- /etc: Archivos de configuración del sistema y de los programas instalados.
- /home: Directorios personales de los usuarios (por ejemplo, */home/jmenduina*).
- /lib: Bibliotecas compartidas necesarias para que funcionen los programas del sistema.

- */media*: Puntos de montaje automático para dispositivos externos (pendrives, CDs).
- */mnt*: Punto de montaje temporal para dispositivos o particiones.
- */opt*: Aplicaciones opcionales instaladas, manualmente, por el usuario.
- */proc*: Archivos virtuales con información del sistema y procesos en ejecución.
- */root*: Directorio personal del usuario administrador (*root*).
- */run*: Información temporal del sistema y procesos activos desde el arranque.
- */sbin*: Programas del sistema usados por el administrador (por ejemplo, *reboot*, *ifconfig*).
- */srv*: Datos de servicios del sistema (por ejemplo, archivos servidos por un servidor *web*).
- */tmp*: Archivos temporales. Se eliminan, automáticamente, al reiniciar.
- */usr*: Contiene la mayoría de los programas y las utilidades de usuario. Incluye */usr/bin*, */usr/lib*, */usr/share*.
- */var*: Archivos variables (logs del sistema, colas de impresión, correos, etc.).

La sigla FHS hace referencia a “*Filesystem Hierarchy Standard*”, es decir, Estándar de Jerarquía del Sistema de Archivos. Es una norma que define la estructura y el propósito de cada directorio en sistemas GNU/Linux y similares a Unix. Su objetivo es garantizar que todas las distribuciones de Linux mantengan una organización coherente y que los programas puedan encontrar archivos y directorios en las mismas rutas, independientemente de la distribución (Debian, Ubuntu, Fedora, etc.).

(c) *Mencionar sistemas de archivos soportados por GNU/Linux.*

GNU/Linux es compatible con una gran cantidad de sistemas de archivos, tanto nativos como de otros sistemas operativos. Algunos de los más importantes son:

- ext2 (Second Extended File System): Fue el sistema estándar en las primeras distribuciones. No posee *journaling* (registro de cambios), lo que lo hace más simple, pero menos seguro ante fallos.
- ext3 (Third Extended File System): Evolución de ext2, agrega *journaling*, mejorando la recuperación ante apagados o errores inesperados.
- ext4 (Fourth Extended File System): Es el más usado actualmente. Ofrece soporte para archivos muy grandes, mejor rendimiento, menor fragmentación y *journaling* más eficiente.
- Btrfs (B-tree File System): Sistema moderno diseñado para reemplazar a ext4. Soporta *snapshots*, compresión, verificación de integridad y administración avanzada de volúmenes.
- XFS: Desarrollado por *Silicon Graphics*, es ideal para sistemas con archivos muy grandes y de alto rendimiento (servidores, bases de datos).
- ReiserFS/Reiser4: Destacado por su eficiencia en manejo de archivos pequeños, aunque, actualmente, está en desuso.

(d) *¿Es posible visualizar particiones del tipo FAT y NTFS (que son de Windows) en GNU/Linux?*

Sí, es posible visualizar particiones del tipo FAT y NTFS (que son de Windows) en GNU/Linux. El *Kernel* de Linux incluye soporte para leer y escribir en varios sistemas de archivos, entre ellos los usados por Windows.

Ejercicio 7: Particiones.

(a) *Definición. Tipos de particiones. Ventajas y desventajas.*

Una partición es una división lógica dentro de un disco duro físico. Cada partición se comporta como una unidad de almacenamiento independiente, lo que permite organizar los datos o instalar varios sistemas operativos en un mismo disco.

Tipos de particiones:

1. Partición primaria:
 - Es la partición principal del disco.
 - Puede haber hasta 4 particiones primarias en un disco (según el esquema MBR).
 - Una de ellas puede marcarse como “activa” para arrancar el sistema operativo.
2. Partición extendida:
 - Es un tipo especial de partición que no almacena datos directamente, sino que contiene particiones lógicas.
 - Sólo puede existir una extendida por disco, pero permite superar el límite de 4 particiones del MBR.
3. Particiones lógicas:
 - Se crean dentro de la partición extendida.
 - Permiten tener más de 4 particiones en total.
 - Se comportan igual que las primarias para el usuario y el sistema operativo.
4. Particiones de arranque, sistema y swap (en Linux):
 - `/boot`: Contiene los archivos necesarios para iniciar el sistema.
 - `/(root)`: Contiene el sistema principal.
 - `swap`: Espacio en disco usado como memoria virtual.

Ventajas de particionar un disco:

- Permite instalar varios sistemas operativos en el mismo equipo (por ejemplo, Windows y Linux).
- Facilita organizar los datos (sistema, usuarios, *backups*, etc.).
- Mejora la seguridad y recuperación de datos: si una partición se daña, las otras pueden mantenerse intactas.
- Posibilita distintos sistemas de archivos en un mismo disco (ext4, NTFS, FAT32, etc.).

Desventajas de particionar un disco:

- Una partición puede quedarse sin espacio mientras otra tiene de sobra.
- La modificación del tamaño de las particiones puede ser riesgosa si no se realiza correctamente.
- Una mala configuración de particiones puede impedir el arranque del sistema.

(b) *¿Cómo se identifican las particiones en GNU/Linux? (Considerar discos IDE, SCSI y SATA).*

En GNU/Linux, las particiones se identifican mediante nombres asignados a los dispositivos de bloque que representan los discos y sus divisiones. Cada disco y partición se muestra como un archivo dentro del directorio `/dev`.

- Discos IDE (antiguos): Se identifican con el prefijo *hd* (*hard disk*).
- Discos SCSI y SATA (actuales): Se identifican con el prefijo *sd* (*SCSI disk*).

(c) *¿Cuántas particiones son necesarias, como mínimo, para instalar GNU/Linux? Nombrarlas, indicando tipo de partición, identificación, tipo de File System y punto de montaje.*

Como mínimo, para instalar GNU/Linux, son necesarias dos particiones: una para el sistema principal (raíz `/`) y otra para la memoria de intercambio (*swap*). Aunque es posible instalar todo en una sola partición (usando un archivo de intercambio en lugar de *swap*), la práctica recomendada es usar dos particiones separadas.

1. Partición raíz (`/`):
 - Tipo de partición: primaria (o lógica).
 - Identificación típica: `/dev/sda1`.
 - Tipo de *File System*: *ext4* (el más común, aunque puede ser *btrfs*, *xfs*, etc.).
 - Punto de montaje: `/`.
2. Partición de intercambio (*swap*):
 - Tipo de partición: primaria (o lógica).
 - Identificación típica: `/dev/sda2`.
 - Tipo de *File System*: *swap*.
 - Punto de montaje: no tiene; el sistema la utiliza, directamente, como memoria virtual.

(d) *Dar ejemplos de diversos casos de particionamiento, dependiendo del tipo de tarea que se deba realizar en el sistema operativo.*

El esquema de particiones en GNU/Linux puede variar según el tipo de usuario o servidor, ya que diferentes tareas requieren diferentes niveles de organización, seguridad y rendimiento. Algunos casos típicos son:

- Instalación básica o de escritorio personal: Ideal para usuarios comunes o principiantes.
- Servidor web o de bases de datos: En servidores, se recomienda separar directorios críticos por seguridad, rendimiento y administración.
- Estación de trabajo o entorno de desarrollo: Pensado para programadores o usuarios avanzados que compilan *software*.

- Servidor de archivos o NAS: Pensado para almacenamiento masivo de datos.

(e) ¿Qué tipo de software para particionar existe? Mencionarlos y comparar.

El *software* de particionado permite crear, modificar, redimensionar, eliminar o formatear particiones en un disco. Estos programas pueden ejecutarse desde el propio sistema operativo o desde un medio externo (por ejemplo, un *Live CD/USB*).

Herramientas de particionado en GNU/Linux:

- *fdisk*: Herramienta clásica para gestionar particiones en discos MBR. Permite crear, borrar y listar particiones. No soporta GPT.
- *parted*: Soporta discos con tablas de partición MBR y GPT. Permite redimensionar particiones y modificar el esquema del disco sin reiniciar.
- *cddisk*: Interfaz más amigable que *fdisk*. Ideal para usuarios que prefieren una vista de menú en consola.
- *gparted*: Versión con interfaz visual de *parted*. Permite mover, redimensionar y crear particiones fácilmente. Incluye soporte para FAT, NTFS, ext4, etc.
- *KDE Partition Manager*: Similar a *GParted*, pero integrado en entornos KDE (como Kubuntu).

Herramientas de particionado en otros sistemas operativos:

- *Disk Management* (Administración de discos): Permite crear y formatear particiones básicas (NTFS, FAT32). Limitado para operaciones avanzadas.
- *Disk Utility* (Utilidad de discos): Permite gestionar volúmenes HFS+, APFS y FAT. Posee funciones de reparación y formateo, pero con poca compatibilidad fuera de macOS.
- *EaseUS Partition Master/AOMEI/MiniTool*: Programas de terceros, con interfaz gráfica avanzada. Permiten redimensionar particiones sin pérdida de datos, clonar discos, etc.

Ejercicio 8: Arranque (*Bootstrap*) de un Sistema Operativo.

(a) ¿Qué es el BIOS? ¿Qué tarea realiza?

BIOS significa *Basic Input/Output System* (Sistema Básico de Entrada/Salida). Es un *firmware* (*software* grabado en un chip de la placa madre) que se ejecuta al encender la computadora.

El BIOS se encarga de inicializar y probar el *hardware* antes de que arranque el sistema operativo. Esta fase se conoce como POST (*Power-On Self Test*). Durante el POST, el BIOS: verifica la memoria RAM; detecta dispositivos conectados (discos, teclado, USB, tarjetas de video, etc.); inicializa controladores básicos del *hardware*.

Las tareas principales del BIOS son:

- Autotest del hardware (POST): Comprueba que todos los componentes esenciales funcionen correctamente.
- Detección de dispositivos de arranque: Busca discos duros, CD/DVD, USB u otros medios desde donde pueda cargar el sistema operativo.
- Cargar el bootstrap/bootloader: Una vez localizado el dispositivo de arranque, el BIOS carga el primer sector (MBR) del disco en la memoria y transfiere el control al *bootloader* (por ejemplo, GRUB en Linux).
- Proporcionar servicios básicos de E/S: Permite que el sistema operativo y los programas interactúen con dispositivos básicos (teclado, pantalla, discos) antes de que existan controladores propios del sistema operativo.

(b) ¿Qué es UEFI? ¿Cuál es su función?

UEFI significa *Unified Extensible Firmware Interface* (Interfaz de *Firmware* Extensible Unificada). Es un reemplazo moderno del BIOS, que proporciona una interfaz más avanzada entre el *firmware* de la placa madre y el sistema operativo. UEFI no sólo inicializa el *hardware*, sino que también soporta discos grandes, arranque seguro y un entorno más flexible que el BIOS clásico.

El UEFI se encarga de:

1. Inicializar y probar el *hardware* al encender la computadora, igual que el BIOS (POST).
2. Detectar y gestionar dispositivos de arranque (discos, USB, red, CD/DVD).
3. Cargar el *bootloader* o el cargador de arranque del sistema operativo.
4. Proporcionar funciones avanzadas de *firmware*, como:
 - Arranque seguro (*Secure Boot*): Garantiza que sólo se ejecute *software* firmado y confiable.
 - Soporte para discos grandes (> 2 TB) mediante GPT (*GUID Partition Table*).
 - Interfaz gráfica y compatibilidad con *mouse*.
 - Capacidad de ejecutar aplicaciones pequeñas antes de cargar el sistema operativo.

(c) ¿Qué es el MBR? ¿Qué es el MBC?

MBR significa *Master Boot Record* (Registro de Arranque Maestro). Es el primer sector de un disco duro (sector 0, generalmente 512 bytes). Contiene información esencial para arrancar el sistema operativo y la tabla de particiones del disco.

MBC significa *Master Boot Code* (Código de Arranque Maestro). Es la parte del MBR que contiene el código ejecutable, es decir, el programa que inicia el arranque del sistema operativo. Cuando el BIOS termina el POST y transfiere el control al MBR, el MBC se ejecuta, decide qué partición es arrancable y carga el *bootloader* de esa partición.

(d) ¿A qué hacen referencia las siglas GPT? ¿Qué sustituye? Indicar cuál es su formato.

GPT significa *GUID Partition Table* (Tabla de Particiones con Identificador Global Único). Es un nuevo esquema de particionamiento que sustituye al MBR en discos modernos. Se utiliza, principalmente, en sistemas con UEFI, aunque también puede usarse en BIOS *Legacy* con compatibilidad.

(e) ¿Cuál es la funcionalidad de un “Gestor de Arranque”? ¿Qué tipos existen? ¿Dónde se instalan? Citar gestores de arranque conocidos.

Un “Gestor de Arranque” (*Bootloader*) es un programa que se ejecuta al inicio del sistema para cargar el sistema operativo en memoria y pasarle el control al *Kernel*. Su funcionalidad es permitir que el sistema operativo se inicie correctamente y, en muchos casos, elegir entre varios sistemas operativos instalados (*dual boot*). Se instalan en MBR (BIOS) o en *EFI System Partition* (UEFI).

Tipos de gestores de arranque:

- Primera etapa (*Stage 1*): Programa mínimo ubicado en el primer sector del disco (MBR o *EFI partition*). Su tarea principal es localizar el *Stage 2* o el *Kernel*.
- Segunda etapa (*Stage 2*): Programa más completo que muestra menú de arranque, carga módulos y pasa parámetros al *Kernel*.
- Gestores de arranque por BIOS vs. UEFI.

Gestores de arranque conocidos:

- GRUB (*GRand Unified Bootloader*): El más común en GNU/Linux. Soporta *multi-boot* y UEFI/BIOS.
- LILO (*Linux Loader*): Más antiguo, sólo BIOS, ya en desuso.
- *systemd-boot*: Ligero, para sistemas UEFI, integrado con *systemd*.
- *rEFInd*: Especialmente útil en sistemas UEFI con varios sistemas operativos.

- Windows *Boot Manager*: Arranca Windows y puede integrarse con otros sistemas operativos vía UEFI.

(f) ¿Cuáles son los pasos que se suceden desde que se prende una computadora hasta que el sistema operativo es cargado (proceso de *bootstrap*)?

El *bootstrap* es el proceso mediante el cual la computadora inicia desde cero y carga el sistema operativo en memoria.

1. Encendido y energía:
 - Al presionar el botón de encendido, la fuente de alimentación entrega energía a todos los componentes del sistema.
2. POST (*Power-On Self Test*):
 - Ejecutado por el BIOS o UEFI.
 - Comprueba que RAM, CPU, teclado, tarjetas de video y discos funcionen correctamente.
 - Si hay errores críticos, se detiene y puede emitir *beeps* o mensajes de error.
3. Inicialización del *hardware*:
 - BIOS/UEFI configura los dispositivos básicos y prepara el entorno para cargar el sistema operativo.
4. Búsqueda del dispositivo de arranque:
 - BIOS/UEFI revisa la secuencia de arranque (disco duro, USB, CD/DVD, red).
 - Encuentra un sector de arranque válido (MBR o *EFI System Partition*).
5. Carga del *bootloader* (gestor de arranque):
 - BIOS/UEFI transfiere el control al *bootloader*, que puede estar en MBR (para BIOS) *EFI Partition* (para UEFI).
6. Ejecución del *bootloader*:
 - El *bootloader* muestra un menú de sistemas operativos si hay más de uno.
 - Carga el *Kernel* del sistema operativo en memoria.
 - Pasa parámetros al *Kernel* (por ejemplo, modo de arranque o resolución de pantalla).
7. Inicialización del *Kernel*:
 - El *Kernel* detecta el *hardware*, monta la partición raíz (/) y configura los controladores y servicios esenciales.
8. Carga del sistema de inicio (*init* o *systemd*):
 - El *Kernel* transfiere el control al sistema de inicio, que inicia los servicios y *deamons*.
 - Configura la interfaz gráfica o de consola según el sistema.
9. Login del usuario:
 - Finalmente, se muestra la pantalla de *login* o el *prompt* de terminal, listo para usar el sistema operativo.

(g) Analizar el proceso de arranque en GNU/Linux y describir sus principales pasos.

El arranque de GNU/Linux sigue los pasos generales del *bootstrap*, pero tiene particularidades propias debido a su *Kernel* y sistema de inicio.

(h) *¿Cuáles son los pasos que se suceden en el proceso de parada (shutdown) de GNU/Linux?*

El *shutdown* es el proceso mediante el cual GNU/Linux termina la ejecución del sistema operativo de manera ordenada, asegurando la integridad de los datos y del *hardware*.

Los pasos que se suceden en el proceso de parada (*shutdown*) de GNU/Linux son:

1. Iniciar el proceso de apagado:
 - Se ejecuta un comando como *shutdown -h now* o mediante el menú gráfico.
 - El sistema avisa a todos los usuarios conectados que el sistema se apagará.
2. Notificar a los procesos:
 - El sistema de inicio (*systemd* o *init*) envía señales SIGTERM a todos los procesos en ejecución.
 - Los procesos deben terminar ordenadamente, cerrando archivos abiertos y guardando datos.
3. Finalización de los procesos:
 - Después de un tiempo de espera, el sistema envía señales SIGKILL a los procesos que no respondieron, forzando su cierre.
4. Detener servicios y *deamons*:
 - *systemd* o *init* detienen todos los servicios activos (redes, servidores gráficos, servidores de impresión, cron, etc.).
5. Desmontar sistemas de archivos:
 - Se desmontan todas las particiones montadas, garantizando que no haya pérdida de datos.
 - Se vacían *buffers* de escritura en disco.
6. Sincronizar y apagar *hardware*:
 - Se sincroniza el disco duro para asegurar que todos los datos se hayan escrito.
 - Se apaga el *hardware* (CPU, ventiladores, dispositivos) o se reinicia si se solicitó.

(i) *¿Es posible tener, en una PC, GNU/Linux y otro sistema operativo instalado? Justificar.*

Sí, es posible tener, en una PC, GNU/Linux y otro sistema operativo instalado. Este esquema se conoce como *dual boot* y permite elegir qué sistema operativo iniciar al encenderla computadora.

- Cada sistema operativo necesita su propio entorno de archivos y controladores.
- Con particiones separadas y un gestor de arranque compatible, no hay conflictos entre los sistemas.

- Es una práctica común para usuarios que necesitan Linux para desarrollo y Windows para aplicaciones específicas.

Ejercicio 9: Archivos y Editores.

(a) *¿Cómo se identifican los archivos en GNU/Linux?*

En GNU/Linux, los archivos se identifican, principalmente, por su nombre y ubicación en el sistema de archivos.

(b) *Investigar el funcionamiento de los editores vim, nano y mcedit, y de los comandos cat, more y less.*

Editores:

- *vim*: Editor de texto avanzado en terminal.
- *nano*: Editor de texto sencillo en terminal.
- *mcedit*: Editor de texto incluido en *Midnight Commander* (MC), un gestor de archivos en terminal.

Comandos:

- *cat*: Muestra el contenido completo de un archivo en pantalla.
- *more*: Muestra contenido de un archivo página por página.
- *less*: Similar a *more*, pero permite avanzar y retroceder en el archivo.

(c) *Crear un archivo llamado “prueba.exe” en el directorio personal usando el vim. El mismo debe contener el número de alumno y el nombre.*

```
vim /home/jmenduina/prueba.exe
12345 - Juan Menduiña
:exit
cat /home/jmenduina/prueba.exe
```

(d) *Investigar el funcionamiento del comando file. Probarlo con diferentes archivos. ¿Qué diferencia se nota?*

El comando *file* determina el tipo de un archivo, examinando su contenido (no se fija sólo en la extensión). Lo hace con una base de datos de “*magic numbers*” (patrones binarios, firmas, cabeceras) y reglas, usando la librería *libmagic*. Puede decir si un archivo es texto, *script*, ejecutable ELF, imagen, tar gzip, dispositivo, enlace simbólico, etc.

(e) *Investigar la funcionalidad y los parámetros de los siguientes comandos relacionados con el uso de archivos:*

- *cd*: Cambia el directorio de trabajo actual.
- *mkdir*: Crea directorios.
- *rmdir*: Elimina directorios vacíos.
- *ln*: Crea enlaces (*links*) entre archivos (*hard links* o *symbolic links*).
- *tail*: Muestra las últimas líneas de un archivo.
- *locate*: Busca archivos por nombre usando una base de datos indexada.
- *ls*: Lista el contenido de directorios.
- *pwd*: Imprime el directorio de trabajo actual.
- *cp*: Copia archivos y directorios.
- *mv*: Mueve o renombra archivos/directorios.
- *find*: Busca archivos y directorios en tiempo real recursivamente, con criterios muy flexibles. Es más lento que *locate*, pero preciso y poderoso.

Ver la ayuda/guía de cada comando:

- *man comando*.
- *comando --help*.

Ejercicio 10.

Indicar qué comando es necesario utilizar para realizar cada una de las siguientes acciones. Investigar el funcionamiento y los parámetros más importantes:

(a) *Crear la carpeta ISOCSO.*

```
mkdir /home/jmenduina/ISOCSO
```

(b) *Acceder a la carpeta.*

```
cd /home/jmenduina/ISOCSO
```

(c) *Crear dos archivos con los nombres isocso.txt e isocso.csv.*

```
touch /home/jmenduina/ISOCSO/isocso.txt /home/jmenduina/ISOCSO/isocso.csv  
ls /home/jmenduina/ISOCSO
```

(d) *Listar el contenido del directorio actual.*

```
ls
```

(e) *Visualizar la ruta donde se está situado.*

```
pwd
```

(f) *Buscar todos los archivos en los que su nombre contiene la cadena “iso*”.*

```
find . -type f -name "iso*"
```

(g) *Informar la cantidad de espacio libre en disco.*

```
df -h
```

(h) *Verificar los usuarios conectados al sistema.*

w

(i) *Editar el archivo isocso.txt e ingresar Nombre y Apellido.*

```
echo "Juan Mendiña" > /home/jmendiña/ISOCSO/isocso.txt  
cat /home/jmendiña/ISOCSO/isocso.txt
```

(j) *Mostrar en pantalla las últimas líneas de un archivo.*

```
tail -n 5 /home/jmendiña/ISOCSO/isocso.txt
```

Ejercicio 11.

Investigar funcionamiento, parámetros y ubicación (directorio) de los siguientes comandos:

(a) *man.*

Muestra la página de manual de otros comandos y utilidades (documentación local).

Ubicación típica: */bin/man* o */usr/bin/man*.

(b) *shutdown.*

Inicia el apagado ordenado del sistema o permite programarlo para más tarde; notifica a usuarios y detiene servicios.

Ubicación típica: */sbin/shutdown* o */usr/sbin/shutdown*.

(c) *reboot.*

Reinicia el sistema (sincroniza, detiene servicios y apaga para, luego, volver a arrancar).

Ubicación típica: */sbin/reboot* o */usr/sbin/reboot*.

(d) *halt.*

Detiene (para) el sistema; en muchos sistemas, *halt* deja el *hardware* en estado detenido.

Ubicación típica: */sbin/halt* o */usr/sbin/halt*.

(e) *uname.*

Muestra información del sistema operativo/*Kernel* (nombre, versión, arquitectura).

Ubicación típica: */bin/uname* o */usr/bin/uname*.

(f) *dmesg.*

Muestra el *buffer* de mensajes del *Kernel* (mensajes de arranque y eventos del *Kernel*: detección de *hardware*, *drivers*, errores). Muy útil para diagnóstico *hardware*/arranque.

Ubicación típica: */bin/dmesg* o */usr/bin/dmesg*.

(g) *lspci*.

Lista los dispositivos PCI detectados (tarjetas gráficas, controladoras, etc.). Forma parte del paquete *pciutils*.

Ubicación típica: */usr/bin/lspci*.

(h) *at*.

Programa la ejecución de comandos una sola vez en un momento futuro (*job scheduler* simple). Interactúa con el demonio *atd*.

Ubicación típica: */usr/bin/at*.

(i) *netstat*.

Muestra conexiones de red, tablas de *routing*, estadísticas de interfaces, puertos abiertos y *sockets*. Forma parte del paquete *net-tools* (antiguo).

Ubicación típica: */bin/netstat* o */usr/bin/netstat*.

(j) *head*.

Muestra las primeras líneas de uno o varios archivos. Muy útil para echar un vistazo rápido.

Ubicación típica: */bin/head* o */usr/bin/head*.

(k) *tail*.

Muestra las últimas líneas de un archivo; tiene modo seguimiento para ver nuevas líneas en tiempo real (ideal para logs).

Ubicación típica: */bin/tail* o */usr/bin/tail*.

Ejercicio 12: Procesos.

(a) *¿Qué es un proceso? ¿A qué hacen referencia las siglas PID y PPID? ¿Todos los procesos tienen estos atributos en GNU/Linux? Justificar. Indicar qué otros atributos tiene un proceso.*

Un proceso es un programa en ejecución, es decir, una instancia activa de un programa que está siendo ejecutada por el sistema operativo. Incluye:

- El código del programa (instrucciones ejecutables).
- Los datos del programa (variables, *buffers*, etc.).
- El estado de ejecución, que permite al sistema operativo retomar la ejecución donde se quedó.
- Recursos asignados, como memoria, archivos abiertos, y dispositivos.

En GNU/Linux, cada tarea que el *Kernel* ejecuta se considera un proceso (incluyendo *daemons* y *threads*).

- PID significa *Process ID* y es el identificador único que el sistema operativo asigna a cada proceso.
- PPID significa *Parent Process ID* y es el identificador del proceso padre, es decir, el que creó o generó este proceso.

En GNU/Linux, todos los procesos tienen PID y PPID. Esto es esencial para la jerarquía de procesos:

- PID único para identificar el proceso en el *Kernel*.
- PPID para establecer la relación padre-hijo, lo que permite terminar procesos hijos al finalizar el padre o que los procesos *zombies* sean adoptados por *init/systemd*.

Los procesos *Kernel threads* pueden tener peculiaridades en su PPID, pero, incluso ellos, tienen un PID para la gestión por el *Kernel*.

Además de PID y PPID, un proceso tiene otros atributos:

- UID/GID: Identificador de usuario y grupo propietario.
- Estado del proceso: *Running, Sleeping, Stopped, Zombie*.
- Prioridad/Nice Value: Controla el tiempo de CPU asignado.
- Memoria asignada: Tamaño de *stack, heap*, código y memoria residente.
- Archivos abiertos: Lista de descriptores de archivos y dispositivos asociados.
- Señales pendientes/máscara de señales: Para comunicación entre procesos.
- Tiempo de CPU usado: Usuario y sistema.
- Comando ejecutado: Nombre del programa o ruta.

(b) *Investigar funcionamiento, parámetros y ubicación (directorio) de los siguientes comandos relacionados a procesos. En caso de que algún comando no venga por defecto en la distribución que se utiliza, se deberá proceder a instalarlo:*

- *top*: Muestra, en tiempo real, los procesos activos y el uso de recursos del sistema (CPU, memoria, etc.). Ubicación típica: */usr/bin/top*.
- *htop*: Versión más amigable de *top* con interfaz colorida e interactiva, permite usar el cursor y menú para matar procesos, cambiar prioridad, etc. Ubicación típica: */usr/bin/htop*.
- *ps*: Muestra un *snapshot* de los procesos activos (no en tiempo real). Ubicación típica: */bin/ps* o */usr/bin/ps*.
- *pstree*: Muestra la jerarquía de procesos en forma de árbol, indicando padres e hijos. Ubicación típica: */usr/bin/pstree*.
- *kill*: Envía señales a un proceso (por defecto, SIGTERM para terminarlo). Ubicación típica: */bin/kill* o */usr/bin/kill*.
- *pgrep*: Busca procesos por nombre o patrón, devuelve PID(s). Ubicación típica: */usr/bin/pgrep*.
- *pkill*: Envía señales a procesos por nombre (como *kill*, pero buscando por nombre). Ubicación típica: */usr/bin/pkill*.
- *killall*: Envía señal a todos los procesos con un nombre exacto. Similar a *pkill*. Ubicación típica: */usr/bin/killall*.
- *renice*: Cambia la prioridad de ejecución (*nice value*) de un proceso activo. Ubicación típica: */usr/bin/renice*.
- *xkill*: Permite cerrar ventanas gráficas con el *mouse* (interfaz X11). Ubicación típica: */usr/bin/xkill*.
- *atop*: Monitor de procesos y recursos en tiempo real y más detallado que *top*. Puede registrar histórico de uso. Ubicación típica: */usr/bin/atop*.
- *nice*: Ejecuta un comando con un valor de prioridad distinto (*nice value*). Ubicación típica: */usr/bin/nice*.

Ver la ayuda/guía de cada comando:

- *man comando*.
- *comando --help*.

Ejercicio 13: Proceso de Arranque de SystemV (<https://github.com/systeminit/si/>).

(a) Enumerar los pasos del proceso de inicio de un sistema GNU/Linux, desde que se prende la PC hasta que se logra obtener el login en el sistema.

A continuación, se enumeran los pasos del proceso de inicio de un sistema GNU/Linux:

1. POST (Power-On Self Test):
 - Cuando se prende la PC, la BIOS (o UEFI en sistemas modernos) realiza un POST, verificando *hardware* básico (memoria RAM, teclado, CPU, tarjetas de video, discos, etc.).
 - Se inicializan los dispositivos esenciales y se preparan para que el sistema operativo arranque.
2. Localización del gestor de arranque:
 - La BIOS busca el Master Boot Record (MBR) del primer disco de arranque.
 - En sistemas con UEFI, busca el EFI *System Partition* (ESP) para localizar un *bootloader* compatible.
 - El gestor de arranque (*bootloader*), como GRUB, se carga en memoria y se ejecuta.
3. Carga del Kernel:
 - El *bootloader* muestra un menú (opcional) y carga el *Kernel* de Linux en memoria junto con la *initramfs* (*filesystem* temporal).
 - La *initramfs* contiene los controladores esenciales y los *scripts* necesarios para montar el sistema de archivos raíz.
4. Inicialización del Kernel:
 - El *Kernel* inicializa gestión de memoria (RAM), dispositivos y controladores y subsistemas del *Kernel* (*scheduler*, manejo de permisos, sistemas de archivos, etc.).
5. Ejecución del proceso *init*:
 - Una vez que el *Kernel* está listo, ejecuta */sbin/init*, que es el primer proceso usuario (PID= 1).
 - En *SystemV*, *init* es responsable de configurar la consola e iniciar *runlevel* correspondiente.
6. Runlevels (niveles de ejecución):
 - *SystemV* organiza servicios en *runlevels*.
 - Según el *runlevel* configurado, *init* ejecuta los *scripts* de inicio en */etc/rc.d/* o */etc/rc#.d/*.
7. Inicialización de servicios:
 - Se ejecutan servicios esenciales:
 - Red (*networking*).
 - *Daemons* del sistema (*cron*, *syslog*, etc.).
 - Montaje de sistemas de archivos adicionales (*/home*, */var*, etc.).
 - Cualquier servicio configurado para ese *runlevel*.
8. Login:
 - Finalmente, se lanza el *getty* en las terminales virtuales (TTY) o el *display manager* (si es modo gráfico).

- El usuario ve la pantalla de *login* y puede autenticarse para empezar a usar el sistema.

(b) Proceso INIT. ¿Quién lo ejecuta? ¿Cuál es su objetivo?

El proceso *INIT* (en sistemas *SystemV*) es el primer proceso que ejecuta el *Kernel* de Linux una vez que termina su propia inicialización. En particular:

- El *Kernel* lo lanza automáticamente.
- Se encuentra en la ruta */sbin/init*.
- Su PID (*Process ID*) siempre es 1, lo que lo convierte en el padre de todos los procesos del sistema.

El objetivo del proceso *init* es inicializar el espacio de usuario (*user space*) del sistema operativo. Esto incluye:

- Ejecutar los *scripts* de arranque definidos en */etc/inittab* y en los directorios */etc/rc.d/* o */etc/init.d/*.
- Montar sistemas de archivos, iniciar servicios básicos (*red*, *syslog*, etc.).
- Cambiar al nivel de ejecución (*runlevel*) configurado por defecto (por ejemplo, modo texto o modo gráfico).
- Iniciar los procesos *getty* que muestran los *prompts* de *login* en las terminales.
- Mantener el control de los procesos del sistema (reiniciar servicios, apagar correctamente, etc.).

(c) RunLevels. ¿Qué son? ¿Cuál es su objetivo?

Los *RunLevels* (niveles de ejecución) son modos de operación predefinidos en los sistemas GNU/Linux basados en *SystemV* (*SysVInit*). Cada *runlevel* representa un estado del sistema que determina qué servicios y procesos deben estar activos. En otras palabras, un *runlevel* indica qué conjunto de procesos el sistema debe iniciar o detener según la fase o el propósito de uso (por ejemplo, modo monousuario, modo multiusuario, modo gráfico, etc.).

El objetivo de los *runlevels* es permitir al administrador controlar el estado operativo del sistema, facilitando tareas como:

- Iniciar el sistema con distintos niveles de servicio.
- Cambiar entre modos (por ejemplo, del modo texto al gráfico).
- Detener o reiniciar el sistema de manera controlada.

(d) ¿A qué hace referencia cada nivel de ejecución según el estándar? ¿Dónde se define qué RunLevel ejecutar al iniciar el sistema operativo? ¿Todas las distribuciones respetan estos estándares?

En los sistemas GNU/Linux basados en *SystemV Init*, los niveles de ejecución (*RunLevels*) representan diferentes modos de operación del sistema. Cada nivel indica qué servicios y procesos deben estar activos:

<i>RunLevel</i>	Nombre/Estado	Descripción
0	<i>Halt</i> (Apagado)	Apaga el sistema. Todos los procesos se detienen
1	<i>Single-user mode</i>	Modo de mantenimiento o recuperación. Sólo un usuario <i>root</i> , sin servicios de red
2	<i>Multi-user</i> (sin red)	Modo multiusuario, pero sin servicios de red
3	<i>Multi-user</i> (con red)	Modo multiusuario completo, en modo texto
4	<i>Undefined/Custom</i>	No se usa por defecto; reservado para configuraciones personalizadas
5	<i>Graphical</i> (X11)	Igual que runlevel 3, pero con entorno gráfico
6	<i>Reboot</i>	Reinicia el sistema

En sistemas con *SysVInit*, el *RunLevel* a ejecutar al iniciar el sistema operativo se define en el archivo */etc/inittab*.

Si bien el estándar *SysVInit* definió esos *runlevels*, no todas las distribuciones modernas lo respetan estrictamente, ya que muchas usan *systemd* como sistema de inicialización. En *systemd*, los *runlevels* fueron reemplazados por “*targets*”, que cumplen la misma función pero con mayor flexibilidad.

(e) Archivo */etc/inittab*. ¿Cuál es su finalidad? ¿Qué tipo de información se almacena en él? ¿Cuál es la estructura de la información que en él se almacena?

El archivo */etc/inittab* pertenece al sistema de inicialización *SystemV Init* (*SysVInit*). Su función principal es definir el comportamiento del proceso *init*, que es el primer proceso del sistema operativo (PID 1). En concreto, este archivo le indica a *init*:

- Qué *runlevel* debe cargarse por defecto.
- Qué procesos o *scripts* ejecutar en cada *runlevel*.
- Qué acciones realizar cuando se producen determinados eventos del sistema (reinicio, apagado, etc.).

Entonces, dentro de */etc/inittab*, se almacena información de configuración como:

- *RunLevel* por defecto (en qué modo arranca el sistema).

- Procesos que deben iniciarse, automáticamente, en cada *runlevel*.
- Configuración de terminales (por ejemplo, consolas *tty1*, *tty2*, etc.).
- Acciones asociadas a eventos como reinicio o apagado.

Cada línea de */etc/inittab* sigue la siguiente estructura: *id:runlevels:acción:proceso*, donde:

- *id*: Identificador único de la entrada.
- *runlevels*: Nivel(es) de ejecución donde se aplica esta entrada (0-6).
- *acción*: Qué tipo de acción debe realizar *init*.
- *proceso*: Comando o *script* que se ejecutará.

(f) Suponer que se encuentra en el *runlevel* <X>. Indicar qué comando(s) se deberá ejecutar para cambiar al *runlevel* <Y>. ¿Este cambio es permanente? ¿Por qué?

En los sistemas GNU/Linux basados en *SystemV Init* (*SysVInit*), el comando que se deberá ejecutar para cambiar al *runlevel* <Y> es *init <Y>*. Este cambio no es permanente, sólo dura mientras el sistema esté encendido; cuando el sistema se reinicia, vuelve al *runlevel* por defecto definido en el archivo */etc/inittab* (o su equivalente moderno en *systemd*).

(g) *Scripts RC*. ¿Cuál es su finalidad? ¿Dónde se almacenan? Cuando un sistema GNU/Linux arranca o se detiene, se ejecutan *scripts*. Indicar cómo determina qué *script* ejecutar ante cada acción. ¿Existe un orden para llamarlos? Justificar.

Los *scripts RC* (*Run Commands*) son *scripts* de inicio y apagado del sistema en GNU/Linux. Su funcionalidad es automatizar la carga y la detención de servicios (como red, impresoras, *daemons*, entorno gráfico, etc.) durante el arranque y el apagado del sistema. Se almacenan en */etc/init.d/*.

Cuando el sistema cambia de *runlevel* (por ejemplo, al iniciar o apagar), el proceso *init*:

- Consulta el nuevo *runlevel* al que debe cambiar.
- Busca en el directorio correspondiente (por ejemplo, */etc/rc3.d/*).
- Ejecuta los *scripts* dentro de ese directorio en orden alfabético.

Cada archivo comienza con una letra:

- S (de *Start*): Indica que el servicio se inicia.
- K (de *Kill*): Indica que el servicio se detiene.

Existe un orden para llamar a estos *scripts*. El orden lo define el número que sigue a la letra S o K y los *scripts* se ejecutan en orden ascendente, lo cual permite controlar las dependencias.

Ejercicio 14: *SystemD* (<https://github.com/systemd/systemd/>).

(a) *¿Qué es SystemD?*

SystemD es un sistema de inicialización y gestión de servicios moderno para sistemas operativos GNU/Linux, que reemplaza al clásico *SystemV Init* (*SysVInit*). Su principal objetivo es inicializar el sistema más rápido y de forma más eficiente, además de gestionar los servicios y los procesos del sistema durante toda su ejecución.

(b) *¿A qué hace referencia el concepto de Unit en SystemD?*

El concepto de *Unit* en *SystemD* hace referencia al componente básico de configuración y control. Cada *unit* representa un recurso del sistema que *SystemD* puede gestionar, supervisar o controlar, como un servicio, un punto de montaje, un dispositivo, un temporizador, etc. En otras palabras, una *unit* le indica a *SystemD* qué debe iniciarse, cuándo, cómo y en qué orden.

(c) *¿Para qué sirve el comando `systemctl` en SystemD?*

En *SystemD*, el comando *systemctl* sirve es la herramienta principal de administración. Permite controlar, inspeccionar y administrar tanto el estado general del sistema como los servicios (*units*) que gestiona *SystemD*.

(d) *¿A qué hace referencia el concepto de target en SystemD?*

En *SystemD*, el concepto de *target* hace referencia a una unidad especial (un tipo de *unit*) que agrupa y coordina el inicio o la detención de otros servicios, *sockets*, dispositivos, montajes, etc. En otras palabras, los *targets* definen estados o etapas del sistema, equivalentes a los niveles de ejecución (*runlevels*) del sistema *SysVInit*, pero con mayor flexibilidad.

(e) *Ejecutar el comando `ps tree`. ¿Qué es lo que se puede observar a partir de la ejecución de este comando?*

Lo que se puede observar, a partir de la ejecución del comando *ps tree*, es que el sistema muestra una vista jerárquica (en forma de árbol) de todos los procesos, actualmente, en ejecución, mostrando qué procesos son padres o hijos de otros. Es muy útil para comprender cómo se estructura el sistema de procesos, detectar relaciones y diagnosticar problemas de ejecución o dependencias entre procesos.

Ejercicio 15: Usuarios.

(a) *¿Qué archivos son utilizados en un sistema GNU/Linux para guardar la información de los usuarios?*

En un sistema GNU/Linux, para guardar la información de los usuarios, los archivos que son utilizados son:

- */etc/passwd*: Contiene la información básica de todas las cuentas de usuario.
- */etc/shadow*: Guarda las contraseñas cifradas y los parámetros de seguridad.
- */etc/group*: Contiene la información de los grupos del sistema.
- */etc/gshadow*: Similar a */etc/shadow*, pero para grupos.

(b) *¿A qué hacen referencia las siglas UID y GID? ¿Pueden coexistir UIDs iguales en un sistema GNU/Linux? Justificar.*

UID significa *User Identifier* (Identificador de Usuario). Es el número entero que el sistema asigna a cada usuario para identificarlo internamente (más allá de su nombre). Se usa en lugar del nombre de usuario para asignar propiedad y permisos sobre archivos, procesos y recursos del sistema.

GID significa *Group Identifier* (Identificador de Grupo). Indica a qué grupo pertenece, por defecto, el usuario. También se usa para determinar los permisos de grupo sobre archivos y procesos.

En un sistema GNU/Linux, pueden coexistir UIDs iguales, pero no es recomendable. El sistema permite crear dos cuentas distintas con el mismo UID. Sin embargo, si esto ocurre, ambos usuarios comparten los mismos permisos y propiedad sobre los archivos (ya que el sistema identifica por UID, no por nombre). En consecuencia, se pierde la separación de privilegios, lo que puede generar problemas de seguridad y administración. Por eso, cada usuario debe tener un UID único, salvo casos muy específicos (por ejemplo, cuentas técnicas que, intencionalmente, comparten permisos).

(c) *¿Qué es el usuario root? ¿Puede existir más de un usuario con este perfil en GNU/Linux? ¿Cuál es el UID de root?*

El usuario *root* es el superusuario del sistema GNU/Linux. Tiene todos los privilegios administrativos, es decir: puede acceder, modificar o eliminar cualquier archivo del sistema; puede crear, modificar o borrar usuarios; puede instalar o eliminar *software*; puede cambiar configuraciones del sistema, *Kernel*, red, etc. Este usuario es esencial para la administración y el mantenimiento del sistema operativo.

En GNU/Linux, no puede existir más de un usuario con este perfil, pero sí es posible tener varios usuarios con privilegios equivalentes al *root*.

El UID de *root* es 0. Este valor es reservado y reconocido por el *Kernel* como el identificador del superusuario.

(d) *Agregar un nuevo usuario llamado isocso a la instalación de GNU/Linux, especificar que su home sea creada en /home/isocso y hacerlo miembro del grupo informatica (si no existe, se deberá crear). Luego, sin iniciar sesión como este usuario, crear un archivo en su home personal que le pertenezca. Luego de todo esto, borrar el usuario y verificar que no queden registros de él en los archivos de información de los usuarios y grupos.*

```
getent group informatica || groupadd informatica
useradd -m -d /home/isocso -g informatica isocso
touch /home/isocso/isocso.txt
userdel -r isocso
grep isocso /etc/passwd
grep isocso /etc/shadow
grep informatica /etc/group
grep informatica /etc/gshadow
```

(e) *Investigar la funcionalidad y los parámetros de los siguientes comandos:*

- *useradd* y *adduser*: Crean un nuevo usuario en el sistema.
- *usermod*: Modifica un usuario existente.
- *userdel*: Elimina un usuario del sistema.
- *su*: Cambia de usuario en la sesión actual.
- *groupadd*: Crea un nuevo grupo.
- *who*: Muestra la información de usuarios conectados al sistema.
- *groupdel*: Elimina un grupo del sistema.
- *passwd*: Cambia la contraseña de un usuario.

Ver la ayuda/guía de cada comando:

- *man comando*.
- *comando --help*.

Ejercicio 16: File System y Permisos.

(a) *¿Cómo son definidos los permisos sobre archivos en un sistema GNU/Linux?*

En un sistema GNU/Linux, los permisos sobre archivos definen quién puede leer, escribir o ejecutar un archivo o directorio. Estos permisos están asociados a tres tipos de usuarios y a tres tipos de acciones.

Tipos de usuarios:

Cada archivo o directorio pertenece a:

- Usuario (u): El propietario del archivo.
- Grupo (g): Un conjunto de usuarios que comparten ciertos permisos.
- Otros (o): Todos los demás usuarios del sistema.

Tipos de permisos:

- Lectura (r):
 - Archivo: Permite leer el contenido del archivo.
 - Directorio: Permite listar el contenido del directorio.
- Escritura (w):
 - Archivo: Permite modificar o borrar el archivo.
 - Directorio: Permite crear, renombrar o eliminar archivos dentro del directorio.
- Ejecución (x):
 - Archivo: Permite ejecutar el archivo (si es un programa o un *script*).
 - Directorio: Permite acceder al directorio.

(b) *Investigar la funcionalidad y los parámetros de los siguientes comandos relacionados con los permisos en GNU/Linux:*

- *chmod*: Cambia los permisos de lectura (r), escritura (w) y ejecución (x) de un archivo o directorio.
- *chown*: Permite cambiar el usuario propietario y/o el grupo al que pertenece un archivo o directorio.
- *chgrp*: Cambia, únicamente, el grupo de un archivo o directorio (sin modificar el propietario).

Ver la ayuda/guía de cada comando:

- *man comando*.
- *comando --help*.

(c) *Al utilizar el comando *chmod*, generalmente, se utiliza una notación octal asociada para definir permisos. ¿Qué significa esto? ¿A qué hace referencia cada valor?*

La notación octal es una forma numérica (en base 8) de representar los permisos de archivos en GNU/Linux. Cada permiso (lectura, escritura, ejecución) se asocia a un valor numérico y la suma de esos valores define los permisos para cada categoría de usuarios.

Valores numéricos de los permisos:

- Lectura (r): 4.
- Escritura (w): 2.
- Ejecución (x): 1.

Tabla resumen de valores comunes:

Valor	Permisos	Significado
0	---	Sin permisos
1	--x	Sólo ejecución
2	-w-	Sólo escritura
3	-wx	Escritura y ejecución
4	r--	Sólo lectura
5	r-x	Lectura y ejecución
6	rw-	Lectura y escritura
7	rwX	Lectura, escritura y ejecución

(d) *¿Existe la posibilidad de que algún usuario del sistema pueda acceder a determinado archivo para el cual no posee permisos? Indicarlo y realizar las pruebas correspondientes.*

No, en principio, no existe la posibilidad de que algún usuario del sistema pueda acceder a determinado archivo para el cual no posee permisos. Sin embargo, el usuario *root* (superusuario) puede acceder, modificar o eliminar cualquier archivo, sin importar sus permisos. Esto se debe a que *root* tiene el UID 0 y el *Kernel* le otorga control total sobre el sistema.

(e) *Explicar los conceptos de “full path name” (path absoluto) y “relative path name” (path relativo). Dar ejemplos claros de cada uno de ellos.*

Full path name (path absoluto) es la ruta completa desde el directorio raíz (/) hasta el archivo o carpeta deseada. Siempre comienza con /, que representa el *root directory*. Ejemplo: */home/jmenduina/informe.txt*.

Relative path name (path relativo) indica la ubicación en relación con el directorio actual (el que muestra *pwd*). No empieza con / y su interpretación depende de dónde se esté ubicado. Ejemplo: Suponiendo que se está en */home/jmenduina* y se quiere acceder a */home/jmenduina/informe.txt*, se puede hacer de forma relativa con *cat informe.txt*.

(f) *¿Con qué comando se puede determinar en qué directorio se encuentra actualmente? ¿Existe alguna forma de ingresar al directorio personal sin necesidad de escribir todo el path completo? ¿Se podría utilizar la misma idea para acceder a otros directorios? ¿Cómo? Explicar con un ejemplo.*

Con el comando *pwd* se puede determinar en qué directorio se encuentra actualmente.

Existen formas de ingresar al directorio personal sin necesidad de escribir todo el *path* completo. En particular, con *cd ~* o, simplemente, *cd*. Se puede utilizar la misma idea para acceder a otros directorios. Ejemplo: Suponiendo que existe un usuario llamado *juan*, se puede acceder a su carpeta personal (si se tienen los permisos) con *cd ~juan*.

(g) *Investigar la funcionalidad y los parámetros de los siguientes comandos relacionados con el uso del File System:*

- *umount*: Desmonta un sistema de archivos previamente montado para que deje de estar accesible desde el sistema.
- *du*: Muestra el espacio ocupado por archivos y directorios.
- *df*: Muestra el espacio disponible y usado en todos los sistemas de archivos montados.
- *mount*: Permite montar un dispositivo (partición, disco, ISO, USB) en un directorio del sistema.
- *mkfs*: Crea un sistema de archivos en una partición o disco.
- *fdisk* (con cuidado): Permite crear, eliminar o modificar particiones en discos duros.
- *write*: Envía mensajes a otro usuario conectado en el mismo sistema.
- *losetup*: Permite tratar un archivo como un dispositivo de bloque (útil para imágenes de disco o ISO).
- *stat*: Muestra información completa de un archivo o directorio (permisos, propietario, tamaño, fechas, inodo, etc.).

Ver la ayuda/guía de cada comando:

- *man comando*.
- *comando --help*.

Ejercicio 17: Procesos.

(a) *¿Qué significa que un proceso se está ejecutando en Background? ¿Y en Foreground?*

Que un proceso se está ejecutando en *Background* significa que se ejecuta “detrás de la terminal”, permitiendo seguir usando la terminal, por lo que no bloquea la entrada de comandos.

Que un proceso se está ejecutando en *Foreground* significa que se ejecuta de manera interactiva en la terminal y, mientras lo hace, bloquea la terminal, es decir, no se pueden ingresar otros comandos hasta que termine.

(b) *¿Cómo se puede hacer para ejecutar un proceso en Background? ¿Cómo se puede hacer para pasar un proceso de background a foreground y viceversa?*

Para ejecutar un proceso en *Background*, simplemente, se agrega `&` al final del comando.

Para pasar un proceso de *background* a *foreground*, se presiona `Ctrl+Z`, lo cual suspende, temporalmente, el proceso y lo pone en estado detenido (*stopped*), y, luego, se envía a *background* con `bg %N`, donde `%N` es el número del *job*.

Para pasar un proceso de *foreground* a *background*, se usa `fg %N`.

(c) *Pipe (|). ¿Cuál es su finalidad? Citar ejemplos de su utilización.*

La finalidad de *Pipe* (|) es conectar la salida estándar de un comando con la entrada estándar de otro. En otras palabras, permite encadenar comandos, de modo que el resultado de uno se use como entrada del siguiente. Su principal objetivo es procesar información en una sola línea sin necesidad de crear archivos intermedios. Permite combinar comandos para filtrar, ordenar, contar o transformar datos.

La sintaxis general es: *comando1 | comando2*. Esto significa “Ejecutá *comando1*, y el resultado pásaselo, directamente, a *comando2*”.

Ejemplos de su utilización:

- `ls -l | grep ".txt"`: Muestra sólo los archivos *.txt* del listado detallado.
- `ps aux | wc -l`: Muestra la cantidad de procesos en ejecución.
- `who | grep root`: Muestra si el usuario *root* tiene una sesión activa.
- `du -ah | sort -rh | head -n 10`: *du* calcula tamaños, *sort* ordena, *head* muestra los primeros 10.

(d) *Redirección. ¿Qué tipo de redirecciones existen? ¿Cuál es su finalidad? Citar ejemplos de su utilización.*

La redirección permite cambiar el destino o el origen de los flujos de datos de un comando. Por defecto:

- Entrada estándar (*stdin*) → viene del teclado.
- Salida estándar (*stdout*) → se muestra en pantalla.
- Salida de error (*stderr*) → también se muestra en pantalla, pero separada del *stdout*.

Con la redirección, se pueden enviar o recibir datos desde archivos u otros comandos.

Tipos de redirecciones:

Tipo	Descriptor	Símbolo	Finalidad
Entrada estándar	0	<	Leer datos desde un archivo en lugar del teclado
Salida estándar	1	>	Enviar la salida normal de un comando a un archivo (sobrescribe o lo crea si no existe)
Salida estándar (<i>append</i>)	1	>>	Agregar la salida al final de un archivo existente (si no existe, lo crea)
Salida de error estándar	2	2>	Redirigir sólo los errores a un archivo (sobrescribe o lo crea si no existe)
Salida de error (<i>append</i>)	2	2>>	Agregar errores al final de un archivo existente (si no existe, lo crea)
Combinar salida y error	1 y 2	&> o 2>&1	Redirigir tanto salida como error al mismo destino

Ejemplos de su utilización:

- *sort < nombres.txt*: Lee los datos del archivo *nombres.txt* y los ordena (sin escribir en disco).
- *ls > listado.txt*: Guarda el resultado del comando *ls* en el archivo *listado.txt* (si existe, lo sobrescribe).
- *echo "Nueva línea" >> archivo.txt*: Agrega texto al final de *archivo.txt* sin borrar su contenido.

- *ls /directorio_que_no_existe 2> errores.txt*: Guarda el mensaje de error en *errores.txt*.
- *ls /home /directorio_que_no_existe &> salida_total.txt*: Guarda tanto la salida como los errores en el mismo archivo.

Ejercicio 18: Otros Comandos de Linux (Indicar Funcionalidad y Parámetros).

(a) ¿A qué hace referencia el concepto de empaquetar archivos en GNU/Linux?

En GNU/Linux, el concepto de empaquetar archivos hace referencia al proceso de unir varios archivos y directorios en un solo contenedor, sin necesariamente comprimirlos. Esto se hace, principalmente, para facilitar su distribución, copia o respaldo.

(b) Seleccionar 4 archivos dentro de algún directorio al que se tenga permiso y sumar el tamaño de cada uno de estos archivos. Crear un archivo empaquetado conteniendo estos 4 archivos y comparar los tamaños de los mismos. ¿Qué característica se nota?

```
cd /home/jmenduina
touch file1.txt file2.txt file3.txt file4.txt
echo "Archivo 1" > file1.txt
echo "Archivo 2" > file2.txt
echo "Archivo 3" > file3.txt
echo "Archivo 4" > file4.txt
ls -l file*.txt
du -ch file*.txt | grep total
tar -cvf archivos.tar file*.txt
ls -l archivos.tar
```

La característica que se nota es que empaquetar no reduce el tamaño de los archivos (de hecho, lo aumenta), sólo los combina en uno solo.

(c) ¿Qué acciones se deben llevar a cabo para comprimir 4 archivos en uno solo? Indicar la secuencia de comandos ejecutados.

Las acciones que se deben llevar a cabo para comprimir 4 archivos en uno solo son:

```
cd /home/jmenduina
touch file1.txt file2.txt file3.txt file4.txt
echo "Archivo 1" > file1.txt
echo "Archivo 2" > file2.txt
echo "Archivo 3" > file3.txt
echo "Archivo 4" > file4.txt
ls -l file*.txt
du -ch file*.txt | grep total
tar -cvf archivos.tar file*.txt
ls -l archivos.tar
tar -czvf archivos.tar.gz file*.txt
ls -l archivos.tar.gz
```

(d) *¿Pueden comprimirse un conjunto de archivos utilizando un único comando?*

Sí, un conjunto de archivos pueden comprimirse utilizando un único comando:

tar -czvf archivos.tar.gz file.txt.*

(e) *Investigar la funcionalidad de los siguientes comandos:*

- *tar*: Sirve para empaquetar (agrupar) varios archivos o directorios en uno solo, sin comprimirlos necesariamente. También puede desempaquetar o extraer archivos desde un paquete *.tar* o *.tar.gz*.
- *grep*: Busca texto o patrones dentro de archivos. Es muy usado para filtrar información en la terminal.
- *gzip*: Comprime archivos individuales usando el algoritmo *gzip* (GNU *zip*). Por defecto, reemplaza el archivo original por su versión comprimida *.gz*.
- *zgrep*: Permite buscar texto dentro de archivos comprimidos con *gzip* (*.gz*) sin tener que descomprimirlos primero. Funciona igual que *grep*, pero sobre archivos *.gz*.
- *wc*: Cuenta líneas, palabras y caracteres de un archivo o entrada estándar.

Ejercicio 19.

Indicar qué acción realiza cada uno de los comandos indicados a continuación, considerando su orden. Suponer que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asumir que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando, indicar la razón:

1. ls -l > prueba:
Lista el contenido detallado del directorio actual y lo guarda en el archivo *prueba* (creándolo o sobrescribiéndolo).
2. ps > PRUEBA:
Guarda la salida del comando *ps* (procesos del usuario actual) en el archivo *PRUEBA*.
3. chmod 710 prueba:
Establece permisos *rwX--X---* (propietario: lectura/escritura/ejecución, grupo: ejecución, otros: ninguno) sobre el archivo *prueba*.
4. chown root:root PRUEBA:
Intenta cambiar propietario y grupo de *PRUEBA* a *root:root*. No se puede ejecutar desde un usuario normal, ya que sólo *root* puede cambiar la propiedad a otro usuario.
5. chmod 777 PRUEBA:
Establece permisos *rwXrwXrwX* (todos pueden leer/escribir/ejecutar) sobre el archivo *PRUEBA*.
6. chmod 700 /etc/passwd:
Intenta establecer permisos *rwX-----* sobre */etc/passwd* (propietario: lectura/escritura/ejecución, grupo: ninguno, otros: ninguno). No se puede ejecutar desde un usuario normal, ya que sólo *root* puede cambiar los permisos del archivo */etc/passwd*.
7. passwd root:
Intenta cambiar la contraseña del usuario *root*. No se puede ejecutar desde un usuario normal, ya que sólo *root* puede cambiar otra cuenta.
8. rm PRUEBA:
Borra el archivo *PRUEBA*.
9. man /etc/shadow:
Intenta localizar la página del manual */etc/shadow*. No se puede ejecutar desde un usuario normal, ya que sólo *root* puede acceder a él.
10. find / -name *.conf:
Busca, recursivamente, desde el directorio raíz (/) archivos cuyo nombre coincida con la máscara **.conf*. Se puede ejecutar desde un usuario normal, pero el usuario no tiene permiso de lectura/ejecución en todos los directorios del sistema, por lo que *find* producirá muchos mensajes "*Permission denied*" en rutas restringidas. Aun así, encontrará y listará los archivos dentro de rutas donde el usuario sí tiene permiso de lectura/ejecución (por ejemplo, dentro de su *home* o áreas públicas).
11. usermod root -d /home/newroot -L:
Intenta modificar la cuenta *root* (nombre y/o directorio *home*). No se puede ejecutar desde un usuario normal, ya que sólo *root* puede cambiar otra cuenta.
12. cd /root:

Intenta cambiar el directorio actual al directorio */root* (*home* de *root*). No se puede ejecutar desde un usuario normal, ya que sólo *root* puede acceder a su directorio.

13. *rm **:

Intenta borrar todos los archivos del directorio actual (según la expansión del ***). Se puede ejecutar desde un usuario normal, para los archivos sobre los que éste tenga permiso de escritura.

14. *cd /etc*:

Cambia el directorio actual al directorio */etc*.

15. *cp */home -R*:

Intenta copiar, recursivamente, todos los elementos del directorio actual al directorio */home*. No se puede ejecutar desde un usuario normal, ya que sólo *root* puede escribir en el directorio */home*.

16. *shutdown*:

Ordena el apagado/reinicio del sistema. No se puede ejecutar desde un usuario normal.

Ejercicio 20.

Indicar qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

(a) *Terminar el proceso con PID 23.*

kill 23

(b) *Terminar el proceso llamado init o systemd. ¿Qué resultados se obtuvieron?*

kill 1

No es posible terminar este proceso, ya que *systemd/init* mantiene el sistema en funcionamiento (arranca servicios, monta discos, gestiona sesiones y coordina el apagado). Matarlo equivaldría a “apagar el corazón del sistema operativo”.

(c) *Buscar todos los archivos de usuarios en los que su nombre contiene la cadena “.conf”.*

find / -name ".conf"*

(d) *Guardar una lista de procesos en ejecución en el archivo **/home/<su nombre de usuario>/procesos**.*

*ps aux > /home/jmenduina/procesos
cat /home/jmenduina/procesos*

(e) *Cambiar los permisos del archivo **/home/<su nombre de usuario>/xxxx** a:*

- *Usuario: Lectura, escritura, ejecución.*
- *Grupo: Lectura, ejecución.*
- *Otros: Ejecución.*

*chmod 751 /home/jmenduina/procesos
ls -l /home/jmenduina/procesos*

(f) *Cambiar los permisos del archivo **/home/<su nombre de usuario>/yyyy** a:*

- *Usuario: Lectura, escritura.*

- *Grupo: Lectura, ejecución.*
- *Otros: Ninguno.*

```
chmod 650 /home/jmenduina/procesos  
ls -l /home/jmenduina/procesos
```

(g) *Borrar todos los archivos del directorio **/tmp**.*

```
rm -f /tmp/*  
ls /tmp
```

(h) *Cambiar el propietario del archivo **/opt/isodata** al usuario **isocso**.*

```
touch /opt/isodata  
chown isocso /opt/isodata  
ls -l /opt/isodata
```

(i) *Guardar, en el archivo **/home/<su nombre de usuario>/donde**, el directorio donde me encuentro en este momento. En caso de que el archivo exista, no se debe eliminar su contenido anterior.*

```
pwd >> /home/jmenduina/directorio  
cat /home/jmenduina/directorio
```

Ejercicio 21.

Indicar qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

(a) *Ingresar al sistema como usuario “root”.*

su -

(b) *Crear un usuario. Elegir, como nombre, por convención, la primera letra del nombre seguida del apellido. Asignarle una contraseña de acceso.*

*useradd -m jmenduina
passwd jmenduina*

(c) *¿Qué archivos fueron modificados luego de crear el usuario y qué directorios se crearon?*

Luego de crear el usuario, los archivos que fueron modificados fueron: */etc/passwd*, */etc/shadow*, */etc/group*, */etc/gshadow*. Y el directorio que se creó fue */home/jmenduina*, con sus archivos ocultos de configuración: */.bash_logout*, */.bashrc* y */.profile*.

(d) *Crear un directorio en /tmp llamado miCursada.*

*mkdir /tmp/miCursada
cd /tmp/miCursada*

(e) *Copiar todos los archivos de /var/log al directorio antes creado.*

cp -r /var/log/ /tmp/miCursada
ls /tmp/miCursada*

(f) *Para el directorio antes creado (y los archivos y subdirectorios contenidos en él), cambiar el propietario y grupo al usuario creado y grupo users.*

*chown -r jmenduina:users /tmp/miCursada
ls -l /tmp/miCursada*

(g) *Agregar permiso total al dueño, de escritura al grupo y escritura y ejecución a todos los demás usuarios para todos los archivos dentro de un directorio en forma recursiva.*

```
chmod -r 723 /tmp/miCursada  
ls -l /tmp/miCursada
```

(h) *Acceder a otra terminal para loguearse con el usuario antes creado.*

```
su jmenduina
```

(i) *Una vez logueado con el usuario antes creado, averiguar cuál es el nombre de la terminal.*

```
tty
```

(j) *Verificar la cantidad de procesos activos que hay en el sistema.*

```
ps aux | wc -l
```

(k) *Verificar la cantidad de usuarios conectados al sistema.*

```
who | wc -l
```

(l) *Volver a la terminal del usuario root y enviarle un mensaje al usuario anteriormente creado, enviándole que el sistema va a ser apagado.*

```
write jmenduina  
El sistema va a ser apagado.
```

(m) *Apagar el sistema.*

```
shutdown -h now
```

Ejercicio 22.

Indicar qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

(a) *Crear un directorio cuyo nombre sea el número de legajo e ingresar a él.*

```
mkdir /home/jmenduina/24981  
cd /home/jmenduina/24981
```

(b) *Crear un archivo utilizando el editor de textos vi e introducir información personal: Nombre, Apellido, Número de alumno y dirección de correo electrónico. El archivo debe llamarse "LEAME".*

```
vi /home/jmenduina/249819/LEAME.txt  
Juan Ignacio Menduiña, 24981/9, menduinajuan@gmail.com  
:exit  
cat /home/jmenduina/249819/LEAME.txt
```

(c) *Cambiar los permisos del archivo "LEAME", de manera que se puedan ver reflejados los siguientes permisos:*

- Dueño: Ningún permiso.
- Grupo: Permiso de ejecución.
- Otros: Todos los permisos.

```
chmod 017 /home/jmenduina/249819/LEAME.txt  
ls -l /home/jmenduina/249819/LEAME.txt
```

(d) *Ir al directorio /etc y verificar su contenido. Crear un archivo dentro del directorio personal cuyo nombre sea "leame", donde el contenido del mismo sea el listado de todos los archivos y directorios contenidos en /etc. ¿Cuál es la razón por la cual se puede crear este archivo si ya existe un archivo llamado "LEAME" en este directorio?*

```
cd /etc  
ls  
ls > /home/jmenduina/249819/leame.txt  
cat /home/jmenduina/249819/leame.txt
```

La razón por la cual se puede crear este archivo "leame.txt" si ya existe un archivo llamado "LEAME.txt" es que ambos nombres son distintos, ya que, en los sistemas Linux (y, en general, en todos los sistemas de archivos *Unix-Like*), los nombres de archivos distinguen entre mayúsculas y minúsculas.

(e) *¿Qué comando se utilizaría y de qué manera si se tuviera que localizar un archivo dentro del file system? ¿Y si se tuvieran que localizar varios archivos con características similares? Explicar el concepto teórico y ejemplificar.*

Si se tuviera que localizar un archivo dentro del *file system*, el comando que se utilizaría sería *find*:

find / -name "archivo.txt", que busca, en todo el sistema (/), un archivo llamado, exactamente, *archivo.txt*.

Y, si se tuvieran que localizar varios archivos con características similares, el comando que se utilizaría también sería *find*:

find / -name ".conf"*, que busca, en todo el sistema (/), todos los archivos terminados en *.conf*.

(f) *Utilizando los conceptos aprendidos en el inciso anterior, buscar todos los archivos cuya extensión sea .so y almacenar el resultado de esta búsqueda en un archivo dentro del directorio creado en el primer inciso. El archivo deberá llamarse ejercicioF.*

find / -type f -name ".so" > /home/jmenduina/249819/ejercicioF*
cat /home/jmenduina/249819/ejercicioF

Ejercicio 23.

Indicar qué acción realiza cada uno de los comandos indicados a continuación, considerando su orden. Suponer que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asumir que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando, indicar la razón:

(a) Iniciar 2 sesiones utilizando nombre de usuario y contraseña. En una sesión, ir siguiendo, paso a paso, las órdenes que se encuentran escritas en el cuadro superior. En la otra sesión, crear, utilizando algún editor de textos, un archivo que se llame “*explicacion_de_ejercicio*” dentro del directorio creado en el ejercicio 22 y, para cada uno de los comandos que se ejecute en la otra sesión, realizar una breve explicación de los resultados obtenidos.

(b) Completar los comandos 19 y 20, de manera tal que realicen la siguiente acción:

- 19: Copiar el directorio *iso* y todo su contenido al directorio creado en el ejercicio 23.a.
- 20: Copiar el resto de los archivos y directorios que se crearon en este ejercicio al directorio creado en el ejercicio 23.a.

(c) Ejecutar las órdenes 19 y 20 y comentarlas en el archivo creado en el inciso (a).

1. *mkdir iso:*
Crea, en el directorio actual (*/home*), un directorio llamado *iso*.
2. *cd ./iso; ps > f0:*
Cambia el directorio actual al directorio */home/iso*. Y, además, guarda la salida del comando *ps* (procesos del usuario actual) en el archivo *f0* (creándolo o sobrescribiéndolo).
3. *ls > f1:*
Lista el contenido del directorio *iso* y lo guarda en el archivo *f1* (creándolo o sobrescribiéndolo).
4. *cd /:*
Cambia el directorio actual al directorio raíz (*/*).
5. *echo \$HOME:*
Muestra la ruta del directorio *home* del usuario actual.
6. *ls -l > \$HOME/iso/ls:*
Lista el contenido detallado del directorio *iso* y lo guarda en el archivo *ls* (creándolo o sobrescribiéndolo).
7. *cd \$HOME; mkdir f2:*
Cambia el directorio actual al directorio *home* del usuario y crea, allí, un directorio llamado *f2*.
8. *ls -ld f2:*
Lista el contenido detallado del directorio *f2*.
9. *chmod 341 f2:*
Establece permisos *-wxr----* (propietario: escritura/ejecución, grupo: escritura, otros: ejecución) sobre el directorio *f2*.
10. *touch dir:*

- Crea un archivo llamado *dir* en el directorio *home*.
11. *cd f2*:
Cambia el directorio actual al directorio *f2*.
 12. *cd ~/iso*:
Cambia el directorio actual al directorio */home/iso*.
 13. *pwd > f3*:
Guarda la ruta completa del directorio actual (*/home/iso*) en el archivo *f3* (creándolo o sobrescribiéndolo).
 14. *ps | grep 'ps' | wc -l >> ../f2/f3*:
Cuenta cuántos procesos *ps* están corriendo y guarda el resultado al final del archivo *f3* ubicado en el directorio */home/f2* (si no existe, lo crea).
 15. *chmod 700 ../f2 ; cd ..*:
Establece permisos *rwX-----* (propietario: lectura/escritura/ejecución, grupo: ninguno, otros: ninguno) sobre el directorio *f2*. Y, además, cambia el directorio actual al directorio (*/home*).
 16. *find . -name etc/passwd*:
Intenta buscar, recursivamente, desde el directorio actual (*/home*), pero el parámetro *-name* en *find* sólo busca por el nombre de un archivo, no por rutas completas.
 17. *find / -name etc/passwd*:
Intenta buscar, recursivamente, desde el directorio raíz (*/*), pero el parámetro *-name* en *find* sólo busca por el nombre de un archivo, no por rutas completas.
 18. *mkdir ejercicio5*:
Crea, en el directorio actual (*/home*), un directorio llamado *ejercicio5*.
 19. *cp -r /home/iso /home/ejercicio5*.
 20. *cp -r /home/f2 /home/dir /home/ejercicio5*.

Ejercicio 24.

Crear una estructura desde el directorio /home que incluya varios directorios, subdirectorios y archivos, según el esquema siguiente:

**Creación de estructura:**

```

mkdir /home/jmenduina/dir1
mkdir /home/jmenduina/dir2
touch /home/jmenduina/f1 /home/jmenduina/f2
mkdir /home/jmenduina/dir1/dir11
touch /home/jmenduina/dir1/f3
touch /home/jmenduina/dir2/f4

```

Asumir que “usuario” indica cuál es el nombre de usuario. Además, se deberá tener en cuenta que dirX hace referencia a directorios y fX hace referencia a archivos. Utilizando la estructura de directorios anteriormente creada, indicar qué comandos son necesarios para realizar las siguientes acciones:

(a) Mover el archivo “f3” al directorio de trabajo /home/usuario.

```

mv /home/jmenduina/dir1/f3 /home/jmenduina
ls /home/jmenduina

```

(b) Copiar el archivo “f4” en el directorio “dir11”.

```

cp /home/jmenduina/dir2/f4 /home/jmenduina/dir1/dir11
ls /home/jmenduina/dir1/dir11

```

(c) Hacer lo mismo que en el inciso anterior, pero el archivo de destino se debe llamar “f7”.

```

cp /home/jmenduina/dir2/f4 /home/jmenduina/dir1/dir11/f7
ls /home/jmenduina/dir1/dir11

```

(d) Crear el directorio “copia” dentro del directorio usuario y copiar, en él, el contenido de “dir1”.

```
mkdir /home/jmenduina/copia
cp -r /home/jmenduina/dir1/* /home/jmenduina/copia
ls /home/jmenduina/copia
```

(e) Renombrar el archivo “f1” por el nombre “archivo” y ver los permisos del mismo.

```
mv /home/jmenduina/f1 /home/jmenduina/archivo
ls -l /home/jmenduina
```

(f) Cambiar los permisos del archivo llamado “archivo”, de manera de reflejar lo siguiente:

- Usuario: Permisos de lectura y escritura.
- Grupo: Permisos de ejecución.
- Otros: Todos los permisos.

```
chmod 617 /home/jmenduina/archivo
ls -l /home/jmenduina/archivo
```

(g) Renombrar los archivos “f3” y “f4” de manera que se llamen “f3.exe” y “f4.exe”, respectivamente.

```
mv /home/jmenduina/f3 /home/jmenduina/f3.exe
mv /home/jmenduina/dir2/f4 /home/jmenduina/dir2/f4.exe
ls /home/jmenduina /home/jmenduina/dir2
```

(h) Utilizando un único comando, cambiar los permisos de los dos archivos renombrados en el inciso anterior, de manera de reflejar lo siguiente:

- Usuario: Ningún permiso.
- Grupo: Permisos de escritura.
- Otros: Permisos de escritura y ejecución.

```
chmod 023 /home/jmenduina/f3.exe /home/jmenduina/dir2/f4.exe
ls -l /home/jmenduina/f3.exe /home/jmenduina/dir2/f4.exe
```

Ejercicio 25.

Indicar qué comando/s es/son necesario/s para realizar cada una de las acciones de la siguiente secuencia de pasos (considerando su orden de aparición):

(a) *Crear un directorio llamado logs en el directorio /tmp.*

```
mkdir /tmp/logs  
cd /tmp/logs
```

(b) *Copiar todo el contenido del directorio /var/log en el directorio creado en el inciso anterior.*

```
cp -r /var/log/* /tmp/logs  
ls /tmp/logs
```

(c) *Empaquetar el directorio creado en (a); el archivo resultante se debe llamar “misLogs.tar”.*

```
tar -cvf /tmp/misLogs.tar -C /tmp logs  
ls /tmp
```

(d) *Empaquetar y comprimir el directorio creado en (a); el archivo resultante se debe llamar “misLogs.tar.gz”.*

```
tar -czvf /tmp/misLogs.tar.gz -C /tmp logs  
ls /tmp
```

(e) *Copiar los archivos creados en (c) y (d) al directorio de trabajo del usuario.*

```
cp /tmp/misLogs.tar /tmp/misLogs.tar.gz /home/jmenduina  
ls /home/jmenduina
```

(f) *Eliminar el directorio creado en (a), logs.*

```
rm -rf /tmp/logs  
ls /tmp/logs
```

(g) *Desempaquetar los archivos creados en (c) y (d) en 2 directorios diferentes.*

```
mkdir /tmp/desempaquetado_tar
mkdir /tmp/desempaquetado_targz
tar -xvf /home/jmenduina/misLogs.tar -C /tmp/desempaquetado_tar
tar -xvzf /home/jmenduina/misLogs.tar.gz -C /tmp/desempaquetado_targz
ls /tmp/desempaquetado_tar /tmp/desempaquetado_targz
```

Trabajo Práctico N° 2:

Definición de SO. Componentes de un SO. Apoyo del *Hardware*: Modos de Ejecución, Interrupciones. Llamadas al Sistema. Programa y Proceso. Planificación de CPU. Colas de Planificación. *Context Switch*. Creación de Procesos. Espacio de Direcciones. Direcciones Lógicas y Físicas. Particiones Fijas y Dinámicas. Paginación y Segmentación. Fragmentación. MMU. Algoritmos de Administración.

Ejercicio 1.

Ejercicio 2.

Ejercicio 3.

Ejercicio 4.

Ejercicio 5.

Ejercicio 6.

Ejercicio 7.

Ejercicio 8.

Ejercicio 9.

Ejercicio 10.

Ejercicio 11.

Ejercicio 12.

Ejercicio 13.

Ejercicio 14.

Ejercicio 15.

Ejercicio 16.

Ejercicio 17.

Ejercicio 18.

Ejercicio 19.

Ejercicio 20.

Ejercicio 21.

Ejercicio 22.

Ejercicio 23.

Ejercicio 24.

Ejercicio 25.

Ejercicio 26.

Ejercicio 27.

Ejercicio 28.

Ejercicio 29.

Ejercicio 30.

Ejercicio 31.

Ejercicio 32.

Ejercicio 33.

Ejercicio 34.

Ejercicio 35.

Ejercicio 36.

Ejercicio 37.

Ejercicio 38.

Trabajo Práctico N° 3: **Bash. Script. Sintaxis. GNU/Linux.**

Ejercicio 1.

¿Qué es el Shell Scripting? ¿A qué tipos de tareas están orientados los script? ¿Los scripts deben compilarse? ¿Por qué?

El *Shell Scripting* es el proceso de escribir programas o secuencias de comandos (*scripts*) que se ejecutan en un intérprete de comandos del sistema operativo, como Bash, Zsh, Sh, Ksh, etc. Un *shell script* es, básicamente, un archivo de texto con instrucciones que el sistema interpreta línea por línea, igual que si se escribieran, manualmente, en la terminal. Generalmente, su extensión es `.sh`.

Los *scripts* de *shell* están orientados, principalmente, a tareas repetitivas o administrativas en sistemas GNU/Linux (aunque también en Windows con *PowerShell*).

Los *scripts* no deben compilarse, se interpretan. Esto significa que no se traducen a código máquina antes de ejecutarse, como ocurre con los programas en C o Java. En cambio, el intérprete del *shell* lee y ejecuta cada línea del *script*, directamente, en tiempo de ejecución.

No se compilan porque el *shell* es un intérprete, no un compilador. Su diseño busca flexibilidad y rapidez en la escritura y la ejecución, no velocidad de ejecución máxima. La idea es poder modificar y probar *scripts* fácilmente, sin necesidad de un proceso de compilación intermedio.

Ejercicio 2.

(a) *Investigar la funcionalidad de los comandos echo y read.*

El comando *echo* muestra texto o el valor de variables por pantalla (en la terminal). Es útil para dar mensajes al usuario o mostrar resultados. Ejemplo:

```
echo "Hola mundo"  
echo "Tu nombre es $nombre"
```

El comando *read* permite leer datos ingresados por el usuario desde el teclado y guardarlos en una variable. Se usa para hacer *scripts* interactivos. Ejemplo:

```
echo "Ingresar nombre:"  
read nombre  
echo "Hola $nombre, ¡bienvenido!"
```

(b) *¿Cómo se indican los comentarios dentro de un script?*

Los comentarios en *shell script* se indican con el símbolo #. Todo lo que sigue después del # en esa línea no se ejecuta. Ejemplo:

```
# Éste es un comentario  
echo "Hola mundo" # Esto también es un comentario al final de una línea
```

(c) *¿Cómo se declaran y se hace referencia a variables dentro de un script?*

Las variables se declaran sin espacios entre el nombre, el signo = y el valor. Ejemplo:

```
nombre="Juan"  
edad=25
```

Para usar o mostrar el valor de una variable, se antepone el signo \$ al nombre de la variable. Ejemplo:

```
echo "Mi nombre es $nombre y tengo $edad años."
```

Ejercicio 3.

Crear, dentro del directorio personal del usuario logueado, un directorio llamado practica-shell-script y, dentro de él, un archivo llamado mostrar.sh cuyo contenido sea el siguiente:

```
#!/bin/bash
# Comentarios acerca de lo que hace el script
# Siempre comento mis scripts, si no lo hago hoy,
# mañana ya no me acuerdo de lo que quise hacer
echo "Introduzca su nombre y apellido:"
read nombre apellido
echo "Fecha y hora actual:"
date
echo "Su apellido y nombre es:"
echo "$apellido $nombre"
echo "Su usuario es: `whoami`"
echo "Su directorio actual es:"
```

(a) *Asignar, al archivo creado, los permisos necesarios de manera que se pueda ejecutar.*

```
mkdir /home/jmenduina/practica-shell-script
cd /home/jmenduina/practica-shell-script
touch /home/jmenduina/practica-shell-script/mostrar.sh
ls -l /home/jmenduina/practica-shell-script

nano /home/jmenduina/practica-shell-script/mostrar.sh

chmod +x /home/jmenduina/practica-shell-script/mostrar.sh
ls -l /home/jmenduina/practica-shell-script
```

(b) *Ejecutar el archivo creado de la siguiente manera: ./mostrar.sh.*

```
./mostrar.sh
```

(c) *¿Qué resultado se visualiza?*

El resultado que se visualiza es:

```
Introduzca su nombre y apellido:
Juan Menduiña
Fecha y hora actual:
Sun Nov 9 13:32:34 -03 2025
Su apellido y nombre es:
Menduiña Juan
```

Su usuario es: root
Su directorio actual es:

(d) *Las backquotes (') entre el comando whoami ilustran el uso de la sustitución de comandos. ¿Qué significa esto?*

La sustitución de comandos en *shell* significa que el resultado (salida) de un comando se reemplaza, directamente, en la línea donde aparece. El *shell* ejecuta el comando que está entre las comillas invertidas o dentro de \$(), toma su salida y la sustituye en el lugar donde estaba el comando.

(e) *Realizar modificaciones al script anteriormente creado, de manera de poder mostrar distintos resultados (cuál es su directorio personal, el contenido de un directorio en particular, el espacio libre en disco, etc.). Pedir que se introduzcan por teclado (entrada estándar) otros datos.*

Sólo agrego lo siguiente:

```
echo "Su directorio actual es:"  
pwd
```

Ahora, el resultado que se visualiza es:

```
Introduzca su nombre y apellido:  
Juan Menduiña  
Fecha y hora actual:  
Sun Nov 9 13:32:34 -03 2025  
Su apellido y nombre es:  
Menduiña Juan  
Su usuario es: root  
Su directorio actual es:  
/home/jmenduina/practica-shell-script
```

Ejercicio 4.

Parametrización: ¿Cómo se acceden a los parámetros enviados al script al momento de su invocación? ¿Qué información contienen las variables \$#, \$, \$? y \$HOME dentro de un script?*

Cuando se ejecuta un *script* en Bash, se le pueden pasar parámetros o argumentos al invocarlo desde la línea de comandos, separados por espacios: `./mi_script.sh juan menduiña 30`. A los parámetros enviados al *script* al momento de su invocación se accede mediante variables especiales numeradas: `$0` (nombre del *script*), `$1` (primer parámetro), `$2` (segundo parámetro), ...

Existen las siguientes variables especiales en Bash:

- `$#`: Cantidad de parámetros pasados al *script*.
- `$*`: Todos los parámetros en una sola cadena (separados por espacios).
- `$?`: Código de salida del último comando ejecutado (0= éxito, distinto de 0= error).
- `$HOME`: Directorio personal del usuario actual.

Ejercicio 5.

¿Cuál es la funcionalidad del comando `exit`? ¿Qué valores recibe como parámetro y cuál es su significado?

La funcionalidad del comando *exit* es detener, inmediatamente, la ejecución del *script* y devuelve un valor numérico (el código de salida o *exit status*) al sistema.

El valor que recibe como parámetro *n* opcional numérico puede ser entre 0 y 255, que representa el estado de finalización del *script* (0= éxito, distinto de 0= error). Si no se especifica ningún número, Bash usa el código de salida del último comando ejecutado.

Ejercicio 6.

El comando `expr` permite la evaluación de expresiones. Su sintaxis es: `expr arg1 op arg2`, donde `arg1` y `arg2` representan argumentos y `op` la operación de la expresión. Investigar qué tipo de operaciones se pueden utilizar.

El comando `expr` (abreviatura de *expression*) sirve para evaluar expresiones y mostrar su resultado por pantalla. Se usa, principalmente, en *scripts* antiguos o cuando no se dispone de la expansión aritmética `$(( ))`.

El tipo de operaciones que se pueden utilizar son:

1. Operaciones aritméticas: + (suma), - (resta), * (multiplicación), / (división entera), % (módulo).
2. Operaciones relacionales o de comparación: = (igualdad de cadenas), != (desigualdad de cadenas), > (mayor que), < (menor que), >= (mayor o igual que), <= (menor o igual que).
3. Operaciones lógicas: | (OR, o lógico), & (AND, y lógico).

Ejercicio 7.

El comando `test expresion` permite evaluar expresiones y generar un valor de retorno, `true` o `false`. Este comando puede ser reemplazado por el uso de corchetes de la siguiente manera: `[ expresión ]`. Investigar qué tipo de expresiones pueden ser usadas con el comando `test`. Tener en cuenta operaciones para: evaluación de archivos, evaluación de cadenas de caracteres y evaluaciones numéricas.

El comando `test` (y su forma equivalente con corchetes `[ expresión ]`) sirve para evaluar condiciones en un *script*. Su salida es un valor de retorno: 0 (verdadero), 1 (falso). Se usa, normalmente, en estructuras *if*, *while*, etc.

El tipo de expresiones que pueden ser usadas con el comando `test` son:

1. Evaluación de archivos: Sirven para comprobar si un archivo o directorio existe o cumple ciertas condiciones.

Ejemplos:

- `-e archivo` (verdadero si existe el archivo).
- `-f archivo` (verdadero si existe y es un archivo regular).
- `-d archivo` (verdadero si existe y es un directorio).
- `-r archivo` (verdadero si el archivo es legible).
- `-w archivo` (verdadero si el archivo es escribible).
- `-x archivo` (verdadero si el archivo es ejecutable).
- `-s archivo` (verdadero si el archivo no está vacío).
- `archivo1 -nt archivo2` (verdadero si `archivo1` es más nuevo que `archivo2`).
- `archivo1 -ot archivo2` (verdadero si `archivo1` es más viejo que `archivo2`).

2. Evaluación de cadenas de caracteres: Sirven para comparar o verificar cadenas.

Ejemplos:

- `-z cadena` (verdadero si la longitud es cero).
- `-n cadena` (verdadero si la longitud no es cero).
- `cadena1 = cadena2` (verdadero si son iguales).
- `cadena1 != cadena2` (verdadero si son distintas).

3. Evaluaciones numéricas: Sirven para comparar valores enteros (no cadenas).

Ejemplos:

- `-eq` (igual).
- `-ne` (distinto).
- `-gt` (mayor que).
- `-lt` (menor que).
- `-ge` (mayor o igual que).
- `-le` (menor o igual que).

Ejercicio 8.

Estructuras de control. Investigar la sintaxis de las siguientes estructuras de control incluidas en shell scripting:

(a) if.

Se usa para evaluar una condición y ejecutar comandos según sea verdadera o falsa.

```
if [ condición ]; then
    comandos_si_verdadero
elif [ otra_condición ]; then
    comandos_si_se_cumple_elif
else
    comandos_si_falso
fi
```

(b) case.

Se usa para evaluar una variable frente a varios posibles valores (como un *switch* en otros lenguajes).

```
case variable in
    valor1)
        comandos
        ;;
    valor2|valor3)
        otros_comandos
        ;;
    *)
        comandos_por_defecto
        ;;
esac
```

(c) while.

Ejecuta un bloque de comandos mientras se cumpla una condición.

```
while [ condición ]; do
    comandos
done
```

(d) for.

Permite recorrer una lista de elementos o un rango.

```
for variable in lista; do  
  comandos  
done
```

(e) *select*.

Se usa para crear menús interactivos (útil en *scripts* de usuario).

```
select variable in lista; do  
  comandos  
done
```

Ejercicio 9.

¿Qué acciones realizan las sentencias `break` y `continue` dentro de un bucle? ¿Qué parámetros reciben?

Las acciones que realizan las sentencias *break* y *continue* dentro de un bucle y los parámetros que reciben son:

- *break*: El programa sale del ciclo y continúa ejecutando las instrucciones que siguen después del bucle.
- *continue*: Salta a la siguiente iteración del bucle, sin ejecutar las instrucciones restantes de la iteración actual.

Ambas sentencias pueden recibir un parámetro *n* opcional numérico, que indica cuántos niveles de bucles anidados deben afectarse:

- *break*: Sale del bucle actual (por defecto).
- *break n*: Sale de *n* niveles de bucles anidados.
- *continue*: Salta a la siguiente iteración del bucle actual (por defecto).
- *continue n*: Salta a la siguiente iteración del *n*-ésimo bucle exterior.

Ejercicio 10.

¿Qué tipo de variables existen? ¿Es shell script fuertemente tipado? ¿Se pueden definir arreglos? ¿Cómo?

Existen los siguientes tipo de variables:

1. Variables del usuario (o locales): Son las que se definen dentro del *script*. Sólo existen mientras el *script* se está ejecutando.
2. Variables de entorno: Son variables del sistema o del entorno del usuario. Están disponibles para todos los procesos y los programas.

Shell Script no es fuertemente tipado, lo cual significa que todas las variables se manejan como cadenas de texto, aunque contengan números.

Se pueden definir arreglos en Bash (no en todos los *shell* antiguos). Ejemplo: *numeros=(1 2 3 4 5)*.

Ejercicio 11.

¿Pueden definirse funciones dentro de un script? ¿Cómo? ¿Cómo se maneja el pasaje de parámetros de una función a la otra?

Sí, dentro de un *script*, pueden definirse funciones. Hay dos formas válidas de definir funciones:

```
nombre_funcion() {  
    comandos  
}
```

```
function nombre_funcion {  
    comandos  
}
```

El pasaje de parámetros de una función a la otra se maneja igual que en un *script*, se pasan al invocar la función, separados por espacios. Dentro de la función, se accede a ellos con las variables posicionales: *\$1* (primer parámetro), *\$2* (segundo parámetro), ...

Bash no devuelve valores “numéricos” directamente, como otros lenguajes. Hay dos formas de devolver resultados:

1. Usando *echo*: Permite capturar el valor en una variable.
2. Usando *return*: Sólo para códigos de estado 0-255.

Ejercicio 12: Evaluación de Expresiones.

(a) Realizar un script que le solicite al usuario 2 números, los lea de la entrada standard e imprima la multiplicación, suma, resta y cuál es el mayor de los números leídos.

```
#!/bin/bash
# Script: Ejercicio12a.sh
# Uso: ./Ejercicio12a.sh

# Solicitud de números al usuario
echo -n "Introducir primer número: "
read num1
echo -n "Introducir segundo número: "
read num2

# Algoritmo
suma=$((num1 + num2))
resta=$((num1 - num2))
multiplicacion=$((num1 * num2))
if [ $num1 -gt $num2 ]; then
    mayor=$num1
elif [ $num1 -lt $num2 ]; then
    mayor=$num2
else
    mayor="Son iguales"
fi

# Resultados
echo "-----"
echo "RESULTADOS:"
echo " - La suma es: $suma"
echo " - La resta es: $resta"
echo " - La multiplicación es: $multiplicacion"
echo " - El mayor es: $mayor"
echo "-----"
```

(b) Modificar el script creado en el inciso anterior para que los números sean recibidos como parámetros. El script debe controlar que los dos parámetros sean enviados.

```
#!/bin/bash
# Script: Ejercicio12b.sh
# Uso: ./Ejercicio12b.sh <num1> <num2>

# Control de cantidad de parámetros
if [ $# -ne 2 ]; then
    echo "Error: Se deben pasar, exactamente, 2 parámetros"
    echo "Uso: $0 <num1> <num2>"
fi
```



```

    exit 1
fi

# Asignación de parámetros
num1=$1
num2=$2

# Algoritmo
suma=$((num1 + num2))
resta=$((num1 - num2))
multiplicacion=$((num1 * num2))
if [ $num1 -gt $num2 ]; then
    mayor=$num1
elif [ $num1 -lt $num2 ]; then
    mayor=$num2
else
    mayor="Son iguales"
fi

# Resultados
echo "-----"
echo "RESULTADOS:"
echo " - La suma es: $suma"
echo " - La resta es: $resta"
echo " - La multiplicación es: $multiplicacion"
echo " - El mayor es: $mayor"
echo "-----"

```

(c) Realizar una calculadora que ejecute las 4 operaciones básicas: +, -, *, %. Esta calculadora debe funcionar recibiendo la operación y los números como parámetros.

```

#!/bin/bash
# Script: Ejercicio12c.sh
# Uso: ./Ejercicio12c.sh <operación> <num1> <num2>

# Control de cantidad de parámetros
if [ $# -ne 3 ]; then
    echo "Error: Se deben pasar, exactamente, 3 parámetros"
    echo "Uso: $0 <operación> <num1> <num2>"
    echo "Operaciones válidas: + - * %"
    exit 1
fi

# Asignación de parámetros
operacion=$1
num1=$2
num2=$3

```

```
# Validación de números
```

```
if ! [[ $num1 =~ ^-[0-9]+$ && $num2 =~ ^-[0-9]+$ ]]; then  
    echo "Error: Los operandos deben ser números enteros"  
    exit 1  
fi
```

```
# Algoritmo
```

```
case $operacion in
```

```
    "+")
```

```
        resultado=$((num1 + num2))
```

```
        ;;
```

```
    "-")
```

```
        resultado=$((num1 - num2))
```

```
        ;;
```

```
    "*")
```

```
        resultado=$((num1 * num2))
```

```
        ;;
```

```
    "%")
```

```
        if [ $num2 -eq 0 ]; then
```

```
            echo "Error: No se puede dividir por cero"
```

```
            exit 1
```

```
        fi
```

```
        resultado=$((num1 % num2))
```

```
        ;;
```

```
    *)
```

```
        echo "Operación inválida. Se debe usar: + - * %"
```

```
        exit 1
```

```
        ;;
```

```
esac
```

```
# Resultado
```

```
echo "Resultado: $resultado"
```

Ejercicio 13: Uso de las Estructuras de Control.

(a) Realizar un script que visualice por pantalla los números del 1 al 100, así como sus cuadrados.

```
#!/bin/bash
# Script: Ejercicio13a.sh
# Uso: ./Ejercicio13a.sh

# Algoritmo
for ((i = 1; i <= 100; i++)); do
    echo "Número: $i - Cuadrado: $((i * i))"
done
```

(b) Crear un script que muestre 3 opciones al usuario: *Listar*, *DondeEstoy* y *QuienEsta*. Según la opción elegida, se le debe mostrar:

- *Listar*: Lista el contenido del directorio actual.
- *DondeEstoy*: Muestra la ruta del directorio donde me encuentro ubicado.
- *QuienEsta*: Muestra los usuarios conectados al sistema.

```
#!/bin/bash
# Script: Ejercicio13b.sh
# Uso: ./Ejercicio13b.sh

# Selección de opción por parte del usuario
echo "Seleccionar una opción:"
echo "1. Listar"
echo "2. DondeEstoy"
echo "3. QuienEsta"
read -p "Opción: " opcion

# Algoritmo
case $opcion in
    1|Listar|listar)
        echo "Contenido del directorio actual:"
        ls
        ;;
    2|DondeEstoy|dondeestoy)
        echo "Ruta actual:"
        pwd
        ;;
    3|QuienEsta|quienesta)
        echo "Usuarios conectados:"
        who
        ;;
    *)
```

```
    echo "Opción inválida"
;;
esac
```

(c) Crear un script que reciba como parámetro el nombre de un archivo e informe si el mismo existe o no y, en caso afirmativo, indique si es un directorio o un archivo. En caso de que no exista el archivo/directorio, cree un directorio con el nombre recibido como parámetro.

```
#!/bin/bash
# Script: Ejercicio13c.sh
# Uso: ./Ejercicio13c.sh

# Control de cantidad de parámetros
if [ $# -ne 1 ]; then
    echo "Error: Se debe pasar, exactamente, 1 parámetro"
    echo "Uso: $0 <nombre_archivo>"
    exit 1
fi

# Asignación de parámetros
nombre=$1

# Algoritmo
if [ -e "$nombre" ]; then
    if [ -d "$nombre" ]; then
        echo "'$nombre' existe y es un directorio"
    elif [ -f "$nombre" ]; then
        echo "'$nombre' existe y es un archivo"
    else
        echo "'$nombre' existe, pero no es ni directorio ni archivo"
    fi
else
    echo "'$nombre' no existe. Se creará un directorio con ese nombre..."
    mkdir "$nombre"
    echo "Directorio '$nombre' creado correctamente"
fi
```

Ejercicio 14: Renombrando Archivos.

Hacer un script que renombre sólo archivos de un directorio pasado como parámetro, agregándole una CADENA, contemplando las opciones:

- “-a CADENA”: Renombra el fichero concatenando CADENA al final del nombre del archivo.
- “-b CADENA”: Renombra el fichero concatenando CADENA al comienzo del nombre del archivo.

Ejemplos: Si se tienen los siguientes archivos /tmp/a y /tmp/b, al ejecutar ./renombrar /tmp/ -a EJ, se obtendrá, como resultado, /tmp/aEJ y /tmp/bEJ. Y, si se ejecuta ./renombrar /tmp/ -b EJ, el resultado será /tmp/EJa /tmp/EJb.

```
#!/bin/bash
# Script: Ejercicio14.sh
# Uso: ./Ejercicio14.sh

# Control de cantidad de parámetros
if [ $# -ne 3 ]; then
    echo "Error: Se deben pasar, exactamente, 3 parámetros"
    echo "Uso: $0 <directorio> -a|-b <CADENA>"
    exit 1
fi

# Asignación de parámetros
directorio=$1
opcion=$2
cadena=$3

# Verificación de que el directorio exista
if [ ! -d "$directorio" ]; then
    echo "Error: '$directorio' no es un directorio válido"
    exit 1
fi

# Algoritmo
case $opcion in
    -a)
        for archivo in "$directorio"/*; do
            if [ -f "$archivo" ]; then
                nombre=$(basename "$archivo")
                mv "$archivo" "$directorio/${nombre}${cadena}"
            fi
        done
        echo "Archivos renombrados agregando '$cadena' al final"
        ;;
    -b)
        for archivo in "$directorio"/*; do
```

```
    if [ -f "$archivo" ]; then
        nombre=$(basename "$archivo")
        mv "$archivo" "$directorio/${cadena}${nombre}"
    fi
done
echo "Archivos renombrados agregando '$cadena' al comienzo"
;;
*)
echo "Opción inválida. Se debe usar: -a o -b"
exit 1
;;
esac
```

Ejercicio 15.

El comando cut nos permite procesar las líneas de la entrada que reciba (archivo, entrada estándar, resultado de otro comando, etc.) y cortar columnas o campos, siendo posible indicar cuál es el delimitador de las mismas. Investigar los parámetros que puede recibir este comando y citar ejemplos de uso.

El comando *cut* en *Shell* sirve para extraer secciones específicas de cada línea de texto en un archivo o entrada estándar (*stdin*). Se puede pensar como una forma de “recortar columnas” o “campos” de texto según su posición o delimitador.

La sintaxis general es: *cut [opciones] [archivo]*. Si no se indica archivo, *cut* lee de la entrada estándar (por ejemplo, se puede usar con *pipe*).

Los parámetros que puede recibir este comando son:

- *-b LISTA*: Selecciona bytes específicos de cada línea (por posición).
- *-c LISTA*: Selecciona caracteres específicos de cada línea.
- *-f LISTA*: Selecciona campos (*fields*), separados por un delimitador.
- *-d 'DELIM'*: Define el delimitador de campos (por defecto es tabulación).
- *--complement*: Muestra todo menos los bytes/campos/caracteres indicados.
- *-s*: Suprime líneas que no contienen el delimitador (útil con *-f*).

Ejemplos de uso:

- *cut -b 1-3 archivo.txt*: Muestra los primeros 3 bytes (útil para datos binarios o sin acentos).
- *cut -c 1-5 archivo.txt*: Muestra los primeros 5 caracteres de cada línea del archivo.
- *cut -d ',' -f 1 datos.csv*: Muestra sólo la primera columna (nombre).
- *cut -d ',' -f 1 --complement datos.csv*: Muestra todos los campos excepto el primero.

Ejercicio 16.

Realizar un script que reciba como parámetro una extensión y haga un reporte con 2 columnas, el nombre de usuario y la cantidad de archivos que posee con esa extensión. Se debe guardar el resultado en un archivo llamado reporte.txt.

```
#!/bin/bash
# Script: Ejercicio16.sh
# Uso: ./Ejercicio16.sh

# Control de cantidad de parámetros
if [ $# -ne 1 ]; then
    echo "Error: Se debe pasar, exactamente, 1 parámetro"
    echo "Uso: $0 <extension>"
    exit 1
fi

# Asignación de parámetros
extension=$1
salida="reporte.txt"

# Limpieza del archivo de salida
> "$salida"

# Algoritmo
echo "Nombre de usuario | Cantidad de archivos con extensión .$extension" >>
"$salida"
echo "-----" >> "$salida"
for dir in /home/*; do
    if [ -d "$dir" ]; then
        usuario=$(basename "$dir")
        cantidad=$(find "$dir" -type f -name ".*$extension" 2>/dev/null | wc -l)
        echo "$usuario | $cantidad" >> "$salida"
    fi
done
echo "Resultado guardado en $salida"
```


Ejercicio 17.

Escribir un script que, al ejecutarse, imprima en pantalla los nombres de los archivos que se encuentran en el directorio actual, intercambiando minúsculas por mayúsculas, además de eliminar la letra a (mayúscula o minúscula).

Por ejemplo, si, en el directorio actual, están los siguientes archivos:

- *IsO.*
- *pepE.*
- *Maria.*

y se ejecutó ./ejercicio17 , se obtendrá, como resultado:

- *iSo.*
- *PEPe.*
- *mRI.*

Ayuda: Investigar el comando tr.

```
#!/bin/bash
```

```
# Script: Ejercicio17.sh
```

```
# Uso: ./Ejercicio17.sh
```

```
# Algoritmo
```

```
for archivo in *; do
```

```
    if [ -f "$archivo" ]; then
```

```
        nuevo_nombre=$(echo "$archivo" | tr 'a-zA-Z' 'A-Za-z' | tr -d 'aA')
```

```
        echo "$nuevo_nombre"
```

```
    fi
```

```
done
```

Ejercicio 18.

Crear un script que verifique, cada 10 segundos, si un usuario se ha logueado en el sistema (el nombre del usuario será pasado por parámetro). Cuando el usuario, finalmente, se loguee, el programa deberá mostrar el mensaje “Usuario XXX logueado en el sistema” y salir.

```
#!/bin/bash
# Script: Ejercicio18.sh
# Uso: ./Ejercicio18.sh

# Control de cantidad de parámetros
if [ $# -ne 1 ]; then
    echo "Error: Se debe pasar, exactamente, 1 parámetro"
    echo "Uso: $0 <nombre_usuario>"
    exit 1
fi

# Asignación de parámetros
usuario=$1

# Algoritmo
while true; do
    if who | grep -wq "$usuario"; then
        echo "Usuario $usuario logueado en el sistema"
        exit 0
    else
        echo "Esperando que $usuario se loguee..."
        sleep 10
    fi
done
```

Ejercicio 19.

Escribir un Programa de “Menú de Comandos Amigable con el Usuario” llamado menú, el cual, al ser invocado, mostrará un menú con la selección para cada uno de los scripts creados en esta práctica. Las instrucciones de cómo proceder deben mostrarse junto con el menú. El menú deberá iniciarse y permanecer activo hasta que se seleccione “Salir”. Por ejemplo:

MENU DE COMANDOS

03. Ejercicio 3

12. Evaluar Expresiones

13. Probar estructuras de control

...

Ingrese la opción a ejecutar: 03

#!/bin/bash

Script: Ejercicio19.sh

Uso: ./Ejercicio19.sh

Ruta donde están los scripts creados en esta práctica

SCRIPTS_DIR="/home/jmenduina/practica-shell-script"

Función "mostrar_menu"

mostrar_menu() {

clear

echo "=====

echo "MENÚ DE COMANDOS"

echo "=====

echo "3. Ejercicio 3"

echo "12a. Ejercicio 12a"

echo "12b. Ejercicio 12b"

echo "12c. Ejercicio 12c"

echo "13a. Ejercicio 13a"

echo "13b. Ejercicio 13b"

echo "13c. Ejercicio 13c"

echo "17. Ejercicio 17"

echo "19. Ejercicio 19"

echo "20. Ejercicio 20"

echo "21. Ejercicio 21"

echo "22. Ejercicio 22"

echo "23. Ejercicio 23"

echo "24. Ejercicio 24"

echo "25. Ejercicio 25"

echo "26. Ejercicio 26"

echo "27. Ejercicio 27"

echo "28. Ejercicio 28"

echo "29. Ejercicio 29"

echo "30a. Ejercicio 30a"

echo "30b. Ejercicio 30b"

```

echo "31. Ejercicio 31"
echo "99. Salir"
echo "-----"
echo
echo -n "Introducir número de script a ejecutar: "
}

```

```

# Algoritmo
while true; do
    mostrar_menu
    read opcion
    echo
    case "$opcion" in
        3)
            echo "Ejecutando Ejercicio 3..."
            echo
            bash "$SCRIPTS_DIR/ejercicio3.sh"
            ;;
        12a)
            echo "Ejecutando Ejercicio 12a..."
            echo
            bash "$SCRIPTS_DIR/ejercicio12a.sh"
            ;;
        12b)
            echo "Ejecutando Ejercicio 12b..."
            echo
            bash "$SCRIPTS_DIR/ejercicio12b.sh"
            ;;
        12c)
            echo "Ejecutando Ejercicio 12c..."
            echo
            bash "$SCRIPTS_DIR/ejercicio12c.sh"
            ;;
        13a)
            echo "Ejecutando Ejercicio 13a..."
            echo
            bash "$SCRIPTS_DIR/ejercicio13a.sh"
            ;;
        13b)
            echo "Ejecutando Ejercicio 13b..."
            echo
            bash "$SCRIPTS_DIR/ejercicio13b.sh"
            ;;
        13c)
            echo "Ejecutando Ejercicio 13c..."
            echo
            bash "$SCRIPTS_DIR/ejercicio13c.sh"
            ;;
        17)
            echo "Ejecutando Ejercicio 17..."

```

```
    echo
    bash "$SCRIPTS_DIR/ejercicio17.sh"
    ;;
19)
    echo "Ejecutando Ejercicio 19..."
    echo
    bash "$SCRIPTS_DIR/ejercicio19.sh"
    ;;
20)
    echo "Ejecutando Ejercicio 20..."
    echo
    bash "$SCRIPTS_DIR/ejercicio20.sh"
    ;;
21)
    echo "Ejecutando Ejercicio 21..."
    echo
    bash "$SCRIPTS_DIR/ejercicio21.sh"
    ;;
22)
    echo "Ejecutando Ejercicio 22..."
    echo
    bash "$SCRIPTS_DIR/ejercicio22.sh"
    ;;
23)
    echo "Ejecutando Ejercicio 23..."
    echo
    bash "$SCRIPTS_DIR/ejercicio23.sh"
    ;;
24)
    echo "Ejecutando Ejercicio 24..."
    echo
    bash "$SCRIPTS_DIR/ejercicio24.sh"
    ;;
25)
    echo "Ejecutando Ejercicio 25..."
    echo
    bash "$SCRIPTS_DIR/ejercicio25.sh"
    ;;
26)
    echo "Ejecutando Ejercicio 26..."
    echo
    bash "$SCRIPTS_DIR/ejercicio26.sh"
    ;;
27)
    echo "Ejecutando Ejercicio 27..."
    echo
    bash "$SCRIPTS_DIR/ejercicio27.sh"
    ;;
28)
    echo "Ejecutando Ejercicio 28..."
```

```
    echo
    bash "$SCRIPTS_DIR/ejercicio28.sh"
    ;;
29)
    echo "Ejecutando Ejercicio 29..."
    echo
    bash "$SCRIPTS_DIR/ejercicio29.sh"
    ;;
30a)
    echo "Ejecutando Ejercicio 30a..."
    echo
    bash "$SCRIPTS_DIR/ejercicio30a.sh"
    ;;
30b)
    echo "Ejecutando Ejercicio 30b..."
    echo
    bash "$SCRIPTS_DIR/ejercicio30b.sh"
    ;;
31)
    echo "Ejecutando Ejercicio 31..."
    echo
    bash "$SCRIPTS_DIR/ejercicio31.sh"
    ;;
99)
    echo "Saliendo del menú. ¡Hasta luego!"
    echo
    break
    ;;
*)
    echo "Opción inválida. Intentar nuevamente"
    ;;
esac
echo
echo -n "Presionar ENTER para continuar..."
read
done
```

Ejercicio 20.

Realizar un script que simule el comportamiento de una estructura de PILA e implementar las siguientes funciones aplicables sobre una estructura global definida en el script:

- *push*: Recibe un parámetro y lo agrega en la pila.
- *pop*: Saca un elemento de la pila.
- *length*: Devuelve la longitud de la pila.
- *print*: Imprime todos elementos de la pila.

Dentro del mismo script y, utilizando las funciones implementadas:

- Agregar 10 elementos a la pila
- Sacar 3 de ellos.
- Imprimir la longitud de la pila.
- Luego, imprimir la totalidad de los elementos que en ella se encuentran.

```
#!/bin/bash
# Script: Ejercicio20.sh
# Uso: ./Ejercicio20.sh

# ESTRUCTURA
pila=()

# FUNCIONES

# push: Recibe un parámetro y lo agrega a la pila
push() {
    pila+=("$1")
}

# pop: Saca un elemento de la pila
pop() {
    if [ ${#pila[@]} -eq 0 ]; then
        echo "Error: Pila vacía. No se puede hacer pop"
        return 1
    fi
    unset 'pila[-1]'
}

# length: Devuelve la longitud de la pila
length() {
    echo "${#pila[@]}"
}

# print: Imprime todos los elementos de la pila
print() {
    echo "${pila[@]}"
}
```

```
}
```

```
#PRUEBAS
```

```
echo "Agregando 10 elementos a la pila..."
```

```
for i in {1..10}; do
```

```
    push "Elemento_$i"
```

```
done
```

```
echo "Sacando 3 elementos de la pila..."
```

```
for i in {1..3}; do
```

```
    pop
```

```
done
```

```
echo "Longitud de la pila: $(length)"
```

```
echo "Impresión de la totalidad de los elementos en la pila: $(print)"
```


Ejercicio 21.

Dada la siguiente declaración al comienzo de un script:

num=(10 3 5 7 9 3 5 4). (la cantidad de elementos del arreglo puede variar).

Implementar la función productoria dentro de este script, cuya tarea sea multiplicar todos los números que el arreglo contiene.

```
#!/bin/bash
# Script: Ejercicio21.sh
# Uso: ./Ejercicio21.sh

# Vector
num=(10 3 5 7 9 3 5 4)

# Algoritmo
productoria() {
    local prod=1
    for n in "${num[@]}; do
        prod=$((prod * n))
    done
    echo "$prod"
}

# Resultado
echo "La productoria del vector (${num[@]}) es: $(productoria)"
```

Ejercicio 22.

Implementar un script que recorra un arreglo compuesto por números e imprima en pantalla sólo los números pares y que cuente sólo los números impares y los informe en pantalla al finalizar el recorrido.

```
#!/bin/bash
# Script: Ejercicio22.sh
# Uso: ./Ejercicio22.sh

# Vector
num=(10 3 5 7 9 3 5 4)

# Algoritmo
impares=0
for n in "${num[@]}; do
    if (( n % 2 == 0 )); then
        echo "Número par: $n"
    else
        ((impares++))
    fi
done

# Resultado
echo "Cantidad de números impares: $impares"
```

Ejercicio 23.

Dada la definición de 2 vectores del mismo tamaño y cuyas longitudes no se conocen.

vector1=(1 .. N).

vector2=(1.. N).

Por ejemplo:

vector1=(1 80 65 35 2) y vector2=(5 98 3 41 8).

Completar este script de manera tal de implementar la suma elemento a elemento entre ambos vectores y que la misma sea impresa en pantalla de la siguiente manera:

- *La suma de los elementos de la posición 0 de los vectores es 6.*
- *La suma de los elementos de la posición 1 de los vectores es 178.*
- *La suma de los elementos de la posición 4 de los vectores es 10.*

```
#!/bin/bash
```

```
# Script: Ejercicio23.sh
```

```
# Uso: ./Ejercicio23.sh
```

```
# Vectores
```

```
vector1=(1 80 65 35 2)
```

```
vector2=(5 98 3 41 8)
```

```
# Algoritmo
```

```
for ((i=0; i<${#vector1[@]}; i++)); do
```

```
    suma=$(( ${vector1[i]} + ${vector2[i]} )
```

```
    echo "La suma de los elementos de la posición $i de los vectores es $suma"
```

```
done
```

Ejercicio 24.

Realizar un script que agregue, en un arreglo, todos los nombres de los usuarios del sistema pertenecientes al grupo "users". Adicionalmente, el script puede recibir como parámetro:

- "-b n": Retorna el elemento de la posición n del arreglo si el mismo existe. Caso contrario, un mensaje de error.
- "-l": Devuelve la longitud del arreglo.
- "-i": Imprime todos los elementos del arreglo en pantalla.

```
#!/bin/bash
# Script: Ejercicio24.sh
# Uso: ./Ejercicio24.sh

# Grupo
group="users"

# Obtención de todos los nombres de los usuarios del sistema pertenecientes al grupo "$group"
usuarios=$(getent group "$group" | cut -d: -f4 | tr ' ' '\n')

# Verificación de usuarios encontrados
if [ ${#usuarios[@]} -eq 0 ]; then
    echo "No se encontraron usuarios del sistema pertenecientes al grupo '$group'"
    exit 1
fi

# Algoritmo
case "$1" in
    -b)
        n=$2
        if [ -z "$n" ]; then
            echo "Error: Se debe especificar un número después de '-b'"
            exit 1
        elif [ "$n" -ge 0 ] && [ "$n" -lt "${#usuarios[@]}" ]; then
            echo "Elemento en la posición $n: ${usuarios[$n]}"
        else
            echo "Error: No existe la posición $n en el arreglo"
        fi
        ;;
    -l)
        echo "La longitud del arreglo es: ${#usuarios[@]}"
        ;;
    -i)
        echo "Usuarios en el grupo '$group':"
        for u in "${usuarios[@]}; do
            echo "- $u"
        done
    ;;
esac
```

```
;;
*)
echo "Uso: $0 [-b n | -l | -i]"
echo "-b n : Retorna el elemento de la posición n del arreglo si el mismo existe"
echo "-l   : Retorna la longitud del arreglo"
echo "-i   : Imprime todos los elementos del arreglo en pantalla"
;;
esac
```

Ejercicio 25.

Escribir un script que reciba una cantidad desconocida de parámetros al momento de su invocación (debe validar que, al menos, se reciba uno). Cada parámetro representa la ruta absoluta de un archivo o directorio en el sistema. El script deberá iterar por todos los parámetros recibidos y, sólo para aquellos parámetros que se encuentren en posiciones impares (el primero, el tercero, etc.), verificar si el archivo o directorio existen en el sistema, imprimiendo en pantalla qué tipo de objeto es (archivo o directorio). Además, se deberá informar la cantidad de archivos o directorios inexistentes en el sistema.

```
#!/bin/bash
# Script: Ejercicio25.sh
# Uso: ./Ejercicio25.sh

# Control de cantidad de parámetros
if [ $# -lt 1 ]; then
    echo "Error: Se debe pasar, al menos, 1 parámetro"
    echo "Uso: $0 <ruta1> <ruta2> ..."
    exit 1
fi

# Algoritmo
inexistentes=0
posicion=1
for ruta in "$@"; do
    if (( posicion % 2 != 0 )); then
        if [ -e "$ruta" ]; then
            if [ -d "$ruta" ]; then
                echo "Posición $posicion: '$ruta' existe y es un directorio"
            elif [ -f "$ruta" ]; then
                echo "Posición $posicion: '$ruta' existe y es un archivo"
            else
                echo "Posición $posicion: '$ruta' existe, pero no es ni directorio ni archivo"
            fi
        else
            echo "Posición $posicion: '$ruta' no existe"
            ((inexistentes++))
        fi
    fi
    ((posicion++))
done

# Resultado
echo "Cantidad de archivos o directorios inexistentes: $inexistentes"
```

Ejercicio 26.

Realizar un script que implemente, a través de la utilización de funciones, las operaciones básicas sobre arreglos:

- *inicializar*: Crea un arreglo llamado array vacío.
- *agregar_elem* <parametro1>: Agrega al final del arreglo el parámetro recibido.
- *eliminar_elem* <parametro1>: Elimina del arreglo el elemento que se encuentra en la posición recibida como parámetro. Debe validar que se reciba una posición válida.
- *longitud*: Imprime la longitud del arreglo en pantalla.
- *imprimir*: Imprime todos los elementos del arreglo en pantalla.
- *inicializar_con_valores* <parametro1><parametro2>: Crea un arreglo con longitud <parametro1> y, en todas las posiciones, asigna el valor <parametro2>.

```
#!/bin/bash
```

```
# Script: Ejercicio26.sh
```

```
# Uso: ./Ejercicio26.sh
```

```
# FUNCIONES
```

```
# inicializar: Crea un arreglo llamado array vacío
```

```
inicializar() {
    array=()
    echo "Arreglo llamado 'array' creado vacío"
}
```

```
# agregar_elem: Agrega al final del arreglo el parámetro recibido
```

```
agregar_elem() {
    if [ $# -ne 1 ]; then
        echo "Error: Se debe indicar el valor para agregar"
        return 1
    fi
    array+=("$1")
    echo "Elemento '$1' agregado al final del arreglo"
}
```

```
# eliminar_elem: Elimina del arreglo el elemento que se encuentra en la posición recibida como parámetro
```

```
eliminar_elem() {
    if [ $# -ne 1 ]; then
        echo "Error: Se debe indicar la posición a eliminar"
        return 1
    fi
    local pos=$1
    if [ "$pos" -lt 0 ] || [ "$pos" -ge "${#array[@]}" ]; then
        echo "Error: Posición fuera de rango (0 a ${#array[@]} - 1)"
        return 1
    fi
}
```

```

fi
unset 'array[pos]'
array=("${array[@]}")
echo "Elemento en posición $pos eliminado"
}

# longitud: Imprime la longitud del arreglo en pantalla
longitud() {
    echo "Longitud del arreglo: ${#array[@]}"
}

# imprimir: Imprime todos los elementos del arreglo en pantalla
imprimir() {
    if [ ${#array[@]} -eq 0 ]; then
        echo "El arreglo está vacío"
        return 1
    fi
    echo "Elementos del arreglo: ${array[@]}"
}

# inicializar_con_valores: Crea un arreglo con longitud <parametro1> y, en todas las
posiciones, asigna el valor <parametro2>
inicializar_con_valores() {
    if [ $# -ne 2 ]; then
        echo "Error: Se debe indicar longitud y valor"
        return 1
    fi
    local longitud=$1
    local valor=$2
    array=()
    for ((i = 0; i < longitud; i++)); do
        array+=("$valor")
    done
    echo "Arreglo llamado 'array' creado con longitud $longitud y valor '$valor' en todas
las posiciones"
}

# PRUEBAS

inicializar
imprimir
agregar_elem "A"
agregar_elem "B"
agregar_elem "C"
imprimir
eliminar_elem 1
imprimir
longitud
inicializar_con_valores 3 X
imprimir

```


Ejercicio 27.

Realizar un script que reciba como parámetro el nombre de un directorio. Se deberá validar que el mismo exista y, de no existir, causar la terminación del script con código de error 4. Si el directorio existe, deberá contar, por separado, la cantidad de archivos que en él se encuentran para los cuales el usuario que ejecuta el script tiene permiso de lectura y escritura, e informar dichos valores en pantalla. En caso de encontrar subdirectorios, no deberán procesarse y tampoco deberán ser tenidos en cuenta para la suma a informar.

```
#!/bin/bash
# Script: Ejercicio27.sh
# Uso: ./Ejercicio27.sh

# Control de cantidad de parámetros
if [ $# -ne 1 ]; then
    echo "Error: Se debe pasar, exactamente, 1 parámetro"
    echo "Uso: $0 <directorio>"
    exit 1
fi

# Asignación de parámetros
directorio="$1"

# Verificación de que el directorio exista
if [ ! -d "$directorio" ]; then
    echo "Error: '$directorio' no es un directorio válido"
    exit 4
fi

# Algoritmo
lectura=0
escritura=0
for archivo in "$directorio"/*; do
    if [ -f "$archivo" ]; then
        if [ -r "$archivo" ]; then
            ((lectura++))
        fi
        if [ -w "$archivo" ]; then
            ((escritura++))
        fi
    fi
done

# Resultados
echo "Archivos con permiso de lectura: $lectura"
echo "Archivos con permiso de escritura: $escritura"
```

Ejercicio 28.

Implementar un script que agregue a un arreglo todos los archivos del directorio /home cuya terminación sea .doc. Adicionalmente, implementar las siguientes funciones que permitan acceder a la estructura creada:

- *verArchivo <nombre_de_archivo>*: Imprime el archivo en pantalla si el mismo se encuentra en el arreglo. Caso contrario, imprime el mensaje de error "Archivo no encontrado" y devuelve, como valor de retorno, 5.
- *cantidadArchivos*: Imprime la cantidad de archivos del /home con terminación .doc.
- *borrarArchivo <nombre_de_archivo>*: Consulta al usuario si quiere eliminar el archivo lógicamente. Si el usuario responde "Sí", elimina el elemento sólo del arreglo. Si el usuario responde "No", elimina el archivo del arreglo y también del FileSystem. Debe validar que el archivo exista en el arreglo. En caso de no existir, imprime el mensaje de error "Archivo no encontrado" y devuelve, como valor de retorno, 10.

```
#!/bin/bash
# Script: Ejercicio28.sh
# Uso: ./Ejercicio28.sh

# Vector
touch "/home/existe.doc"
archivos=$(find /home -maxdepth 1 -type f -name "*.doc" 2>/dev/null))

# FUNCIONES

# verArchivo: Imprime el archivo en pantalla si el mismo se encuentra en el arreglo
verArchivo() {
    local archivo="$1"
    for elem in "${archivos[@]}"; do
        if [[ "$(basename "$elem")" == "$archivo" ]]; then
            echo "Mostrando archivo: $elem"
            cat "$elem"
            return 0
        fi
    done
    echo "Archivo no encontrado"
    return 5
}

# cantidadArchivos: Imprime la cantidad de archivos del /home con terminación .doc
cantidadArchivos() {
    echo "Cantidad de archivos en el arreglo: ${#archivos[@]}"
    echo "Cantidad de archivos .doc en /home: $(find /home -maxdepth 1 -type f -name
    "*.doc" | wc -l)"
}
```

borrarArchivo: Elimina un archivo del arreglo y, opcionalmente, del FileSystem

borrarArchivo() {

local archivo="\$1"

local index=-1

for i in "\${!archivos[@]}"; do

if [["\$(basename "\${archivos[\$i]}")" == "\$archivo"]]; then

index=\$i

break

fi

done

if [[\$index -eq -1]]; then

echo "Archivo no encontrado"

return 10

fi

local ruta="\${archivos[\$index]}"

echo -n "¿Desea eliminar el archivo lógicamente (sólo arreglo) o físicamente (arreglo y FileSystem)? (Sí/No): "

read respuesta

if [["\$respuesta" == "Sí" || "\$respuesta" == "Si" || "\$respuesta" == "sí" || "\$respuesta" == "si"]]; then

unset 'archivos[index]'

archivos=("\${archivos[@]}")

echo "Archivo eliminado del arreglo"

elif [["\$respuesta" == "No" || "\$respuesta" == "no"]]; then

unset 'archivos[index]'

archivos=("\${archivos[@]}")

rm -f "\$ruta"

echo "Archivo eliminado del arreglo y del FileSystem"

else

echo "Respuesta inválida. Se debe responder: 'Sí' o 'No'"

fi

}

PRUEBAS

echo "Archivos encontrados: \${archivos[@]}"

verArchivo "noexiste.doc"

verArchivo "existe.doc"

cantidadArchivos

borrarArchivo "noexiste.doc"

borrarArchivo "existe.doc"

cantidadArchivos

Ejercicio 29.

Realizar un script que mueva todos los programas del directorio actual (archivos ejecutables) hacia el subdirectorio "bin" del directorio HOME del usuario actualmente logueado. El script debe imprimir en pantalla los nombres de los que mueve e indicar cuántos ha movido o que no ha movido ninguno. Si el directorio "bin" no existe, deberá ser creado.

```
#!/bin/bash
# Script: Ejercicio29.sh
# Uso: ./Ejercicio29.sh

# Directorio bin del usuario actual
BIN_DIR="/home/jmenduina/bin"

# Creación del directorio bin si no existe
if [ ! -d "$BIN_DIR" ]; then
    mkdir "$BIN_DIR"
fi

# Algoritmo
count=0
for archivo in *; do
    if [ -f "$archivo" ] && [ -x "$archivo" ]; then
        echo "Moviendo: $archivo"
        mv "$archivo" "$BIN_DIR/"
        ((count++))
    fi
done

# Resultado
if [ "$count" -gt 0 ]; then
    echo "Se han movido $count archivo(s)"
else
    echo "No se ha movido ningún archivo ejecutable"
fi
```

Ejercicio 30.

Implementar la estructura de datos Set (Conjunto de valores) en Bash. Un conjunto se define como una colección de valores únicos, es decir, que sólo almacena una vez cada valor, aun cuando se intente agregar el mismo valor más de una vez. La implementación debe soportar las siguientes operaciones mediante funciones:

- *initialize*: Inicializa el set vacío.
- *initialize_with*: Inicializa el set con un conjunto de valores que recibe como argumento (debe validar que se reciba, al menos, uno).
- *add*: Agrega un valor al conjunto, el cual recibe como argumento. No debe agregar elementos repetidos. El resultado de la operación será un éxito sólo si el valor puede ser agregado al conjunto.
- *remove*: Elimina uno o más valores del conjunto, los cuales recibe como argumentos. Si la operación elimina, al menos, un valor, se considera un éxito.
- *contains*: Chequea si el conjunto contiene un valor recibido como argumento. El resultado será éxito si el valor está en el conjunto.
- *print*: Imprime los elementos del conjunto, de a uno por línea.
- *print_sorted*: Imprime los elementos del conjunto, de a uno por línea y ordenados alfabéticamente. Tip: Investigar cómo combinar el comando sort con la función print.

En un script separado, incorporar y utilizar las funciones implementadas para desarrollar un juego de bingo. El bingo deberá generar números aleatorios dentro de un rango entre 0 y un valor máximo que puede especificarse mediante un argumento del script, de manera opcional. El valor máximo no puede ser 0 ni superior a 32.767 y, en caso de no especificarse, se tomará como valor por defecto 99. En cada ronda, se generará un nuevo número que no haya sido utilizado y se lo cantará, imprimiendo en la salida estándar. Luego de esto, se esperará entrada del usuario para saber si se debe cantar “BINGO” para finalizar la partida o se debe cantar un nuevo número. Al finalizar, el script deberá imprimir, en orden, los números que se cantaron hasta que se produjo el bingo.

Tip: Investigar la variable de entorno \$RANDOM de Bash para obtener valores aleatorios.

Script (a):

```
#!/bin/bash
# Script: Ejercicio30a.sh
# Uso: ./Ejercicio30a.sh

# FUNCIONES

# initialize: Inicializa el set vacío
initialize() {
    set=()
}
```

initialize_with: Inicializa el set con un conjunto de valores que recibe como argumento

```
initialize_with() {
    if [ $# -lt 1 ]; then
        echo "Error: Se debe proporcionar, al menos, un valor"
        return 1
    fi
    set=()
    for val in "$@"; do
        add "$val" > /dev/null
    done
}
```

add: Agrega un valor al conjunto, el cual recibe como argumento (no debe agregar elementos repetidos)

```
add() {
    local value="$1"
    for elem in "${set[@]}; do
        if [[ "$elem" == "$value" ]]; then
            return 1
        fi
    done
    set+=("$value")
    return 0
}
```

remove: Elimina uno o más valores del conjunto, los cuales recibe como argumentos

```
remove() {
    local removed=1
    for val in "$@"; do
        local new_set=()
        local found=1
        for elem in "${set[@]}; do
            if [[ "$elem" == "$val" ]]; then
                found=0
            else
                new_set+=("$elem")
            fi
        done
        set=("${new_set[@]}")
        if [[ $found -eq 0 && $removed -eq 1 ]]; then
            removed=0
        fi
    done
    return $removed
}
```

contains: Chequea si el conjunto contiene un valor recibido como argumento

```
contains() {
    local val="$1"
    for elem in "${set[@]}; do
```

```

    if [[ "$elem" == "$val" ]]; then
        return 0
    fi
done
return 1
}

# print: Imprime los elementos del conjunto, de a uno por línea
print() {
    for elem in "${set[@]"; do
        echo "$elem"
    done
}

# print_sorted: Imprime los elementos del conjunto, de a uno por línea y ordenados
alfabéticamente
print_sorted() {
    print | sort
}

# PRUEBAS

echo "Iniciando set sin valores..."
initialize

echo "Iniciando set con valores A B C..."
initialize_with A B C
echo "Imprimiendo set:"
print

echo "Iniciando set con valores C B A..."
initialize_with C B A
echo "Imprimiendo set (con elementos ordenados alfabéticamente):"
print_sorted

echo "Intentando agregar valor A..."
add A && echo "Se pudo agregar" || echo "No se pudo agregar (repetido)"
echo "Intentando agregar valor D..."
add D && echo "Se pudo agregar" || echo "No se pudo agregar (repetido)"
echo "Imprimiendo set:"
print

contains A && echo "Contiene A" || echo "No contiene A"
contains Z && echo "Contiene Z" || echo "No contiene Z"

echo "Intentando eliminar valores A y Z..."
remove A Z && echo "Se eliminó algo" || echo "No se eliminó nada"
echo "Imprimiendo set:"
print

```

Script (b):

```
#!/bin/bash
# Script: Ejercicio30b.sh
# Uso: ./Ejercicio30b.sh

# Importación de 'Ejercicio30a.sh'
source ./ejercicio30a.sh >/dev/null 2>&1

# Configuración del valor máximo
MAX=99
if [ $# -eq 1 ]; then
    if ! [[ "$1" =~ ^[0-9]+$ ]]; then
        echo "Error: El valor máximo debe ser un número entero"
        exit 1
    fi
    if [ "$1" -eq 0 ] || [ "$1" -gt 32767 ]; then
        echo "Error: El valor máximo debe ser > 0 y <= 32767"
        exit 2
    fi
    MAX="$1"
fi

# Inicialización de set de números ya cantados
initialize
echo " 🎲 ¡ Bingo iniciado (0 a $MAX) ! 🎲 "
echo "Presionar ENTER para cantar número o escribir BINGO para terminar"
echo

# Algoritmo
while true; do
    read -p "> " input
    if [[ "$input" == "BINGO" || "$input" == "bingo" || "$input" == "Bingo" ]]; then
        echo " 🎉 ¡ Se cantó BINGO ! 🎉 "
        break
    fi
    while true; do
        num=$(( RANDOM % (MAX + 1) ))
        if ! contains "$num"; then
            add "$num"
            echo "Número cantado: $num"
            break
        fi
    done
done

# Resultados
echo
echo " 📋 Números cantados (ordenados alfabéticamente) hasta que se produjo bingo:"
print_sorted
```


Ejercicio 31.

Realizar un script que reciba como argumento una lista de posibles nombres de usuarios del sistema y, para cada uno de los que, efectivamente, existan en el sistema y posean un directorio personal configurado que sea válido, realice las modificaciones necesarias en su directorio personal para que tenga un subdirectorio llamado “directorio_iso” con la siguiente estructura:

```

directorio_iso
├── 2025
│   ├── 01
│   │   └── archivo.txt
│   ├── 02
│   │   └── archivo.txt
│   ├── 03
│   │   └── archivo.txt
│   ├── 04
│   │   └── archivo.txt
│   ├── 05
│   │   └── archivo.txt
│   ├── 06
│   │   └── archivo.txt
│   ├── 07
│   │   └── archivo.txt
│   ├── 08
│   │   └── archivo.txt
│   ├── 09
│   │   └── archivo.txt
│   ├── 10
│   │   └── archivo.txt
│   ├── 11
│   │   └── archivo.txt
│   └── 12
│       └── archivo.txt
└── 2026

```

Para resolver la creación de los directorios y archivos, utilizar la funcionalidad “Brace Expansion” brindada por bash: https://www.gnu.org/software/bash/manual/html_node/Brace-Expansion.html.

```

#!/bin/bash
# Script: Ejercicio31.sh
# Uso: ./Ejercicio31.sh

# Control de cantidad de parámetros
if [ $# -lt 1 ]; then
    echo "Error: Se debe pasar, al menos, 1 parámetro"
    echo "Uso: $0 <nombre_usuario1> <nombre_usuario2> ..."
    exit 1
fi

```

```
# Algoritmo
for usuario in "$@"; do

    if ! id "$usuario" &>/dev/null; then
        echo "El usuario $usuario no existe en el sistema"
        continue
    fi

    home_dir=$(getent passwd "$usuario" | cut -d: -f6)

    if [ ! -d "$home_dir" ]; then
        echo "El usuario '$usuario' no tiene home válido"
        continue
    fi

    echo "Creando estructura en $home_dir/directorio_iso..."

    mkdir -p "$home_dir/directorio_iso/{2025/{01..12},2026/{01..12}}"

    for year in {2025..2026}; do
        for month in {01..12}; do
            touch "$home_dir/directorio_iso/$year/$month/archivo.txt"
        done
    done

    echo "Estructura creada en $home_dir/directorio_iso"

done
```

Trabajo Práctico N° 4:
Paginación por Demanda. Algoritmos de Reemplazos.
Memoria Virtual. Entrada-Salida. Dispositivos.
Administración de Discos. Sistemas de Archivos. *Buffer*
Caché.

PARTE I: Administración de Memoria Principal.

Ejercicio 1: Memoria Virtual.

(a) *Describir qué beneficios introduce este esquema de administración de la memoria.*

Los beneficios que introduce este esquema de administración de la memoria son:

1. Aumento del espacio de direcciones disponible:
 - Cada proceso percibe un espacio de direcciones lógico mucho mayor que la memoria física real.
 - Permite ejecutar programas más grandes que la RAM disponible (“ilusión” de memoria ilimitada).
2. Protección y aislamiento entre procesos:
 - Cada proceso tiene su propio espacio de direcciones virtual, separado del resto.
 - Un proceso no puede leer ni escribir memoria de otro, aumentando seguridad y estabilidad.
3. Multiprogramación más eficiente:
 - Permite que más procesos estén cargados, parcialmente, en memoria al mismo tiempo.
 - Aumenta la utilización de la CPU y el rendimiento del sistema.
4. Carga diferencial de páginas (*paging on demand*):
 - Sólo se cargan en memoria las partes, realmente, utilizadas del programa.
 - Reduce la cantidad de RAM necesaria para ejecutar procesos.
5. Simplificación para el programador:
 - El programador no necesita gestionar, manualmente, memoria física.
 - Puede trabajar con direcciones continuas, aunque, físicamente, la memoria esté fragmentada.
6. Flexibilidad para compartir memoria:
 - Permite compartir páginas entre procesos (por ejemplo, bibliotecas dinámicas).
 - Ahorra memoria cuando varios procesos usan el mismo código.
7. Soporte para técnicas avanzadas:
 - Memoria compartida entre procesos.
 - *Copy-on-write*.
 - *Swapping*.
 - *Demand paging* y algoritmos de reemplazo.

(b) *¿En qué se debe apoyar el SO para su implementación?*

El sistema operativo, para su implementación, se debe apoyar en:

1. Unidad de Gestión de Memoria (MMU - *Memory Management Unit*):
 - Es el *hardware* encargado de traducir direcciones lógicas a físicas.
 - Realiza el mapeo mediante tablas de páginas.
 - Detecta fallos de página (*page faults*).
2. Tablas de páginas:
 - Estructuras que mantienen:
 - El marco de página en RAM (si está presente).
 - Bits de control: presente/ausente, modificado, referencia, protección.
 - Son esenciales para la traducción y para aplicar algoritmos de reemplazo.
3. Mecanismos de interrupción:
 - Interrupciones por fallo de página: Notifican al sistema operativo que una página requerida no está en RAM.
 - Permiten que el sistema operativo ejecute rutinas de manejo de páginas (traer la página desde disco, elegir víctima, etc.).
4. Memoria secundaria (disco o SSD):
 - Alberga la parte del espacio de direcciones que no cabe en RAM.
 - Incluye:
 - Área de *swap*.
 - Archivos ejecutables mapeados.
 - Sirve como respaldo para almacenar páginas ausentes.
5. Algoritmos de reemplazo de páginas:
 - El sistema operativo debe implementar criterios para decidir qué página sacar cuando la RAM está llena (FIFO, LRU, *Second Chance*, OPT -teórico-).
6. Rutinas del sistema operativo para manejo de páginas:
 - El sistema operativo debe:
 - Cargar la página desde disco.
 - Actualizar las tablas de páginas.
 - Marcar bits (modificado, referencia).
 - Reanudar el proceso interrumpido.
 - Estas rutinas constituyen el *page fault handler*.

(c) *Al implementar esta técnica utilizando paginación por demanda, las tablas de páginas de un proceso deben contar con información adicional además del marco donde se encuentra la página. ¿Cuál es esta información? ¿Por qué es necesaria?*

Cuando se implementa memoria virtual utilizando paginación por demanda, las tablas de páginas de un proceso deben contar con información adicional además del marco donde se encuentra la página. Esto se debe a que muchas páginas no están cargadas en memoria física y el sistema operativo debe poder gestionar qué páginas están presentes, cuáles fueron modificadas, etc.

Esta información es:

1. Bit de presencia/ausencia (*Present bit*):
 - Indica si la página está cargada en RAM.
 - Necesario: Sin este bit, la MMU no puede determinar si debe generar un *page fault* cuando la página aún no está en memoria.
2. Bit de modificado (*Dirty bit*):
 - Indica si la página fue escrita desde que se cargó.
 - Necesario: Si está en 1, la página debe copiarse al disco antes de ser reemplazada; si no está en 1, se puede descartar.
3. Bit de referencia (*Referenced/Accessed bit*):
 - Indica si la página fue usada recientemente.
 - Necesario: Es fundamental para los algoritmos de reemplazo (LRU aproximado, *Clock*) para decidir qué página es candidata a ser expulsada.
4. Bits de protección:
 - Especifican permisos: lectura, escritura, ejecución.
 - Necesario: Previene accesos inválidos o maliciosos; permite que un *page fault* se use también para protección.
5. Bit de validación del contenido (*Valid/Invalid*):
 - Diferencia entre:
 - Página válida, pero no presente (*swap*).
 - Página inválida (dirección que el proceso no debería usar).
 - Necesario: Evita accesos ilegales al espacio del proceso.
6. Información sobre el lugar en disco (parcial o completa):
 - Puede ser:
 - Índice del bloque en el área de *swap*.
 - Identificador en un archivo ejecutable mapeado.
 - Necesario: Para que el sistema operativo sepa dónde buscar la página cuando ocurre un *page fault*.

En resumen, utilizando paginación por demanda, cada entrada de tabla de páginas debe contener: Presente/Ausente, Modificado, Referenciado, Permisos, Validez y Ubicación en disco. Toda esta información es necesaria porque permite: generar y manejar *page faults*; decidir qué páginas reemplazar; proteger el acceso a memoria; recuperar o descartar páginas desde/hacia disco.

Ejercicio 2: Fallos de Página (Page Faults).

(a) *¿Cuándo se producen?*

Un fallo de página (*page fault*) se produce cuando un proceso intenta acceder a una página que no se encuentra cargada en la memoria física (RAM). Esto ocurre cuando la entrada de la tabla de páginas para esa página indica que no está presente (bit de presencia= 0). La MMU detecta que la dirección virtual no puede traducirse porque la página no está en RAM. Entonces, genera una interrupción al sistema operativo.

(b) *¿Quién es responsable de detectar un fallo de página?*

El responsable de detectar un fallo de página es la Unidad de Gestión de Memoria (MMU - *Memory Management Unit*).

La MMU intenta traducir la dirección virtual a una dirección física utilizando la tabla de páginas. Si la entrada correspondiente indica que la página no está presente en memoria (bit de presencia= 0) o es inválida, la MMU no puede completar la traducción y, entonces, genera una interrupción de *hardware* o *trap*. Esa interrupción se envía al sistema operativo, que, luego, se encarga de manejar el fallo (traer la página desde disco, elegir víctima, actualizar la tabla de páginas, etc.), pero la detección inicial es, puramente, responsabilidad de la MMU.

(c) *Describir las acciones que emprende el Kernel cuando se produce un fallo de página.*

Las acciones que emprende el *Kernel* cuando se produce un fallo de página son:

1. Verificar el tipo de acceso y la validez de la página:
 - El *Kernel* consulta la tabla de páginas para determinar:
 - Si la página pertenece al proceso.
 - Si el acceso es válido (lectura, escritura, ejecución).
 - Si la página, simplemente, no está presente o si el acceso es ilegal.
 - Si es un acceso ilegal, se aborta el proceso (*segmentation fault*).
2. Identificar la ubicación de la página en disco:
 - Si el acceso es válido, determina dónde se encuentra la página:
 - En el área de swap.
 - En un archivo ejecutable o de datos mapeado.
 - O si es una página nueva (*heap/stack*) a inicializar.
3. Buscar un marco libre en memoria física:
 - El *Kernel* necesita un marco para cargar la página.
 - Si hay marcos libres, usa uno.
 - Si no hay marcos libres, debe aplicar un algoritmo de reemplazo de páginas (FIFO, LRU, *Second Chance*, etc.) para elegir una “víctima”.
4. Si la página víctima está modificada, escribirla en disco:
 - Si el *dirty bit*= 1, la página debe escribirse en el área de *swap*.

- Si no está modificada, simplemente, se descarta (no hay que copiar nada).
- 5. Cargar la nueva página desde disco hacia RAM:
 - La página válida solicitada se copia desde:
 - *Swap*.
 - Archivo ejecutable.
 - O se inicializa si es una página nueva.
- 6. Actualizar la tabla de páginas del proceso:
 - Se marca la entrada como presente.
 - Se actualiza el número de marco.
 - Se reinician los bits de referencia y modificado.
 - Se realiza cualquier actualización de protección o metadatos.
- 7. Actualizar la TLB (si es necesario):
 - Si la entrada estaba en la TLB con estado inválido, se invalida y se reemplaza.
 - Puede requerir un TLB *flush* parcial o selectivo.
- 8. Reanudar la ejecución del proceso:
 - Una vez que la página está disponible:
 - La instrucción interrumpida se reintenta.
 - El proceso continúa como si nada hubiera pasado.

Ejercicio 3.

Suponer que la tabla de páginas para un proceso que se está ejecutando es la que se muestra a continuación:

<i>Página</i>	<i>Bit V</i>	<i>Bit R</i>	<i>Bit M</i>	<i>Marco</i>
0	1	1	0	4
1	1	1	1	7
2	0	0	0	-
3	1	0	0	2
4	0	0	0	-
5	1	0	1	0

Asumiendo que:

- El tamaño de la página es de 512 bytes.
- Cada dirección de memoria referencia 1 byte.
- Los marcos se encuentran contiguos y en orden en memoria (0, 1, 2, ...) a partir de la dirección física 0.

¿Qué dirección física, si existe, correspondería a cada una de las siguientes direcciones virtuales? (No gestionar ningún fallo de página en caso de producirse).

(a) 1052.

Página= $1052 \text{ DIV } 512$

Página= 2.

Desplazamiento= $1052 \text{ MOD } 512$

Desplazamiento= 28.

Por lo tanto, no existe dirección física para esta dirección virtual, ya que la página 2 no está presente en memoria.

(b) 2221.

Página= $2221 \text{ DIV } 512$

Página= 4.

Desplazamiento= $2221 \text{ MOD } 512$

Desplazamiento= 173.

Por lo tanto, no existe dirección física para esta dirección virtual, ya que la página 4 no está presente en memoria.

(c) 5499.

Página= 5499 DIV 512

Página= 10.

Desplazamiento= 5499 MOD 512

Desplazamiento= 379.

Por lo tanto, no existe dirección física para esta dirección virtual, ya que la página 10 no existe en la tabla de páginas.

(d) 3101.

Página= 3101 DIV 512

Página= 6.

Desplazamiento= 3101 MOD 512

Desplazamiento= 29.

Por lo tanto, no existe dirección física para esta dirección virtual, ya que la página 6 no existe en la tabla de páginas.

Ejercicio 4.

Analizar cómo impacta el tamaño de una página (pequeña o grande) en paginación por demanda.

A continuación, se analiza cómo impacta el tamaño de una página en paginación por demanda:

1. Tamaño de página pequeño:

- Ventajas:
 - Menor fragmentación interna: Como las páginas son chicas, es menos probable que un proceso deje “espacio desperdiciado” dentro de una página.
 - Ajuste más fino entre el programa y las páginas: se cargan sólo las partes del programa necesarias; mejor uso de la memoria RAM.
 - Mayor grado de multiprogramación: Como cada proceso usa menos RAM, entran más procesos simultáneamente.
 - Mejor localización de fallos de página: Las páginas pequeñas permiten manejar mejor patrones de acceso dispersos.
- Desventajas:
 - Tablas de páginas más grandes: Como hay más páginas por proceso, la tabla crece y consume más memoria.
 - Mayor *overhead* en el TLB: más entradas necesarias; más *misses*; mayor tiempo de acceso.
 - Más fallos de páginas totales: Dado que cada página contiene menos datos, se necesita traer más páginas desde disco.

2. Tamaño de página grande:

- Ventajas:
 - Tablas de páginas más pequeñas: Menos páginas por proceso, por lo que la tabla es más corta y eficiente.
 - Menos TLB *misses*: Cada entrada cubre más direcciones, por lo que hay mayor probabilidad de aciertos en el TLB.
 - Mayor eficiencia en accesos secuenciales: Para cargas masivas o *streams* grandes, las páginas grandes reducen fallos.
 - Mejor *throughput* del disco: Cargar páginas grandes usa mejor el ancho de banda del disco.
- Desventajas:
 - Mayor fragmentación interna: Mucho espacio dentro de las páginas queda sin usar.
 - Pérdida de precisión en la asignación: Se cargan grandes bloques aunque se necesite muy poco, por lo que hay desperdicio de RAM.
 - Menor multiprogramación: Cada proceso ocupa más memoria, por lo que entran menos procesos al mismo tiempo.
 - Si la localidad es mala, empeora la *performance*: Se traen páginas enormes desde disco que no se usan.

Ejercicio 5: Asignación de Marcos a un Proceso (Conjunto de Trabajo o *Working Set*).

Con la memoria virtual paginada, no se requiere que todas las páginas de un proceso se encuentren en memoria. El SO debe controlar cuántas páginas de un proceso puede tener en la memoria principal. Existen 2 políticas que se pueden utilizar:

- *Asignación Fija.*
- *Asignación Dinámica.*

(a) *Describir cómo trabajan estas 2 políticas.*

A continuación, se describe cómo trabajan estas 2 políticas:

1. Asignación Fija:

- El sistema operativo asigna un número fijo de marcos de página a cada proceso desde el momento en que se crea. Ese número no cambia durante toda la ejecución del proceso.
- Funcionamiento:
 - Cuando el proceso inicia, el sistema operativo le asigna una cantidad determinada de marcos (por ejemplo, 10 marcos).
 - El proceso sólo puede usar esa cantidad, aunque necesite más o aunque le sobren.
 - Si el proceso necesita una página nueva y no tiene marcos libres, debe expulsar una de sus propias páginas (*page replacement local*).
 - No puede pedir marcos adicionales.
- Ventajas:
 - Simplicidad de implementación.
 - Predecible en cuanto a uso de memoria.
 - Evita que un proceso consuma demasiados marcos y afecte a otros.
- Desventajas:
 - Poco flexible.
 - Algunos procesos podrían quedar con marcos de más y otros con marcos insuficientes.
 - Aumenta el riesgo de *thrashing* si el conjunto de trabajo del proceso crece.

2. Asignación Dinámica:

- El sistema operativo asigna un número variable de marcos a cada proceso, que puede aumentar o disminuir durante su ejecución.
- Funcionamiento:
 - El sistema operativo monitorea el comportamiento del proceso (frecuencia de fallos de página, *working set*, etc.).
 - Si el proceso muestra muchos fallos de página, se le otorgan más marcos.
 - Si usa pocos marcos o su *working set* se achica, se le quitan marcos y se reasignan a otros procesos.
 - Permite balancear, dinámicamente, la memoria total entre procesos.
- Ventajas:
 - Mayor eficiencia global.
 - Se adapta a las necesidades reales del proceso.

- Reduce el riesgo de *thrashing* (cada proceso puede recibir marcos según su *working set*).
- Desventajas:
 - Mayor complejidad del sistema operativo.
 - Requiere monitorear fallos de página y comportamiento del proceso.
 - Si no está bien regulado, un proceso puede crecer demasiado y perjudicar a otros.

(b) Dada la siguiente tabla de procesos y las páginas que ellos ocupan, y teniéndose 40 marcos en la memoria principal, ¿cuántos marcos le corresponderá a cada proceso si se usa la técnica de Asignación Fija:

- con Reparto Equitativo?
- con Reparto Proporcional?

Proceso	Total de Páginas Usadas
1	15
2	20
3	20
4	8

Reparto Equitativo: Se reparten los mismos marcos a cada proceso, sin importar su tamaño.

$$\text{Marcos por proceso} = \frac{40 \text{ marcos}}{4 \text{ procesos}}$$

$$\text{Marcos por proceso} = 10.$$

Por lo tanto, si se usa la técnica de Asignación Fija con Reparto Equitativo, a cada proceso, le corresponderá 10 marcos.

Reparto Proporcional: Se reparte según la cantidad de páginas que usa cada proceso, respecto al total.

$$\text{Marcos}_{\text{proceso 1}} = 40 \frac{15}{63}$$

$$\text{Marcos}_{\text{proceso 1}} = 9,52$$

$$\text{Marcos}_{\text{proceso 1}} \cong 9.$$

$$\text{Marcos}_{\text{proceso 2}} = 40 \frac{20}{63}$$

$$\text{Marcos}_{\text{proceso 2}} = 12,7.$$

$$\text{Marcos}_{\text{proceso 2}} \cong 13.$$

$$\text{Marcos}_{\text{proceso 3}} = 40 \frac{20}{63}$$

$$\text{Marcos}_{\text{proceso 3}} = 12,7.$$

$$\text{Marcos}_{\text{proceso 3}} \cong 13.$$

$$Marcos_{proceso\ 4} = 40 \frac{8}{63}$$

$$Marcos_{proceso\ 4} = 5,08$$

$$Marcos_{proceso\ 4} \cong 5.$$

Por lo tanto, si se usa la técnica de Asignación Fija con Reparto Proporcional, a cada proceso, le corresponderá 9, 13, 13 y 5 marcos, respectivamente.

(c) ¿Cuál de los 2 repartos usados en (b) resultó más eficiente? Justificar.

Déficit por proceso= (páginas - marcos) si > 0 .

Reparto Equitativo:

Déficit por proceso=

- P1: 15 - 10= 5.
- P2: 20 - 10= 10.
- P3: 20 - 10= 10.
- P4: 8 - 10= -2.

Déficit total= 5 + 10 + 10

Déficit total= 25.

Reparto Proporcional:

Déficit por proceso=

- P1: 15 - 9= 6.
- P2: 20 - 13= 7.
- P3: 20 - 13= 7.
- P4: 8 - 5= 3.

Déficit total= 6 + 7 + 7 + 3

Déficit total= 23.

Por lo tanto, de los 2 repartos usados en (b), resultó ser más eficiente el Reparto Proporcional, ya que es mayor la probabilidad de que el conjunto de trabajo (*working set*) de cada proceso esté cubierto por sus marcos, lo que reduce fallos de página, aumenta el *throughput* y mejora la utilización de CPU.

Ejercicio 6: Reemplazo de Páginas (Selección de una Víctima).

¿Qué sucede cuando todos los marcos en la memoria principal están usados por las páginas de los procesos y se produce un fallo de página? El SO debe seleccionar una de las páginas que se encuentra en memoria principal (en un marco) como víctima, la que será reemplazada por la nueva página que produjo el fallo.

Considerar los siguientes algoritmos de selección de víctimas básicos:

- *LRU.*
- *FIFO.*
- *OPT (Óptimo).*
- *Second Chance.*

(a) *Clasificar estos algoritmos de malo a bueno de acuerdo a la tasa de fallos de página que se obtienen al utilizarlos.*

A continuación, se clasifican estos algoritmos de malo a bueno de acuerdo a la tasa de fallos de página que se obtienen al utilizarlos:

- FIFO (First-In, First-Out): Es el más simple (expulsa la página más antigua), pero puede comportarse mal frente a ciertas secuencias de referencias (sufrir la anomalía de Belady) y, por eso, suele dar la peor tasa de fallos entre los listados.
- Second Chance: Es una aproximación barata de LRU (usa el bit R para dar una “segunda oportunidad”). Normalmente, tiene rendimiento algo inferior a LRU, pero mucho mejor que FIFO en la mayoría de los casos, y con menor costo de implementación.
- LRU (Least Recently Used): Aproxima el comportamiento óptimo al expulsar la página menos recientemente usada; muy buena tasa de fallos en la práctica y es un algoritmo “*stack*” (no sufre la anomalía de Belady).
- OPT (Óptimo): Conoce el futuro y, por lo tanto, minimiza, exactamente, la cantidad de fallos de página. Es teórico (no implementable en la práctica), pero es la referencia ideal.

(b) *Analizar su funcionamiento. ¿Cómo se podrían implementar?*

1. FIFO (First-In, First-Out):

- **Funcionamiento:**
 - Reemplaza la página que lleva más tiempo en memoria, sin importar si fue usada recientemente o no.
 - Mantiene las páginas en una cola ordenada por tiempo de carga.
- **Implementación:**
 - Cola FIFO: cuando una página entra a memoria, se encola; cuando hay que reemplazar, se desencola la más antigua.
 - Muy fácil de implementar.

- No usa bits R ni M (sólo ayuda a reducir I/O si $M=1$, pero no afecta la elección de víctima).
 - Ventajas:
 - Simple, bajo costo.
 - Desventajas:
 - Mala tasa de fallos.
 - Sufre anomalía de Belady.
2. Second Chance:
- Funcionamiento:
 - Es una mejora de FIFO usando el bit R.
 - Mantiene una cola como FIFO.
 - Cuando la página más antigua tiene $R=1$, no se la reemplaza, se pone $R=0$ y se mueve al final de la cola (segunda oportunidad).
 - Se sigue avanzando en la cola hasta encontrar una página con $R=0$, que será la víctima.
 - Implementación:
 - Cola circular (implementación tipo *Clock*).
 - Requiere un bit R por página y un puntero que recorre los marcos.
 - Ventajas:
 - Mejor que FIFO.
 - Barato de implementar (sólo revisar bits R, no necesita *timestamps*).
 - Desventajas:
 - Peor que LRU.
 - No considera antigüedad real, sólo si fue usada recientemente.
3. LRU (Least Recently Used):
- Funcionamiento:
 - Reemplaza la página que hace más tiempo que no se usa.
 - Imita el comportamiento ideal: Asume que lo que no se usó recientemente es menos probable que vuelva a usarse.
 - Implementación:
 - Difícil en *hardware* puro, por lo que se aplican aproximaciones.
 - Implementación exacta (costosa): Guardar un *timestamp* cada vez que se accede a una página. Para elegir la víctima, buscar la página con *timestamp* más viejo.
 - Implementaciones aproximadas: *Clock* mejorado; NRU (*Not Recently Used*); LRU por bits (matriz de $n \times n$ bits).
 - Ventajas:
 - Excelente tasa de fallos.
 - No sufre anomalía de Belady.
 - Desventajas:
 - Implementación costosa, salvo aproximaciones.
4. OPT (Óptimo):
- Funcionamiento:
 - Expulsa la página que será usada más lejos en el futuro.
 - Implementación:
 - No se puede implementar en la vida real porque se necesitan conocer las referencias futuras.
 - Se usa: como referencia teórica para evaluar otros algoritmos; en simuladores de fallos de página.

- Ventajas:
 - Mínima tasa de fallos posible.
- Desventajas:
 - Sólo teórico.

(c) *Se sabe que la página a ser reemplazada puede estar modificada. ¿Cómo detecta el Kernel esta situación? ¿Qué acciones adicionales deben realizarse cuando sucede esta situación?*

Si la página a ser reemplazada está modificada, el *Kernel* detecta esta situación usando el bit M (*Modified*), que está en la entrada de la tabla de páginas correspondiente a esa página.

Las acciones adicionales que deben realizarse cuando sucede esta situación son:

1. Escribir la página modificada al disco (*swap*).
2. Marcar el bit M= 0.
3. Liberar el marco.

Ejercicio 7: Alcance del Reemplazo.

Al momento de tener que seleccionar una página víctima, el Kernel puede optar por 2 políticas a utilizar:

- *Reemplazo Local.*
- *Reemplazo Global.*

(a) *Describir cómo trabajan estas 2 políticas.*

A continuación, se describe cómo trabajan estas 2 políticas:

1. Reemplazo Local:

- **Funcionamiento:**
 - Cada proceso tiene asignado un conjunto fijo de marcos.
 - Ante un fallo de página, sólo puede reemplazar una página que le pertenece a él mismo, es decir, dentro de su propio conjunto de marcos.
 - No puede tomar marcos que están asignados a otros procesos.
- **Consecuencias:**
 - El desempeño del proceso depende, únicamente, de los marcos que tiene asignados.
 - Si un proceso recibe pocos marcos, puede sufrir muchos fallos de página.
 - Un proceso con baja actividad no libera marcos para otros.
 - Se evita la interferencia entre procesos.
- **Ventajas:**
 - Aísla a los procesos: Uno muy demandante no perjudica a los demás.
 - Fácil de controlar.
- **Desventajas:**
 - Puede llevar al subaprovechamiento de memoria (si un proceso no usa todos sus marcos).
 - Procesos mal asignados pueden entrar en *thrashing* sin que otros puedan ayudarlos.

2. Reemplazo Global:

- **Funcionamiento:**
 - Ante un fallo de página, el proceso puede reemplazar cualquier página del sistema, sin importar a qué proceso pertenezca.
 - La elección de víctima se hace sobre todas las páginas residentes en memoria, no sólo las del proceso actual.
- **Consecuencias:**
 - Procesos con alta demanda pueden “robar” marcos a procesos con baja demanda.
 - La memoria se usa de forma más dinámica y eficiente.
 - Pero un proceso puede afectar, gravemente, el rendimiento de otros.
- **Ventajas:**
 - Aumenta la utilización global de la memoria.
 - Se adapta, dinámicamente, a los procesos que más memoria requieren.
 - Reduce el riesgo de tener marcos ociosos.
- **Desventajas:**

- No aísla procesos: Uno puede perjudicar a los demás quitándoles marcos.
- Puede provocar *thrashing* distribuido entre procesos.
- Más complejo de controlar.

(b) *¿Es posible utilizar la política de “Asignación Fija” de marcos junto con la política de “Reemplazo Global”? Justificar.*

No, no es posible utilizar la política de “Asignación Fija” de marcos junto con la política de “Reemplazo Global”, ya que son conceptos contradictorios:

- Asignación Fija impone un límite rígido de marcos para cada proceso.
- Reemplazo Global rompe ese límite porque permite que un proceso use marcos que, originalmente, pertenecen a otro.

Si el sistema permitiera Reemplazo Global mientras dice usar Asignación Fija, dejaría de ser fija, porque un proceso podría perder marcos o ganar marcos por decisiones globales. El Reemplazo Global, por definición, requiere Asignación Dinámica.

En resumen, no es posible porque la Asignación Fija garantiza un conjunto de marcos exclusivo para cada proceso, mientras que el Reemplazo Global permite que un proceso reemplace páginas pertenecientes a otros, violando, así, la exclusividad de la Asignación Fija.

Ejercicio 8.

Considerar la siguiente secuencia de referencias de páginas:

1, 2, 15, 4, 6, 2, 1, 5, 6, 10, 4, 6, 7, 9, 1, 6, 12, 11, 12, 2, 3, 1, 8, 1, 13, 14, 15, 3, 8.

(a) Si se disponen de 5 marcos, ¿cuántos fallos de página se producirán si se utilizan las siguientes técnicas de selección de víctima? (Considerar una política de Asignación Dinámica y Reemplazo Global).

- *Second Chance.*
- *FIFO.*
- *LRU.*
- *OPT.*

Second Chance:

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1*	1	1	1	4	4	4	4	4
F2		2	2	2	2	2*	2*	2	2	2	2	2	7	7	7
F3			15	15	15	15	15	5	5	5	5	5	5	9	9
F4				4	4	4	4	4	4	10	10	10	10	10	1
F5					6	6	6	6	6*	6*	6	6*	6*	6*	6*
¿PF?	X	X	X	X	X			X		X	X		X	X	X

Marco/Página	6	12	11	12	2	3	1	8	1	13	14	15	3	8	
F1	4	12	12	12*	12*	12*	12*	12	12	12	12	15	15	15	
F2	7	7	11	11	11	11	11	8	8	8	8	8	3	3	
F3	9	9	9	9	2	2	2	2	2	13	13	13	13	8	
F4	1	1	1	1	1	3	3	3	3	3	14	14	14	14	
F5	6*	6	6	6	6	6	1	1	1*	1*	1*	1	1	1	
¿PF?		X	X		X	X	X	X		X	X	X	X	X	

Queue: 1, 2, 15, 4, 6, 1, 2, 5, 10, 6, 4, 7, 9, 1, 6, 12, 11, 2, 3, 1, 12, 8, 13, 14, 1, 15, 3, 8.

Por lo tanto, si se utiliza la técnica de selección de víctima *Second Chance*, se producirán 22 fallos de página.

FIFO:

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1	5	5	5	5	5	5	5	5
F2		2	2	2	2	2	2	2	2	10	10	10	10	10	10
F3			15	15	15	15	15	15	15	15	15	15	7	7	7
F4				4	4	4	4	4	4	4	4	4	4	9	9
F5					6	6	6	6	6	6	6	6	6	6	1
¿PF?	X	X	X	X	X			X		X			X	X	X

Marco/Página	6	12	11	12	2	3	1	8	1	13	14	15	3	8	
F1	6	6	6	6	6	6	1	1	1	1	1	1	3	3	
F2	10	12	12	12	12	12	12	8	8	8	8	8	8	8	
F3	7	7	11	11	11	11	11	11	11	13	13	13	13	13	
F4	9	9	9	9	2	2	2	2	2	2	14	14	14	14	
F5	1	1	1	1	1	3	3	3	3	3	3	15	15	15	
¿PF?	X	X	X		X	X	X	X		X	X	X	X		

Queue: 1, 2, 15, 4, 6, 5, 10, 7, 9, 1, 6, 12, 11, 2, 3, 1, 8, 13, 14, 15, 3.

Por lo tanto, si se utiliza la técnica de selección de víctima FIFO, se producirán 21 fallos de página.

LRU:

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1	1	1	1	1	1	7	7	7
F2		2	2	2	2	2	2	2	2	2	4	4	4	4	4
F3			15	15	15	15	15	5	5	5	5	5	5	9	9
F4				4	4	4	4	4	4	10	10	10	10	10	1
F5					6	6	6	6	6	6	6	6	6	6	6
¿PF?	X	X	X	X	X			X		X	X		X	X	X

Marco/Página	6	12	11	12	2	3	1	8	1	13	14	15	3	8	
F1	7	7	11	11	11	11	11	8	8	8	8	8	3	3	
F2	4	12	12	12	12	12	12	12	12	13	13	13	13	13	
F3	9	9	9	9	2	2	2	2	2	2	14	14	14	14	
F4	1	1	1	1	1	3	3	3	3	3	3	15	15	15	
F5	6	6	6	6	6	6	1	1	1	1	1	1	1	8	
¿PF?		X	X		X	X	X	X		X	X	X	X	X	

Por lo tanto, si se utiliza la técnica de selección de víctima LRU, se producirán 22 fallos de página.

OPT:

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
F2		2	2	2	2	2	2	2	2	2	2	2	2	2	2
F3			15	15	15	15	15	5	5	10	10	10	7	9	9
F4				4	4	4	4	4	4	4	4	4	4	4	4
F5					6	6	6	6	6	6	6	6	6	6	6
¿PF?	X	X	X	X	X			X		X			X	X	

Marco/Página	6	12	11	12	2	3	1	8	1	13	14	15	3	8	
F1	1	1	1	1	1	1	1	1	1	13	14	15	15	15	
F2	2	2	2	2	2	3	3	3	3	3	3	3	3	3	
F3	9	12	12	12	12	12	12	8	8	8	8	8	8	8	
F4	4	4	11	11	11	11	11	11	11	11	11	11	11	11	
F5	6	6	6	6	6	6	6	6	6	6	6	6	6	6	
¿PF?		X	X			X		X		X	X	X			

Por lo tanto, si se utiliza la técnica de selección de víctima OPT, se producirán 16 fallos de página.

(b) Suponiendo que cada atención de un fallo de página requiere de 0,1 seg. Calcular el tiempo consumido por atención a los fallos de páginas para los algoritmos de (a).

Suponiendo que cada atención de un fallo de página requiere de 0,1 seg., el tiempo consumido por atención a los fallos de páginas para los algoritmos de (a) es:

- Second Chance: $22 * 0,1 \text{ seg.} = 2,2 \text{ seg.}$
- FIFO: $21 * 0,1 \text{ seg.} = 2,1 \text{ seg.}$
- LRU: $22 * 0,1 \text{ seg.} = 2,2 \text{ seg.}$
- OPT: $16 * 0,1 \text{ seg.} = 1,6 \text{ seg.}$

Ejercicio 9.

Sean los procesos A, B y C tales que necesitan, para su ejecución, las siguientes páginas:

- A: 1, 3, 1, 2, 4, 1, 5, 1, 4, 7, 9, 4.
- B: 2, 4, 6, 2, 4, 1, 8, 3, 1, 8.
- C: 1, 2, 4, 8, 6, 1, 4, 1.

Si la secuencia de ejecución es tal que los procesos se ejecutan en la siguiente manera:

1. B demanda 2 páginas.
2. A demanda 3 páginas.
3. C demanda 2 páginas.
4. B demanda 3 páginas.
5. A demanda 3 páginas.
6. C demanda 2 páginas.
7. B demanda 2 páginas.
8. C demanda 4 páginas.
9. A demanda 3 páginas.
10. B demanda 3 páginas.
11. C termina.
12. A demanda 3 páginas.
13. B termina.
14. A termina.

(a) Considerando una política de Asignación Dinámica y Reemplazo Global y disponiéndose de 7 marcos. ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctima:

- LRU?
- Second Chance?

LRU:

Marcos/Página	B2	B4	A1	A3	A1	C1	C2	B6	B2	B4	A2	A4	A1	C4	C8	B1	B8
F1	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B2	B1	B1
F2		B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B4	B8
F3			A1	A1	A1	A1	A1	A1	A1	A1	A1	A4	A4	A4	A4	A4	A4
F4				A3	A3	A3	A3	A3	A3	A3	A2	A2	A2	A2	A2	A2	A2
F5						C1	C1	C1	C1	C1	C1	C1	A1	A1	A1	A1	A1
F6							C2	C2	C2	C2	C2	C2	C2	C4	C4	C4	C4
F7								B6	B6	B6	B6	B6	B6	B6	C8	C8	C8
¿PF?	X	X	X	X		X	X	X			X	X	X	X	X	X	X

Marco/Página	C6	C1	C4	C1	A5	A1	A4	B3	B1	B8	CT	A7	A9	A4	BT	AT
F1	B1	B1	B1	B1	B1	B1	A4	A4	A4	A4	A4	A4	A4	A4	A4	
F2	B8	B8	B8	B8	B8	B8	B8	B3	B3	B3	B3	B3	B3	B3		
F3	A4	C1	C1	C1	C1	C1	C1	C1	C1	C1		A7	A7	A7	A7	
F4	C6	C6	C6	C6	C6	C6	C6	C6	B1	B1	B1	B1	B1	B1		
F5	A1	A1	A1	A1	A5	A5	A5	A5	A5	A5	A5	A5	A9	A9	A9	
F6	C4	C4	C4	C4	C4	C4	C4	C4	C4	B8	B8	B8	B8	B8		
F7	C8	C8	C8	C8	C8	A1	A1	A1	A1	A1	A1	A1	A1	A1	A1	
¿PF?	X	X			X	X	X	X	X	X		X	X			

Por lo tanto, si se utiliza la técnica de selección de víctima LRU, se producirán 24 fallos de página.

Second Chance:

Marco/Página	B2	B4	A1	A3	A1	C1	C2	B6	B2	B4	A2	A4	A1	C4	C8	B1	B8
F1	B2	B2	B2	B2	B2	B2	B2	B2	B2*	B2*	B2	B2	B2	B2	B2	B1	B1
F2		B4	B4	B4	B4	B4	B4	B4	B4	B4*	B4	B4	B4	B4	B4	B4	B8
F3			A1	A1	A1*	A1*	A1*	A1*	A1*	A1*	A1	A1	A1*	A1*	A1*	A1*	A1*
F4				A3	A3	A3	A3	A3	A3	A3	A2	A2	A2	A2	A2	A2	A2
F5						C1	C1	C1	C1	C1	C1	A4	A4	A4	A4	A4	A4
F6							C2	C2	C2	C2	C2	C2	C2	C4	C4	C4	C4
F7								B6	B6	B6	B6	B6	B6	B6	C8	C8	C8
¿PF?	X	X	X	X		X	X	X			X	X		X	X	X	X

Marco/Página	C6	C1	C4	C1	A5	A1	A4	B3	B1	B8	CT	A7	A9	A4	BT	AT
F1	B1	B1	B1	B1	B1	B1	A4	A4	A4	A4	A4	A4	A4	A4	A4	
F2	B8	B8	B8	B8	B8	B8	B8	B3	B3	B3	B3	B3	B3	B3		
F3	A1	A1	A1	A1	A1	A1*	A1*	A1*	A1*	A1*	A1*	A1*	A1*	A1*	A1*	
F4	C6	C6	C6	C6	C6	C6	C6	C6	B1	B1	B1	B1	B1	B1		
F5	A4	C1	C1	C1*	C1*	C1*	C1*	C1*	C1*	C1		A7	A7	A7	A7	
F6	C4	C4	C4*	C4*	C4	C4	C4	C4	C4	B8	B8	B8	B8	B8		
F7	C8	C8	C8	C8	A5	A5	A5	A5	A5	A5	A5	A5	A9	A9	A9	
¿PF?	X	X			X		X	X	X	X		X	X			

Queue: B2, B4, A1, A3, C1, C2, B6, B2, B4, A1, A2, A4, C4, C8, B1, B8, A1, C6, C1, C4, A5, A4, B3, A1*, B1, C1, B8, A7, A9.

Por lo tanto, si se utiliza la técnica de selección de víctima *Second Chance*, se producirán 22 fallos de página.

(b) Considerando una política de Asignación Fija con reparto equitativo y Reemplazo Local y disponiéndose de 9 marcos. ¿Cuántos fallos de página se producirán si se utiliza la técnica de selección de víctimas:

- LRU?
- Second Chance?

LRU:

Marco/Página	A1	A3	A1	A2	A4	A1	A5	A1	A4	A7	A9	A4
F1	A1	A1	A1	A1	A1	A1	A1	A1	A1	A1	A9	A9
F2		A3	A3	A3	A4	A4	A4	A4	A4	A4	A4	A4
F3				A2	A2	A2	A5	A5	A5	A7	A7	A7
¿PF?	X	X		X	X		X			X	X	

Marco/Página	B2	B4	B6	B2	B4	B1	B8	B3	B1	B8
F4	B2	B2	B2	B2	B2	B2	B8	B8	B8	B8
F5		B4	B4	B4	B4	B4	B4	B3	B3	B3
F6			B6	B6	B6	B1	B1	B1	B1	B1
¿?	X	X	X			X	X	X		

Marco/Página	C1	C2	C4	C8	C6	C1	C4	C1
F7	C1	C1	C1	C8	C8	C8	C4	C4
F8		C2	C2	C2	C6	C6	C6	C6
F9			C4	C4	C4	C1	C1	C1
¿PF?	X	X	X	X	X	X	X	

Por lo tanto, si se utiliza la técnica de selección de víctima LRU, se producirán 20 fallos de página.

Second Chance:

Marco/Página	A1	A3	A1	A2	A4	A1	A5	A1	A4	A7	A9	A4
F1	A1	A1	A1*	A1*	A1	A1*	A1*	A1*	A1*	A1	A9	A9
F2		A3	A3	A3	A4	A4	A4	A4	A4*	A4	A4	A4*
F3				A2	A2	A2	A5	A5	A5	A7	A7	A7
¿PF?	X	X		X	X		X			X	X	

Queue: A1, A3, A2, A1, A4, A5, A1, A4*, A7, A9.

Marco/Página	B2	B4	B6	B2	B4	B1	B8	B3	B1	B8
F4	B2	B2	B2	B2*	B2*	B2	B8	B8	B8	B8*
F5		B4	B4	B4	B4*	B4	B4	B3	B3	B3
F6			B6	B6	B6	B1	B1	B1	B1*	B1*
¿?	X	X	X			X	X	X		

Queue: B2, B4, B6, B2, B4, B1*, B8*, B3.

Marco/Página	C1	C2	C4	C8	C6	C1	C4	C1
F7	C1	C1	C1	C8	C8	C8	C4	C4
F8		C2	C2	C2	C6	C6	C6	C6
F9			C4	C4	C4	C1	C1	C1*
¿PF?	X	X	X	X	X	X	X	

Queue: C1, C2, C4, C8, C6, C1*, C4.

Por lo tanto, si se utiliza la técnica de selección de víctima *Second Chance*, se producirán 20 fallos de página.

Ejercicio 10.

Sean los procesos A, B y C tales que necesitan, para su ejecución, las siguientes páginas:

- *A: 1, 2, 1, 7, 2, 7, 3, 2.*
- *B: 1, 2, 5, 2, 1, 4, 5.*
- *C: 1, 3, 5, 1, 4, 2, 3.*

Si la secuencia de ejecución es tal que los procesos se ejecutan en la siguiente manera:

- 1. C demanda 1 página.*
- 2. A demanda 2 páginas.*
- 3. C demanda 1 página.*
- 4. B demanda 1 página.*
- 5. A demanda 1 página.*
- 6. C modifica la página 1.*
- 7. B demanda 2 páginas.*
- 8. A demanda 1 página.*
- 9. C demanda 1 página.*
- 10. B modifica la página 2*
- 11. A modifica la página 2.*
- 12. B demanda 2 páginas.*
- 13. A demanda 1 página.*
- 14. B demanda 2 páginas.*
- 15. C demanda 2 páginas.*
- 16. C demanda 1 página.*
- 17. A demanda 1 página.*
- 18. B termina.*
- 19. A demanda 2 páginas.*
- 20. C demanda 1 página.*
- 21. A termina.*
- 22. C termina.*

Considerando una política de Asignación Dinámica y Reemplazo Global y disponiéndose de 7 marcos, debiéndose guardar 1 marco para la gestión de descarga asincrónica de páginas modificadas, ¿cuántos fallos de página se producirán si se utiliza la técnica de selección de víctima:

- *Second Chance?*
- *FIFO?*
- *LRU?*

Second Chance:

Marco/Página	C1	A1	A2	C3	B1	A1	C1M	B2	B5	A7	C5	B2M	A2M	B2
F1	C1	C1	C1	C1	C1	C1	C1 ^{M*}	C1 ^{M*}	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M	-
F2		A1	A1	A1	A1	A1*	A1*	A1*	A1	A1	A1	A1	A1	A1
F3			A2	A2	A2	A2	A2	A2	B5	B5	B5	B5	B5	B5
F4				C3	C3	C3	C3	C3	C3	A7	A7	A7	A7	A7
F5					B1	B1	B1	B1	B1	B1	C5	C5	C5	C5
F6								B2	B2	B2	B2	B2 ^{M*}	B2 ^M	B2 ^{M*}
F7	-	-	-	-	-	-	-	-	-	-	-	-	A2 ^M	A2 ^M
¿PF?	X	X	X	X	X			X	X	X	X		X	

Marco/Página	B1	A2	B4	B5	C1	C4	C2	A7	BT	A3	A2	C3	AT	CT
F1	-	-	-	-	-	-	-	-	-	-	-	-	-	-
F2	B1	B1	B1	B1	B1	C4	C4	C4	C4	C4	C4	C4	C4	C4
F3	B5	B5	B4	B4	B4	B4	C2	C2	C2	C2	C2	C2	C2	C2
F4	A7	A7	A7	B5	B5	B5	B5	A7	A7	A7	A7	A7	A7	A7
F5	C5	C5	C5	C5	C1	C1	C1	C1	C1	C1	C1	C3	C3	C3
F6	B2 ^{M*}	B2 ^{M*}	B2 ^{M*}	B2 ^{M*}	B2 ^{M*}	B2 ^M	B2 ^M	B2 ^M		A3	A3	A3		
F7	A2 ^M	A2 ^{M*}	A2 ^{M*}	A2 ^{M*}	A2 ^{M*}	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^{M*}	A2 ^{M*}		
¿PF?	X		X	X	X	X	X	X		X		X		

Queue: C1, A1, A2, C3, B1, B2, C1, A1, B5, A7, C5, B2, A2, B1, B4, B5, C1, B2, A2*, C4, C2, A7, A3, C3.

Por lo tanto, si se utiliza la técnica de selección de víctima *Second Chance*, se producirán 19 fallos de página.

FIFO:

Marco/Página	C1	A1	A2	C3	B1	A1	C1M	B2	B5	A7	C5	B2M	A2M	B2
F1	C1	C1	C1	C1	C1	C1	C1 ^M	C1 ^M	C1 ^M	-	-	-	-	-
F2		A1	A1	A1	A1	A1	A1	A1	A1	A7	A7	A7	A7	A7
F3			A2	A2	A2	A2	A2	A2	A2	A2	C5	C5	C5	C5
F4				C3	C3	C3	C3	C3	C3	C3	C3	C3	A2 ^M	A2 ^M
F5					B1	B1	B1	B1	B1	B1	B1	B1	B1	B1
F6								B2	B2	B2	B2	B2 ^M	B2 ^M	B2 ^M
F7	-	-	-	-	-	-	-	-	B5	B5	B5	B5	B5	B5
¿PF?	X	X	X	X	X			X	X	X	X		X	

Marco/Página	B1	A2	B4	B5	C1	C4	C2	A7	BT	A3	A2	C3	AT	CT
F1	-	-	-	-	C1	C1	C1	C1	C1	C1	C1	C1	C1	C1
F2	A7	A7	A7	A7	A7	A7	C2	C2	C2	C2	C2	C2	C2	C2
F3	C5	C5	C5	C5	C5	C5	C5	A7	A7	A7	A7	A7	A7	A7
F4	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M		
F5	B1	B1	B4	B4	B4	B4	B4	B4		A3	A3	A3		
F6	B2 ^M	B2 ^M	B2 ^M	B2 ^M	B2 ^M	-	-	-				C3	C3	
F7	B5	B5	B5	B5	B5	C4	C4	C4	C4	C4	C4	C4	C4	
¿PF?			X		X	X	X	X		X		X		

Queue: C1, A1, A2, C3, B1, B2, B5, A7, C5, A2, B4, C1, C4, C2, A7, A3, C3.

Por lo tanto, si se utiliza la técnica de selección de víctima FIFO, se producirán 17 fallos de página.

LRU:

Marco/Página	C1	A1	A2	C3	B1	A1	C1M	B2	B5	A7	C5	B2M	A2M	B2
F1	C1	C1	C1	C1	C1	C1	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M	C1 ^M
F2		A1	A1	A1	A1	A1	A1	A1	A1	A1	A1	A1	A2 ^M	A2 ^M
F3			A2	A2	A2	A2	A2	A2	B5	B5	B5	B5	B5	B5
F4				C3	C3	C3	C3	C3	C3	A7	A7	A7	A7	A7
F5					B1	B1	B1	B1	B1	B1	C5	C5	C5	C5
F6								B2	B2	B2	B2	B2 ^M	B2 ^M	B2 ^M
F7	-	-	-	-	-	-	-	-	-	-	-	-	-	-
¿PF?	X	X	X	X	X			X	X	X	X		X	

Marco/Página	B1	A2	B4	B5	C1	C4	C2	A7	BT	A3	A2	C3	AT	CT
F1	C1 ^M	-	-	-	-	C4	C4	C4	C4	C4	C4	C4	C4	
F2	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	A2 ^M	-	-	-	-		
F3	B5	B5	B4	B4	B4	B4	B4	B4		A3	A3	A3		
F4	A7	A7	A7	B6	B6	B6	B6	B6			A2	A2		
F5	C5	C5	C5	C5	C1	C1	C1	C1	C1	C1	C1	C3	C3	
F6	B2 ^M	B2 ^M	B2 ^M	B2 ^M	B2 ^M	B2 ^M	-	A7	A7	A7	A7	A7		
F7	B1	B1	B1	B1	B1	B1	C2	C2	C2	C2	C2	C2	C2	
¿PF?	X		X	X	X	X	X	X		X	X	X		

Por lo tanto, si se utiliza la técnica de selección de víctima LRU, se producirán 20 fallos de página.

Ejercicio 11: Hiperpaginación (*Thrashing*).

(a) *¿Qué es?*

Es una situación donde el sistema pasa más tiempo realizando *swapping*/paginación entre memoria y disco que ejecutando procesos. Ocurre cuando la demanda de marcos supera, ampliamente, la memoria física disponible y el sistema operativo no puede mantener las páginas necesarias en RAM.

(b) *¿Cuáles pueden ser los motivos que la causan?*

Los motivos que la causan son:

- Multiprogramación excesiva (demasiados procesos compitiendo por pocos marcos).
- Procesos con *Working Set* grande (cada proceso necesita muchas páginas activas).
- Políticas de asignación de marcos inadecuadas (por ejemplo, asignación fija demasiado baja).
- Cambios bruscos en el patrón de acceso (localidad que cambia rápido).
- Falta general de memoria física.

(c) *¿Cómo la detecta el SO?*

El sistema operativo, normalmente, la detecta observando:

- Aumento drástico de la tasa de *page faults*.
- Caída fuerte del uso del CPU (el CPU queda ocioso esperando E/S de paginación).
- Aumento del tráfico de E/S hacia el disco por *swapping* continuo.
- *Working Set* de muchos procesos mayor a la memoria total (en modelos de *Working Set*).

Si la mayor parte del tiempo se gasta en *page faults*, hay *thrashing*.

(d) *Una vez que la detecta, ¿qué acciones puede tomar el Kernel para eliminar este problema?*

Las acciones que puede tomar el *Kernel* para eliminar este problema son:

- Reducir el grado de multiprogramación: Suspender procesos, enviarlos a *swapped out*, o no aceptar nuevos hasta estabilizar el sistema.
- Asignar más marcos a los procesos activos: Si usa políticas adaptativas (por ejemplo, PFF -*Page Fault Frequency*- o *Working Set*).

- Controlar la frecuencia de fallos de página (PFF): Si un proceso supera un umbral de *page faults*, se le asignan más marcos. Si está por debajo, se le quitan.
- Utilizar el modelo del *Working Set*: Asignar marcos suficientes para cubrir el conjunto de páginas activas del proceso.
- Reemplazo de página más eficiente: LRU, *Clock*, variantes de *WSClock*, etc.
- Priorizar procesos interactivos: Darles más marcos para evitar que se “congelan”.

Ejercicio 12.

Analizar y comparar las técnicas de PFF (Page Fault Frequency) y del Working Set. ¿Cómo las mismas permiten determinar la existencia de thrashing?

A continuación, se analizan y comparan las técnicas de PFF (*Page Fault Frequency*) y del *Working Set*:

1. PFF (*Page Fault Frequency*):

- Controla la frecuencia de fallos de página de cada proceso. Si un proceso tiene muchos fallos, necesita más marcos. Si un proceso tiene muy pocos fallos, se le pueden quitar marcos. Cada proceso debe mantener su *page fault rate* dentro de un rango aceptable.
- Esta técnica permite determinar la existencia de *thrashing* de la siguiente manera:
 - Si muchos procesos empiezan a tener frecuencias de fallos muy altas al mismo tiempo, significa que todos están compitiendo por muy pocos marcos, el disco está siendo usado constantemente y el CPU queda ocioso.
 - Entonces, el sistema operativo concluye que hay *thrashing*.
- En conclusión, esta técnica detecta *thrashing* cuando la frecuencia de fallos crece por encima de un umbral. Es una técnica reactiva, basada en el comportamiento del *page fault rate*.

2. Working Set:

- Cada proceso tiene un conjunto de páginas activas en un intervalo reciente de tiempo. Ese conjunto es su *Working Set*. El sistema operativo debe asignar, al menos, tantos marcos como páginas tenga su *Working Set*.
- Esta técnica permite determinar la existencia de *thrashing* de la siguiente manera:
 - Se suman los tamaños de los *Working Sets* de todos los procesos.
 - Si la suma es mayor que la memoria física, el sistema no puede satisfacer la demanda real de memoria.
 - Entonces, inevitablemente, ocurre *thrashing*.
- En conclusión, esta técnica detecta *thrashing* cuando la memoria no alcanza para sostener los *Working Sets* activos. Es una técnica basada en la localidad y en el uso real.

Ejercicio 13.

Considerar un sistema cuya memoria principal se administra mediante la técnica de paginación por demanda que utiliza un dispositivo de paginación, algoritmo de reemplazo global LRU y una política de asignación que reparte marcos, equitativamente, entre los procesos. El nivel de multiprogramación es, actualmente, de 4.

Analizar, para cada caso:

- *¿Qué sucede?*
- *¿Puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU?*
- *¿La paginación está siendo útil para mejorar el rendimiento del sistema?*

(a) *Uso de CPU del 13%, uso del dispositivo de paginación del 97%.*

Esto es el clásico cuadro de hiperpaginación (*thrashing*). El dispositivo de paginación está casi saturado (97%) transfiriendo páginas: la mayor parte del tiempo del sistema se consume en traer/mandar páginas al disco, mientras que el CPU está, mayormente, ocioso (13%). Con asignación equitativa, cada proceso tiene pocos marcos efectivos y provoca muchos *page faults*. El LRU global está moviendo páginas constantemente.

No, no puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU. Subir el número de procesos empeoraría la situación: habría aún más competencia por los mismos marcos, más *page faults* y mayor paginación. La acción opuesta (reducir el nivel de multiprogramación) es la indicada.

No, la paginación no está siendo útil para mejorar el rendimiento del sistema. En este escenario, la paginación está perjudicando el rendimiento (gasta E/S en mover páginas en lugar de ejecutar instrucciones útiles). Es la causa del bajo uso de la CPU.

(b) *Uso de CPU del 87%, uso del dispositivo de paginación del 3%.*

El sistema está *CPU-bound*: el procesador está ocupado (87%) y el dispositivo de paginación casi inactivo (3%). Los *page faults* son pocos y la paginación no es cuello de botella. El LRU global y la asignación actual parecen suficientes para mantener las páginas necesarias en memoria.

Sí, puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU, pero no es tan necesario porque, actualmente, el CPU está bastante ocupado. Hay margen para añadir procesos porque, actualmente, la paginación no está activa. Sin embargo, aumentar la multiprogramación puede, si se exceden los marcos libres, elevar la tasa de *page faults* y degradar rendimiento. Antes de aumentarla, conviene comprobar cuántos marcos libres quedan y, tras añadir procesos, monitorizar la tasa de *page faults*.

Sí, la paginación está siendo útil para mejorar el rendimiento del sistema. El dispositivo de paginación se usa poco porque no se necesita mucho, pero está funcionando bien (mantiene las páginas sin provocar *overhead*).

(c) *Uso de CPU del 13%, uso del dispositivo de paginación del 3%.*

Uso de CPU bajo (13%), pero uso del dispositivo de paginación también bajo (3%). Esto indica no *thrashing*. El CPU está inactivo por otras razones: procesos bloqueados en E/S no relacionado con *swapping* (por ejemplo, esperando disco de datos, red, entrada humana), espera por sincronización (*locks*) o, simplemente, no hay suficientes procesos listos para ejecutar. No hay saturación del *swap*.

Sí, puede incrementarse el nivel de multiprogramación para aumentar el uso de la CPU, pero con precaución. Al igual que en el caso anterior, hay margen para añadir procesos porque, actualmente, la paginación no está activa. Sin embargo, caben las mismas aclaraciones que en el caso anterior.

Sí, la paginación está siendo útil para mejorar el rendimiento del sistema. El dispositivo de paginación se usa poco porque no se necesita mucho, pero está funcionando bien (mantiene las páginas sin provocar *overhead*).

Ejercicio 14.

Considerar un sistema cuya memoria principal se administra mediante la técnica de paginación por demanda. Considerar las siguientes medidas de utilización:

- *Utilización del procesador: 20%.*
- *Utilización del dispositivo de paginación: 97,7%.*
- *Utilización de otros dispositivos de E/S: 5%.*

¿Cuáles de las siguientes acciones pueden mejorar la utilización del procesador?:

- *Instalar un procesador más rápido.*
- *Instalar un dispositivo de paginación mayor.*
- *Incrementar el grado de multiprogramación.*
- *Instalar más memoria principal.*
- *Decrementar el quantum para cada proceso.*

Puede mejorar la utilización del procesador instalar más memoria principal, ya que más RAM permite mantener más páginas en memoria y reduce *page faults* y la carga sobre el dispositivo de paginación: con menos paginación, el CPU podrá ejecutar más instrucciones y su utilización subirá.

Ejercicio 15.

La siguiente fórmula describe el tiempo de acceso efectivo a la memoria al utilizar paginación para la implementación de la memoria virtual:

$$TAE = A_t + [(1 - p) * A_m] + [p * (T_f + A_m)],$$

donde:

- TAE = tiempo de acceso efectivo.
- p = tasa de fallo de página ($0 \leq p \leq 1$).
- A_m = tiempo de acceso a la memoria real.
- T_f = tiempo de atención de un fallo de página.
- A_t = tiempo de acceso a la tabla de páginas. Es igual al tiempo de acceso a la memoria (A_m) si la entrada de la tabla de páginas no se encuentra en la TLB.

Suponer que se tiene una memoria virtual paginada, con tabla de páginas de 1 nivel y donde la tabla de páginas se encuentra, completamente, en la memoria.

Suponer que una falla de página tarda 300 nanosegundos si hay disponible un marco vacío o si la página reemplazada no se ha modificado, y 500 nanosegundos si se ha modificado. El tiempo de acceso a memoria es de 20 nanosegundos y el de acceso a la TLB es de 1 nanosegundo.

(a) Si se supone una tasa de fallos de página de 0,3 y que siempre se cuenta con un marco libre para atender el fallo, ¿cuál será el TAE si el 50% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit)?

$$T_f = 300 \text{ ns.}$$

$$A_m = 20 \text{ ns.}$$

$$p = 0,3.$$

$$A_t = 0,5 * 1 + (1 - 0,5) (1 + 20)$$

$$A_t = 0,5 + 0,5 * 21$$

$$A_t = 0,5 + 10,5$$

$$A_t = 11 \text{ ns.}$$

$$TAE = 11 + [(1 - 0,3) * 20] + [0,3 (300 + 20)]$$

$$TAE = 11 + (0,7 * 20) + (0,3 * 320)$$

$$TAE = 11 + 14 + 96$$

$$TAE = 121 \text{ ns.}$$

Por lo tanto, si se supone una tasa de fallos de página de 0,3 y que siempre se cuenta con un marco libre para atender el fallo, y si el 50% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit), el TAE será 121 ns.

(b) Si se supone una tasa de fallos de página de 0,3 y que el 70% de las ocasiones la página a reemplazar se encuentra modificada, ¿cuál será el TAE si el 60% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit)?

$$A_m = 20 \text{ ns.}$$

$$p = 0,3.$$

$$T_f = 0,7 * 500 + (1 - 0,7) * 300$$

$$T_f = 350 + 0,3 * 300$$

$$T_f = 350 + 90$$

$$T_f = 440 \text{ ns.}$$

$$A_t = 0,6 * 1 + (1 - 0,6) (1 + 20)$$

$$A_t = 0,6 + 0,4 * 21$$

$$A_t = 0,6 + 8,4$$

$$A_t = 9 \text{ ns.}$$

$$\text{TAE} = 9 + [(1 - 0,3) * 20] + [0,3 (440 + 20)]$$

$$\text{TAE} = 9 + (0,7 * 20) + (0,3 * 460)$$

$$\text{TAE} = 9 + 14 + 138$$

$$\text{TAE} = 161 \text{ ns.}$$

Por lo tanto, si se supone una tasa de fallos de página de 0,3 y que el 70% de las ocasiones la página a reemplazar se encuentra modificada, y si el 60% de las veces la entrada de la tabla de páginas se encuentra en la TLB (hit), el TAE será 161 ns.

(c) Si se supone que el 60% de las veces la página a reemplazar está modificada, el 100% de las veces la entrada de la tabla de páginas requerida se encuentra en la TLB (hit) y se espera un TAE menor a 200 nanosegundos, ¿cuál es la máxima tasa aceptable de fallos de página?

$$A_m = 20 \text{ ns.}$$

$$A_t = 1.$$

$$T_f = 0,6 * 500 + (1 - 0,6) * 300$$

$$T_f = 300 + 0,4 * 300$$

$$T_f = 300 + 120$$

$$T_f = 420 \text{ ns.}$$

$$200 = 1 + [(1 - p) * 20] + [p (420 + 20)]$$

$$200 = 1 + 20 - 20p + 440p$$

$$200 = 21 + 420p$$

$$420p = 200 - 21$$

$$420p = 179$$

$$p = \frac{179}{420}$$

$$p \cong 0,426.$$

Por lo tanto, si se supone que el 60% de las veces la página a reemplazar está modificada, el 100% de las veces la entrada de la tabla de páginas requerida se encuentra en la TLB (hit) y se espera un TAE menor a 200 nanosegundos, la máxima tasa aceptable de fallos de página es, aproximadamente, 0,426.

Ejercicio 16.

Considerar el siguiente programa:

```
#define Size 64
int A[Size; Size], B[Size; Size], C[Size;
Size]; int register i, j;
for (j = 0; j < Size; j++)
    for (i = 0; i < Size; i++)
        C[i; j] = A[i; j] + B[i; j];
```

Si se asume que el programa se ejecuta en un sistema que utiliza paginación por demanda para administrar la memoria, donde cada página es de 1 KB. Cada número entero (int) ocupa 4 bytes. Es claro que cada matriz requiere de 16 páginas para almacenarse. Por ejemplo: $A[0,0]..A[0,63]$, $A[1,0]..A[1,63]$, $A[2,0]..A[2,63]$ y $A[3,0]..A[3,63]$ se almacenará en la primera página.

Se asume que el sistema utiliza un working set de 4 marcos para este proceso. Uno de los 4 marcos es utilizado por el programa y los otros 3 se utilizan para datos (las matrices). También se considera que, para los índices “i” y “j”, se utilizan 2 registros, por lo que no es necesario el acceso a la memoria para estas 2 variables.

(a) Analizar cuántos fallos de páginas ocurren al ejecutar el programa (considerar las veces que se ejecuta $C[i, j] = A[i, j] + B[i, j]$).

PF= fallos por bloque * bloques por iteración j * iteraciones j
 PF= 3 * 16 * 64
 PF= 3072.

Por lo tanto, al ejecutar el programa, ocurren 3.072 fallos de página.

(b) ¿Puede ser modificado el programa para minimizar el número de fallos de páginas? En caso de ser posible, indicar la cantidad de fallos de páginas que ocurren.

Sí, el programa puede ser modificado para minimizar el número de fallos de página. En particular, cambiando el orden de los bucles para seguir el orden de almacenamiento por filas, y no por columnas, de la siguiente manera:

```
for (i = 0; i < Size; i++)
    for (j = 0; j < Size; j++)
        C[i][j] = A[i][j] + B[i][j];
```

PF= fallos por bloque * bloques
 PF= 3 * 16
 PF= 48.

Por lo tanto, en este caso, la cantidad de fallos de página que ocurren es 48.

Ejercicio 17.

Considerar las siguientes secuencias de referencias a páginas de los procesos A y B, donde se muestra el instante de tiempo en el que ocurrió cada una (1 a 78):

Proceso A																																							
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	
1	2	1	3	4	3	3	2	1	1	3	4	5	6	1	2	1	3	3	4	5	6	6	6	5	4	3	1	1	2	3	4	5	4	3	2	1	1	1	
4	4	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
2	2	1	2	1	1	2	3	4	5	6	7	8	6	5	3	1	2	3	4	5	6	5	1	1	2	3	4	5	4	2	1	1	2	3	4	5	1	1	

Proceso B																																							
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	
1	2	3	3	3	4	4	5	1	1	1	2	3	4	4	4	4	1	1	2	3	6	5	6	5	4	3	1	1	1	1	4	5	1	3	2	1	1	2	
4	4	4	4	4	4	4	4	4	4	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
2	2	3	3	4	4	4	3	4	5	4	6	1	1	1	2	3	3	3	3	4	5	5	5	1	6	2	3	1	2	3	1	1	1	2	2	2	2	3	3

(a) Considerando una ventana $\Delta = 5$, indicar cuál sería el conjunto de trabajo de los procesos A y B en el instante 24 ($WSA(24)$ y $WSB(24)$).

Intervalo de búsqueda = [20, 21, 22, 23, 24].

$WSA(24) = \{4, 5, 6\}$.

$WSB(24) = \{2, 3, 5, 6\}$.

Por lo tanto, considerando una ventana $\Delta = 5$, el conjunto de trabajo de los procesos A y B en el instante 24 es $\{4, 5, 6\}$ y $\{2, 3, 5, 6\}$, respectivamente.

(b) Considerando una ventana $\Delta = 5$, indicar cuál sería el conjunto de trabajo de los procesos A y B en el instante 60 ($WSA(60)$ y $WSB(60)$).

Intervalo de búsqueda = [56, 57, 58, 59, 60].

$WSA(60) = \{1, 2, 3, 4, 5\}$.

$WSB(60) = \{3, 4, 5\}$.

Por lo tanto, considerando una ventana $\Delta = 5$, el conjunto de trabajo de los procesos A y B en el instante 60 es $\{1, 2, 3, 4, 5\}$ y $\{3, 4, 5\}$, respectivamente.

(c) Para el/los WS obtenidos en el inciso (a), si se cuenta con 8 frames en el sistema, ¿se puede indicar que se está ante una situación de thrashing? ¿Y si se cuenta con 6 frames?

Dados los WS obtenidos en el inciso (a) (con una demanda total de 7 frames):

- Si se cuenta con 8 frames en el sistema, no se está ante una situación de thrashing, ya que la oferta supera a la demanda.

- Si se cuenta con 6 *frames* en el sistema, se está ante una situación de *thrashing*, ya que la demanda supera a la oferta.

(d) Considerando, únicamente, el proceso A y suponiendo que al mismo se le asignaron, inicialmente, 4 marcos, donde el reemplazo de páginas es realizado considerando el algoritmo FIFO. ¿Cuál será la tasa de fallos de páginas en el instante 38 suponiendo que la misma se calcula contando los fallos de páginas que ocurrieron en las últimas 10 unidades de tiempo?

Tiempo	29	30	31	32	33	34	35	36	37	38
Marcos/Página	1	2	3	4	5	4	3	2	1	1
F1	1	1	1	1	5	5	5	5	5	5
F2	4	2	2	2	2	2	2	2	1	1
F3	5	5	3	3	3	3	3	3	3	3
F4	6	6	6	4	4	4	4	4	4	4
¿PF?		X	X	X	X				X	

$$\text{Tasa de fallos} = \frac{5}{10}$$

$$\text{Tasa de fallos} = 0,5.$$

Por lo tanto, la tasa de fallos de páginas en el instante 38, suponiendo que la misma se calcula contando los fallos de páginas que ocurrieron en las últimas 10 unidades de tiempo, es 0,5.

(e) Para el valor obtenido en el inciso (d), si se supone que el SO utiliza, como límites superior e inferior de tasa de fallos de páginas, los valores 2 y 5, respectivamente, ¿qué acción podría tomar el Kernel respecto a la cantidad de marcos asignados al proceso?

Dado que la tasa de fallos alcanza el límite superior, la acción que podría tomar el *Kernel* respecto a la cantidad de marcos asignados al proceso es diagnosticar que el proceso no tiene suficientes marcos para su conjunto de trabajo y está propenso al *thrashing*. El *Kernel* intentará aumentar la cantidad de marcos asignados al proceso A (por ejemplo, de 4 a 5) y, al aumentar el conjunto residente del proceso, se espera que la tasa de fallos caiga por debajo del límite superior, mejorando la eficiencia y evitando el *thrashing*.

PARTE II: Administración de E/S.**Ejercicio 18: Dispositivos.**

Los dispositivos, según la forma de transferir los datos, se pueden clasificar en 2 tipos:

- *Orientados a bloques.*
- *Orientados a flujos.*

(a) *Describir las diferencias entre ambos tipos.*

Las diferencias entre ambos tipos de dispositivos son:

1. Orientados a bloques:
 - Transfieren datos en bloques de tamaño fijo (por ejemplo, 512 bytes, 4 KB).
 - Acceso aleatorio: Se puede leer/escribir, directamente, en cualquier bloque sin recorrer los anteriores.
 - Son ideales para almacenamiento donde es importante acceder, rápidamente, a posiciones específicas.
 - Típicamente, gestionados por el sistema operativo como un conjunto de bloques numerados.
2. Orientados a flujos:
 - Transfieren datos como un flujo continuo de bytes, sin estructura fija.
 - Acceso secuencial: Para llegar a cierto punto del flujo, hay que pasar por los datos anteriores.
 - El sistema no maneja bloques, sino un flujo tipo “tubería” de datos.
 - Se usan para datos que se consumen o producen de manera continua.

(b) *Citar ejemplos de dispositivos de ambos tipos.*

- Ejemplos de dispositivos orientados a bloques: discos rígidos (HDD), SSD, memorias USB, tarjetas SD.
- Ejemplos de dispositivos orientados a flujos: teclado, ratón, *stream* de red, dispositivos de audio, sensores, puertos serie.

(c) *Enunciar las diferencias que existen entre los dispositivos de E/S y que el Kernel debe considerar.*

El *Kernel* debe manejar muchos tipos de dispositivos y, para hacerlo correctamente, tiene que considerar varias diferencias fundamentales entre ellos. Entre las más importantes, se encuentran:

- Velocidad de transferencia: lentos / rápidos.
- Tipo de transferencia: orientados a bloques / orientados a flujos.

- Modo de acceso: aleatorio / secuencial.
- Sincronización y temporización: síncrona / asíncrona.
- Naturaleza del dispositivo: almacenamiento / comunicación / entrada humana / sensores.
- Dirección y tamaño de datos: bytes pequeños / bloques grandes.
- Métodos de notificación: interrupciones / *polling* / DMA.
- Confiabilidad y manejo de errores: fiables / no fiables.
- Compartición: compartibles / no compartibles.
- Persistencia: datos persisten / datos no persisten.

Ejercicio 19: Técnicas de E/S.

Describir cómo trabajan las siguientes técnicas de E/S:

(a) *E/S programada.*

El CPU verifica, continuamente (*polling*), si el dispositivo está listo y copia los datos, byte a byte, entre el dispositivo y la memoria. El CPU trabaja todo el tiempo.

(b) *E/S dirigida por interrupciones.*

El CPU le pide al dispositivo que haga la operación y sigue ejecutando otras tareas. Cuando el dispositivo termina, envía una interrupción y el CPU copia los datos. El CPU trabaja sólo al principio y al final.

(c) *DMA (Acceso Directo a Memoria).*

Un módulo especial (el controlador DMA) transfiere datos entre memoria y dispositivo sin intervención del CPU. El CPU sólo inicia la operación y el DMA hace todo.

Ejercicio 20.

La técnica de E/S programa puede trabajar de dos formas:

- *E/S mapeada.*
- *E/S aislada.*

Indicar cómo trabajan estas 2 técnicas.

A continuación, se indica cómo trabajan estas 2 técnicas:

1. E/S mapeada: En esta técnica, los registros de los dispositivos de E/S (por ejemplo, “estado”, “control”, “dato”) se asignan a direcciones dentro del espacio de memoria del sistema. Esto significa que:
 - El CPU lee y escribe en los dispositivos usando las mismas instrucciones que usa para acceder a la memoria (LOAD, STORE).
 - No existe una instrucción especial para E/S.
 - El *hardware* del sistema distingue si una dirección corresponde a memoria real o a un dispositivo.
2. E/S aislada: En este caso, los dispositivos no comparten el espacio de direcciones de la memoria. Tienen un espacio separado, llamado “puertos de E/S”. Para acceder a esos puertos:
 - El CPU debe usar instrucciones especiales (por ejemplo, IN y OUT en la arquitectura x86).
 - La E/S está “aislada” y separada del acceso a memoria común.

Ejercicio 21.

Enunciar las metas que debe perseguir un SO para la administración de la entrada/salida.

Las metas que debe perseguir un sistema operativo para la administración de la entrada/salida son:

- Eficiencia: Aprovechar al máximo los dispositivos, reduciendo tiempos de espera y evitando que el CPU quede bloqueado.
- Equidad (*fairness*): Asegurar que todos los procesos tengan acceso razonable a los dispositivos, evitando que uno monopolice la E/S.
- Independencia del dispositivo: Que los programas puedan usar dispositivos sin conocer sus detalles físicos (*drivers*, diferencias de *hardware*, etc.).
- Uniformidad en la interfaz: Proveer una forma común de acceder a distintos dispositivos, simplificando el trabajo del programador.
- Concurrencia y multiplexación: Permitir que varios procesos usen dispositivos simultáneamente o que el sistema operativo intercale peticiones de forma ordenada.
- Manejo de errores: Detectar, informar y, si es posible, recuperar errores de dispositivos o transferencias de E/S.
- Protección y seguridad: Evitar accesos indebidos a dispositivos o datos y garantizar que un proceso no interfiera con otro.
- Buffering y manejo eficiente de memoria: Utilizar *buffers* para optimizar la velocidad entre CPU, memoria y dispositivos más lentos.

Ejercicio 22: Drivers.

(a) *¿Qué son?*

Los *drivers* (o controladores de dispositivo) son programas de bajo nivel que permiten que el sistema operativo se comunice con un dispositivo de *hardware*. Actúan como traductores entre el sistema operativo y el dispositivo, ocultando los detalles específicos del *hardware*.

(b) *¿Qué funciones mínimas deben proveer?*

Las funciones mínimas que deben proveer son:

- Inicialización del dispositivo: Configurarlos para que quede listo para operar.
- Lectura y escritura de datos: Proveer rutinas para enviar datos al dispositivo o recibirlos de él.
- Manejo de interrupciones: Responder cuando el dispositivo genera una interrupción.
- Control del dispositivo: Operaciones especiales (cambios de modo, comandos específicos, *reset*, etc.).
- Reporte y manejo de errores: Detectar fallas y comunicarlas al sistema operativo.

(c) *¿Quién determina cuáles deben ser estas funciones?*

El que determina cuáles deben ser estas funciones es el sistema operativo, que establece la API o interfaz estándar que todos los *drivers* deben implementar.

Ejercicio 23.

Realizar un gráfico que marque la relación entre el Subsistema de E/S, los drivers, los controladores de dispositivos y los dispositivos.

- El Subsistema de E/S del sistema operativo gestiona la entrada/salida de forma general.
- Los *drivers* son el puente entre el sistema operativo y el *hardware*.
- Los controladores de dispositivos son *hardware* que reciben órdenes del *driver*.
- El dispositivo es el aparato físico final.

Ejercicio 24.

Describir, mediante un ejemplo, los pasos mínimos que se suceden desde que un proceso genera un requerimiento de E/S hasta que el mismo llega al dispositivo.

Se supone que un proceso necesita leer un archivo. Los pasos mínimos que ocurren son:

1. El proceso solicita la operación: El proceso ejecuta una llamada al sistema (por ejemplo, *read()*). Esto pasa del modo usuario al modo *Kernel*.
2. El Subsistema de E/S recibe el pedido: El *Kernel* verifica qué dispositivo es, si está disponible, si necesita usar un *buffer* y si la operación requiere acceso bloqueante o no. Crea una solicitud de E/S y la coloca en la cola del dispositivo.
3. Se invoca al *driver* del dispositivo: El subsistema de E/S llama al *driver* correspondiente (por ejemplo, *driver* de disco). El *driver*: traduce el pedido del sistema operativo en comandos específicos del dispositivo; prepara los parámetros que necesita el *hardware* (sector, dirección, tamaño, etc.).
4. El *driver* ordena la operación al controlador del dispositivo: El *driver* escribe, en los registros del controlador (*hardware*), los datos necesarios (dirección del bloque a leer, cantidad de bytes, dirección de memoria donde almacenar el resultado).
5. El controlador ejecuta la operación: El controlador de disco toma los comandos y los envía al dispositivo físico. El dispositivo inicia la operación (por ejemplo, mover cabezales y leer el sector).
6. El dispositivo realiza la operación: El disco lee el bloque solicitado y, cuando termina, envía una interrupción al CPU avisando que la lectura finalizó.
7. El *driver* atiende la interrupción: El CPU detiene lo que estaba haciendo y ejecuta la rutina de interrupción del *driver*. El *driver*: confirma que la operación terminó; copia (o valida) los datos en el *buffer* del sistema operativo; marca la solicitud como completada.
8. El Subsistema de E/S devuelve el resultado al proceso: El *Kernel* despierta o reanuda al proceso que estaba esperando el dato. El proceso recibe el bloque leído y continúa ejecutando.

Ejercicio 25.

Enunciar qué servicios provee el Kernel para la administración de E/S.

Los servicios que provee el *Kernel* para la administración de E/S son:

1. Interfaz de llamadas al sistema (*syscalls*): Brinda funciones para que los procesos soliciten operaciones de E/S (*open, read, write, close*, etc.).
2. Administración de dispositivos: Identifica, inicializa y gestiona todos los dispositivos conectados.
3. Planificación de E/S: Ordena y optimiza las solicitudes de E/S (colas de pedidos, algoritmos de *scheduling*).
4. Buffering y caching: Usa *buffers* y cachés para mejorar rendimiento y reducir accesos al dispositivo.
5. Spooling: Maneja dispositivos compartidos (como impresoras), almacenando trabajos en cola.
6. Manejo de interrupciones: Atiende interrupciones generadas por los dispositivos y despierta procesos bloqueados.
7. Control y comunicación con *drivers*: Llama a los *drivers* y les pasa las solicitudes de E/S. Actúa como interfaz estándar entre ellos y los procesos.
8. Protección y seguridad: Controla qué proceso puede usar qué dispositivo y valida accesos.
9. Detección y manejo de errores de E/S: Registra, informa y trata de recuperar fallos en dispositivos o transferencias.
10. Abstracción de dispositivos: Presenta una interfaz uniforme (archivos, puertos, *sockets*), independientemente del *hardware* físico.

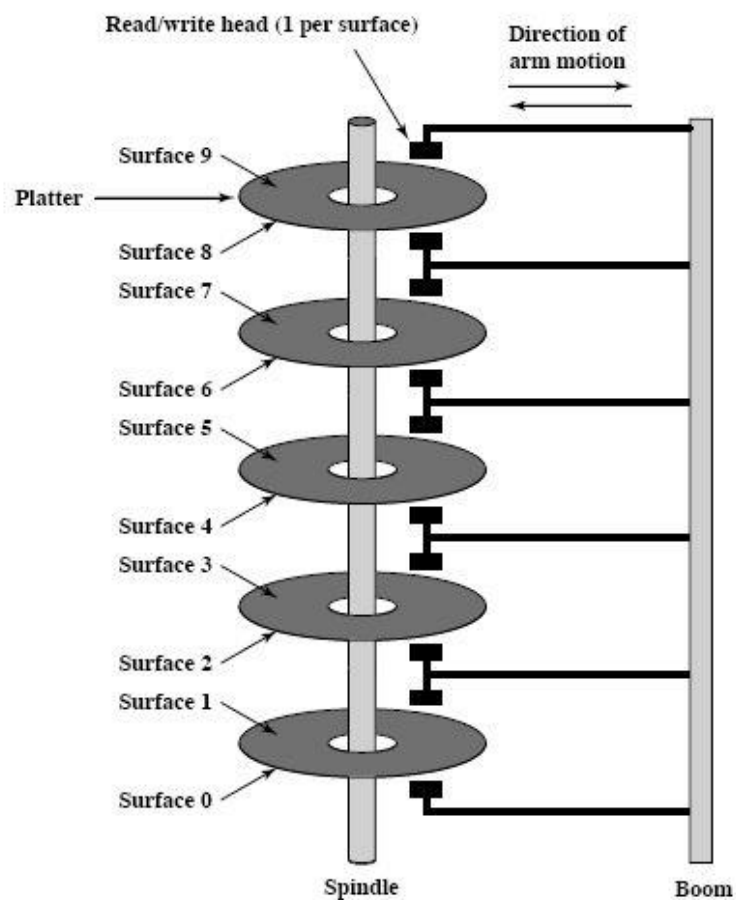
PARTE III: Administración de Discos.**Ejercicio 26.**

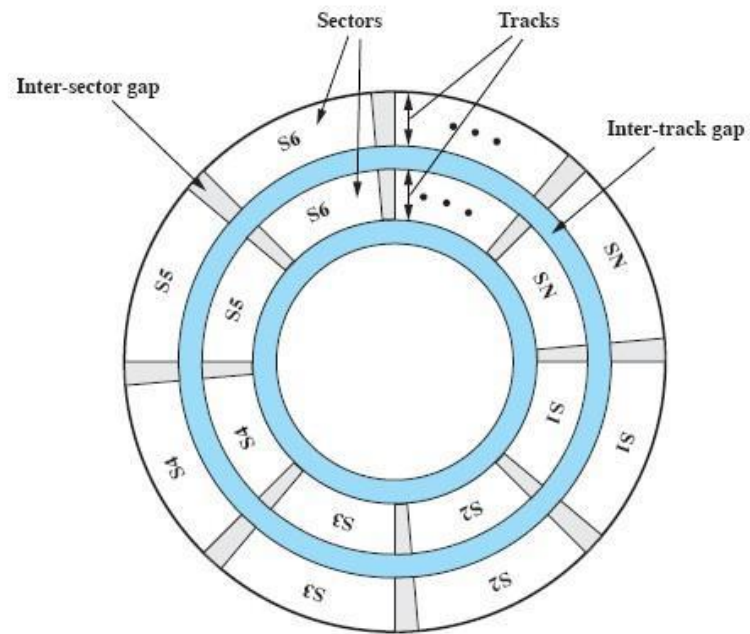
Describir, en forma sintética, cómo es la organización física de un disco. Se pueden utilizar gráficos para mayor claridad.

Un disco rígido (HDD) está formado por uno o más platos metálicos que giran a gran velocidad. Cada plato tiene dos superficies donde se puede guardar información.

La organización física es jerárquica:

1. Platos: Son discos circulares donde se almacena la información.
2. Pistas (tracks): Cada plato está dividido en círculos concéntricos llamados pistas.
3. Sectores: Cada pista se divide en segmentos llamados sectores. Son la unidad mínima física de lectura (típicamente, 512 bytes o 4 KB).
4. Cilindros: Son el conjunto de pistas alineadas, verticalmente, en todos los platos.
5. Cabezal (head): Cada superficie del plato tiene un cabezal que lee y escribe los datos. Todos los cabezales se mueven en conjunto, por lo cual se usan los cilindros como unidad lógica importante.





Ejercicio 27.

La velocidad promedio para la obtención de datos de un disco está dada por la suma de los siguientes tiempos:

- *Seek Time.*
- *Latency Time.*
- *Transfer Time.*

Dar una definición para cada uno.

La velocidad promedio para la obtención de datos de un disco está dada por la suma de los siguientes tiempos:

1. *Seek Time* (posicionamiento): Es el tiempo que tarda el cabezal del disco en moverse hasta la pista donde se encuentran los datos. Es el componente más lento y depende de la distancia que debe recorrer el cabezal.
2. *Latency Time* (latencia): Es el tiempo que tarda el disco en girar hasta que el sector deseado quede debajo del cabezal. Depende de la velocidad de rotación del disco (RPM).
3. *Transfer Time*: Es el tiempo necesario para leer o escribir los datos una vez que el cabezal ya está en el lugar correcto. Es el más rápido de los tres y depende del ancho de banda del disco.

Ejercicio 28.

Suponer un disco con las siguientes características:

- 7 platos con 2 caras utilizables cada uno.
- 1.100 cilindros, con 300 sectores por pista, donde cada sector es de 512 bytes.
- Seek time de 10 ms.
- 9.000 RPM.
- Velocidad de transferencia de 10 MiB/s (MebiBytes por segundo).

(a) *Calcular la capacidad total del disco.*

Capacidad= 512 bytes * sectores/pista * cilindros * cabezas

Capacidad= 512 bytes * 300 * 1100 * 14

Capacidad= 2365440000 bytes.

Por lo tanto, la capacidad total del disco es 2.365.440.000 bytes.

(b) *¿Cuántos sectores de disco se requerirían si se quiere almacenar 3 MiB (MebiBytes) de datos?*

3 MiB= 3 * 1048576 bytes

3 MiB= 3145728 bytes.

Sectores= $\frac{3145728 \text{ bytes}}{512 \text{ bytes}}$

Sectores= 6144.

Por lo tanto, si se quiere almacenar 3 Mib (MebiBytes) de datos, se requerirían 6.144 sectores de disco.

(c) *Calcular el tiempo de latencia.*

Latencia= $0,5 \frac{1}{RPMs}$ ms

Latencia= $0,5 \frac{1}{\frac{9000}{60 \times 1000}}$ ms

Latencia= $0,5 \frac{1}{\frac{9000}{60000}}$ ms

Latencia= $0,5 \frac{1}{0,15}$ ms

Latencia= $0,5 * 6, \hat{6}$ ms

Latencia= $3, \hat{3}$ ms.

Por lo tanto, el tiempo de latencia es $3, \hat{3}$ ms.

Ejercicio 29.

Originalmente, los sectores de un disco se identificaban por la terna de valores (C, H, S) esto es Cylinder-Head-Sector (Cilindro, Cabeza, Sector), con C desde 0 hasta la cantidad de cilindros menos 1, H desde 0 hasta la cantidad de cabezas lecto/grabadoras menos 1 y S desde 1 hasta la cantidad de sectores por pista.

En los '90, los controladores de disco reemplazaron la identificación de los sectores por un modelo numérico denominado LBA (Logical Block Addressing). La numeración comenzaba en 0 y alcanzaba la cantidad total de sectores menos 1. Cada sector en formato (C, H, S) se puede traducir al LBA a partir de la siguiente fórmula:

$$A = (C * N_{heads} + H) * N_{sectors} + (S - 1),$$

donde:

- A es el valor LBA a obtener.
- N_{heads} es la cantidad de cabezas lecto/grabadoras.
- $N_{sectors}$ es la cantidad de sectores por pista.

También se puede obtener el valor de la terna (C, H, S) a partir de un valor LBA con las siguientes fórmulas:

- $C = LBA \text{ DIV } (N_{heads} * N_{sectors})$.
- $H = [LBA \text{ MOD } (N_{heads} * N_{sectors})] \text{ DIV } N_{sectors}$.
- $S = [LBA \text{ MOD } (N_{heads} * N_{sectors})] \text{ MOD } N_{sectors} + 1$.

(a) Si se tiene un disco con 1.020 cilindros, 16 cabezas y 63 sectores por pista, calcular la cantidad total de sectores que tendrá el disco y determinar el valor LBA para las siguientes ternas (C, H, S):

Sectores= sectores/pista * cilindros * cabezas
 Sectores= 63 * 1020 * 16
 Sectores= 1028160.

(i) (3, 2, 1).

$A = (3 * 16 + 2) * 63 + (1 - 1)$
 $A = (48 + 2) * 63 + 0$
 $A = 50 * 63 + 0$
 $A = 3150 + 0$
 $A = 3150$.

(ii) (0, 0, 1).

$A = (0 * 16 + 0) * 63 + (1 - 1)$

$$A = (0 + 0) * 63 + 0$$

$$A = 0 * 63 + 0$$

$$A = 0 + 0$$

$$A = 0.$$

(iii) (1019, 15, 63).

$$A = (1019 * 16 + 15) * 63 + (63 - 1)$$

$$A = (16304 + 15) * 63 + 62$$

$$A = 16319 * 63 + 62$$

$$A = 1028097 + 62$$

$$A = 1028159.$$

(iv) (500, 8, 44).

$$A = (500 * 16 + 8) * 63 + (44 - 1)$$

$$A = (8000 + 8) * 63 + 43$$

$$A = 8008 * 63 + 43$$

$$A = 504504 + 43$$

$$A = 504547.$$

(b) Determinar el valor de la terna (C, H, S) para los siguientes valores LBA:

(i) 1.

$$C = 1 \text{ DIV } (16 * 63)$$

$$C = 1 \text{ DIV } 1008$$

$$C = 0.$$

$$H = [1 \text{ MOD } (16 * 63)] \text{ DIV } 63$$

$$H = (1 \text{ MOD } 1008) \text{ DIV } 63$$

$$H = 1 \text{ DIV } 63$$

$$H = 0.$$

$$S = [1 \text{ MOD } (16 * 63)] \text{ MOD } 63 + 1$$

$$S = (1 \text{ MOD } 1008) \text{ MOD } 63 + 1$$

$$S = 1 \text{ MOD } 63 + 1$$

$$S = 1 + 1$$

$$S = 2.$$

$$(C, H, S) = (0, 0, 2).$$

(ii) 100.

$$C = 100 \text{ DIV } (16 * 63)$$

$$C = 100 \text{ DIV } 1008$$

$$C = 0.$$

$$H = [100 \text{ MOD } (16 * 63)] \text{ DIV } 63$$

$$H = (100 \text{ MOD } 1008) \text{ DIV } 63$$

$$H = 100 \text{ DIV } 63$$

$$H = 1.$$

$$S = [100 \text{ MOD } (16 * 63)] \text{ MOD } 63 + 1$$

$$S = (100 \text{ MOD } 1008) \text{ MOD } 63 + 1$$

$$S = 100 \text{ MOD } 63 + 1$$

$$S = 37 + 1$$

$$S = 38.$$

$$(C, H, S) = (0, 1, 38).$$

(iii) 14890.

$$C = 14890 \text{ DIV } (16 * 63)$$

$$C = 14890 \text{ DIV } 1008$$

$$C = 14.$$

$$H = [14890 \text{ MOD } (16 * 63)] \text{ DIV } 63$$

$$H = (14890 \text{ MOD } 1008) \text{ DIV } 63$$

$$H = 778 \text{ DIV } 63$$

$$H = 12.$$

$$S = [14890 \text{ MOD } (16 * 63)] \text{ MOD } 63 + 1$$

$$S = (14890 \text{ MOD } 1008) \text{ MOD } 63 + 1$$

$$S = 778 \text{ MOD } 63 + 1$$

$$S = 22 + 1$$

$$S = 23.$$

$$(C, H, S) = (14, 12, 23).$$

(iv) 45687.

$$C = 45687 \text{ DIV } (16 * 63)$$

$$C = 45687 \text{ DIV } 1008$$

$$C = 45.$$

$$H = [45687 \text{ MOD } (16 * 63)] \text{ DIV } 63$$

$$H = (45687 \text{ MOD } 1008) \text{ DIV } 63$$

$$H = 327 \text{ DIV } 63$$

$$H = 5.$$

$$S = [45687 \text{ MOD } (16 * 63)] \text{ MOD } 63 + 1$$

$$S = (45687 \text{ MOD } 1008) \text{ MOD } 63 + 1$$

$$S = 327 \text{ MOD } 63 + 1$$

$$S = 12 + 1$$

$$S = 13.$$

$$(C, H, S) = (45, 5, 13).$$

Ejercicio 30.

El Seek Time es el parámetro que posee mayor influencia en el tiempo real necesario para transferir datos desde o hacia un disco. Es importante que el Kernel planifique los diferentes requerimientos al disco para minimizar el movimiento de la cabeza lectora-grabadora.

Se analizan las diferentes políticas de planificación de requerimientos a disco con un ejemplo: Se supone un Head con movimiento en 200 tracks (numerados de 0 a 199), que está en el track 83 atendiendo un requerimiento y, anteriormente, atendió un requerimiento en el track 75.

Si la cola de requerimientos es: {86, 147, 91, 177, 94, 150, 102, 175, 130, 32, 120, 58, 66, 115}. Realizar los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

(a) FCFS (First Come, First Served).

83 a 86= 3.

86 a 147= 61.

147 a 91= 56.

91 a 177= 86.

177 a 94= 83.

94 a 150= 56.

150 a 102= 48.

102 a 175= 73.

175 a 130= 45.

130 a 32= 98.

32 a 120= 88.

120 a 58= 62.

58 a 66= 8.

66 a 115= 49.

Movimientos= 3 + 61 + 56 + 86 + 83 + 56 + 48 + 73 + 45 + 98 + 88 + 62 + 8 + 49

Movimientos= 816.

(b) SSTF (Shortest Seek Time First).

83 a 86= 3.

86 a 91= 5.

91 a 94= 3.

94 a 102= 8.

102 a 115= 13.

115 a 120= 5.

120 a 130= 10.

130 a 147= 17.

147 a 150= 3.
150 a 175= 25.
175 a 177= 2.
177 a 66= 111.
66 a 58= 8.
58 a 32= 26.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 111 + 8 + 26$
Movimientos= 239.

(c) *Scan*.

83 a 86= 3.
86 a 91= 5.
91 a 94= 3.
94 a 102= 8.
102 a 115= 13.
115 a 120= 5.
120 a 130= 10.
130 a 147= 17.
147 a 150= 3.
150 a 175= 25.
175 a 177= 2.
177 a 199= 22.
199 a 66= 133.
66 a 58= 8.
58 a 32= 26.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 22 + 133 + 8 + 26$
Movimientos= 283.

(d) *Look*.

83 a 86= 3.
86 a 91= 5.
91 a 94= 3.
94 a 102= 8.
102 a 115= 13.
115 a 120= 5.
120 a 130= 10.
130 a 147= 17.
147 a 150= 3.
150 a 175= 25.
175 a 177= 2.
177 a 66= 111.
66 a 58= 8.

58 a 32= 26.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 111 + 8 + 26$
Movimientos= 239.

(e) *C-Scan (Circular Scan)*.

83 a 86= 3.

86 a 91= 5.

91 a 94= 3.

94 a 102= 8.

102 a 115= 13.

115 a 120= 5.

120 a 130= 10.

130 a 147= 17.

147 a 150= 3.

150 a 175= 25.

175 a 177= 2.

177 a 199= 22.

0 a 32= 32.

32 a 58= 26.

58 a 66= 8.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 22 + 32 + 26 + 8$
Movimientos= 182.

(f) *C-Look (Circular Look)*.

83 a 86= 3.

86 a 91= 5.

91 a 94= 3.

94 a 102= 8.

102 a 115= 13.

115 a 120= 5.

120 a 130= 10.

130 a 147= 17.

147 a 150= 3.

150 a 175= 25.

175 a 177= 2.

32 a 58= 26.

58 a 66= 8.

Movimientos= $3 + 5 + 3 + 8 + 13 + 5 + 10 + 17 + 3 + 25 + 2 + 26 + 8$
Movimientos= 128.

Ejercicio 31.

¿Alguno de los algoritmos analizados en el ejercicio anterior pueden causar inanición de requerimientos?

Sí, de los algoritmos analizados en el ejercicio anterior, SSTF puede causar inanición de requerimientos, ya que el cabezal atiende siempre el pedido más cercano y, si llegan muchos pedidos cercanos al cabezal, uno que está lejos podría nunca ser atendido.

Ejercicio 32.

Se supone un disco con 300 pistas (numerados de 0 a 299), donde la cabeza se encuentra en la pista 143 atendiendo un requerimiento y, anteriormente, atendió un requerimiento en la pista 125.

Si la cola inicial de requerimientos es: {126, 147, 81, 277, 94, 150, 212, 175, 140, 225, 280, 50, 99, 118, 22, 55} y, luego de 30 movimientos, se incorporan los requerimientos de las pistas {75, 115, 220 y 266}. Realizar los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

(a) FCFS (First Come, First Served).

143 a 126= 17.

126 a 147= 21.

(se incorporan los nuevos requerimientos)

147 a 81= 66.

81 a 277= 196.

277 a 94= 183.

94 a 150= 56.

150 a 212= 62.

212 a 175= 37.

175 a 140= 35.

140 a 225= 85.

225 a 280= 55.

280 a 50= 230.

50 a 99= 49.

99 a 118= 19.

118 a 22= 96.

22 a 55= 33.

55 a 75= 20.

75 a 115= 40.

115 a 220= 105.

220 a 266= 46.

Movimientos= 17 + 21 + 66 + 196 + 183 + 56 + 62 + 37 + 35 + 85 + 55 + 230 + 49 + 19 + 96 + 33 + 20 + 40 + 105 + 46

Movimientos= 1451.

(b) SSTF (Shortest Seek Time First).

143 a 140= 3.

140 a 147= 7.

147 a 150= 3.

150 a 126= 24.

(se incorporan los nuevos requerimientos)

126 a 118= 8.

118 a 115= 3.

115 a 99= 16.
 99 a 94= 5.
 94 a 81= 13.
 81 a 75= 6.
 75 a 55= 20.
 55 a 50= 5.
 50 a 22= 28.
 22 a 175= 153.
 175 a 212= 37.
 212 a 220= 8.
 220 a 225= 5.
 225 a 266= 41.
 266 a 277= 11.
 277 a 280= 3.

Movimientos= $3 + 7 + 3 + 24 + 8 + 3 + 16 + 5 + 13 + 6 + 20 + 5 + 28 + 153 + 37 + 8 + 5$
 $+ 41 + 11 + 3$

Movimientos= 399.

(c) *Scan*.

143 a 147= 4.
 147 a 150= 3.
 150 a 175= 25.
 175 a 212= 37.
 212 a 220= 8.
 220 a 225= 5.
 225 a 266= 41.
 266 a 277= 11.
 277 a 280= 3.
 280 a 299= 19.
 299 a 140= 159.
 140 a 126= 14.
 126 a 118= 8.
 118 a 115= 3.
 115 a 99= 16.
 99 a 94= 5.
 94 a 81= 13.
 81 a 75= 6.
 75 a 55= 20.
 55 a 50= 5.
 50 a 22= 28.

(se incorporan los nuevos requerimientos)

Movimientos= $4 + 3 + 25 + 37 + 8 + 5 + 41 + 11 + 3 + 19 + 159 + 14 + 8 + 3 + 16 + 5 +$
 $13 + 6 + 20 + 5 + 28$

Movimientos= 433.

(d) Look.

143 a 147= 4.

147 a 150= 3.

150 a 175= 25.

(se incorporan los nuevos requerimientos)

175 a 212= 37.

212 a 220= 8.

220 a 225= 5.

225 a 266= 41.

266 a 277= 11.

277 a 280= 3.

280 a 140= 140.

140 a 126= 14.

126 a 118= 8.

118 a 115= 3.

115 a 99= 16.

99 a 94= 5.

94 a 81= 13.

81 a 75= 6.

75 a 55= 20.

55 a 50= 5.

50 a 22= 28.

Movimientos= $4 + 3 + 25 + 37 + 8 + 5 + 41 + 11 + 3 + 140 + 14 + 8 + 3 + 16 + 5 + 13 + 6 + 20 + 5 + 28$

Movimientos= 395.

(e) C-Scan (Circular Scan).

143 a 147= 4.

147 a 150= 3.

150 a 175= 25.

(se incorporan los nuevos requerimientos)

175 a 212= 37.

212 a 220= 8.

220 a 225= 5.

225 a 266= 41.

266 a 277= 11.

277 a 280= 3.

280 a 299= 19.

0 a 22= 22.

22 a 50= 28.

50 a 55= 5.

55 a 75= 20.

75 a 81= 6.

81 a 94= 13.

94 a 99= 5.

99 a 115= 16.

115 a 118= 3.
118 a 126= 8.
126 a 140= 14.

Movimientos= $4 + 3 + 25 + 37 + 8 + 5 + 41 + 11 + 3 + 19 + 22 + 28 + 5 + 20 + 6 + 13 + 5 + 16 + 3 + 8 + 14$

Movimientos= 296.

(f) *C-Look (Circular Look)*.

143 a 147= 4.
147 a 150= 3.
150 a 175= 25.
175 a 212= 37.
212 a 220= 8.
220 a 225= 5.
225 a 266= 41.
266 a 277= 11.
277 a 280= 3.
22 a 50= 28.
50 a 55= 5.
55 a 75= 20.
75 a 81= 6.
81 a 94= 13.
94 a 99= 5.
99 a 115= 16.
115 a 118= 3.
118 a 126= 8.
126 a 140= 14.

(se incorporan los nuevos requerimientos)

Movimientos= $4 + 3 + 25 + 37 + 8 + 5 + 41 + 11 + 3 + 28 + 5 + 20 + 6 + 13 + 5 + 16 + 3 + 8 + 14$

Movimientos= 255.

Ejercicio 33.

Se supone un disco con 300 pistas (numerados de 0 a 299), donde la cabeza se encuentra en la pista 140 atendiendo un requerimiento y, anteriormente, atendió un requerimiento en la pista 135.

Si la cola de requerimientos es: {99, 110, 42, 25, 186, 270, 50, 99, 147PF, 81, 257, 94, 133, 212, 175, 130} y, después de 30 movimientos, se incorporan los requerimientos de las pistas {85, 15PF, 202 y 288} y, después de otros 40 movimientos más, se incorporan los requerimientos de las pistas {75, 149PF, 285, 201 y 59}. Realizar los diagramas para calcular el total de movimientos de head para satisfacer estos requerimientos de acuerdo a los siguientes algoritmos de scheduling de discos:

(a) FCFS (First Come, First Served).

140 a 147PF= 7.

147PF a 99= 48.

(se incorporan los nuevos requerimientos)

99 a 15PF= 84.

(se incorporan los nuevos requerimientos)

15PF a 149PF= 134.

149PF a 110= 39.

110 a 42= 68.

42 a 25= 17.

25 a 186= 161.

186 a 270= 84.

270 a 50= 220.

50 a 99= 49.

99 a 81= 18.

81 a 257= 176.

257 a 94= 163.

94 a 133= 39.

133 a 212= 79.

212 a 175= 37.

175 a 130= 45.

130 a 85= 45.

85 a 202= 117.

202 a 288= 86.

288 a 75= 213.

75 a 285= 210.

285 a 201= 84.

201 a 59= 142.

Movimientos= 7 + 48 + 84 + 134 + 39 + 68 + 17 + 161 + 84 + 220 + 49 + 18 + 176 + 163 + 38 + 79 + 37 + 45 + 45 + 117 + 86 + 213 + 210 + 84 + 142

Movimientos= 2365.

(b) SSTF (Shortest Seek Time First).

140 a 147PF= 7.

147PF a 133= 14.

133 a 130= 3.

130 a 110= 20.

(se incorporan los nuevos requerimientos)

110 a 15PF= 95.

(se incorporan los nuevos requerimientos)

15PF a 149PF= 134.

149PF a 175= 26.

175 a 186= 11.

186 a 201= 15.

201 a 202= 1.

202 a 212= 10.

212 a 257= 45.

257 a 270= 13.

270 a 285= 15.

285 a 288= 3.

288 a 99= 189.

99 a 99= 0.

99 a 94= 5.

94 a 85= 9.

85 a 81= 4.

81 a 75= 6.

75 a 59= 16.

59 a 50= 9.

50 a 42= 8.

42 a 25= 17.

Movimientos= $7 + 14 + 3 + 20 + 95 + 134 + 26 + 11 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 189 + 0 + 5 + 9 + 4 + 6 + 16 + 9 + 8 + 17$

Movimientos= 675.

(c) *Scan*.

140 a 147PF= 7.

147PF a 175= 28.

(se incorporan los nuevos requerimientos)

175 a 15PF= 160.

(se incorporan los nuevos requerimientos)

15PF a 149PF= 134.

149PF a 186= 37.

186 a 201= 15.

201 a 202= 1.

202 a 212= 10.

212 a 257= 45.

257 a 270= 13.

270 a 285= 15.

285 a 288= 3.

288 a 299= 11.

299 a 133= 166.

133 a 130= 3.

130 a 110= 20.

110 a 99= 11.

99 a 99= 0.
 99 a 94= 5.
 94 a 85= 9.
 85 a 81= 4.
 81 a 75= 6.
 75 a 59= 16.
 59 a 50= 9.
 50 a 42= 8.
 42 a 25= 17.

Movimientos= $7 + 28 + 160 + 134 + 37 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 11 + 166 + 3$
 $+ 20 + 11 + 0 + 5 + 9 + 4 + 6 + 16 + 9 + 8 + 17$
 Movimientos= 753.

(d) *Look.*

140 a 147PF= 7.
 147PF a 175= 28.
 175 a 15PF= 160.
 15PF a 149PF= 134.
 149PF a 186= 37.
 186 a 201= 15.
 201 a 202= 1.
 202 a 212= 10.
 212 a 257= 45.
 257 a 270= 13.
 270 a 285= 15.
 285 a 288= 3.
 288 a 133= 155.
 133 a 130= 3.
 130 a 110= 20.
 110 a 99= 11.
 99 a 99= 0.
 99 a 94= 5.
 94 a 85= 9.
 85 a 81= 4.
 81 a 75= 6.
 75 a 59= 16.
 59 a 50= 9.
 50 a 42= 8.
 42 a 25= 17.

(se incorporan los nuevos requerimientos)

(se incorporan los nuevos requerimientos)

Movimientos= $7 + 28 + 160 + 134 + 37 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 155 + 3 + 20$
 $+ 11 + 0 + 5 + 9 + 4 + 6 + 16 + 9 + 8 + 17$
 Movimientos= 735.

(e) *C-Scan (Circular Scan).*

140 a 147PF= 7.
 147PF a 175= 28. (se incorporan los nuevos requerimientos)
 175 a 15PF= 160. (se incorporan los nuevos requerimientos)
 15PF a 149PF= 134.
 149PF a 186= 37.
 186 a 201= 15.
 201 a 202= 1.
 202 a 212= 10.
 212 a 257= 45.
 257 a 270= 13.
 270 a 285= 15.
 285 a 288= 3.
 288 a 299= 11.
 0 a 25= 25.
 25 a 42= 17.
 42 a 50= 8.
 50 a 59= 9.
 59 a 75= 16.
 75 a 81= 6.
 81 a 85= 4.
 85 a 94= 9.
 94 a 99= 5.
 99 a 99= 0.
 99 a 110= 11.
 110 a 130= 20.
 130 a 133= 3.

Movimientos= $7 + 28 + 160 + 134 + 37 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 11 + 25 + 17$
 $+ 8 + 9 + 16 + 6 + 4 + 9 + 5 + 0 + 11 + 20 + 3$
 Movimientos= 612.

(f) C-Look (Circular Look).

140 a 147PF= 7.
 147PF a 175= 28. (se incorporan los nuevos requerimientos)
 175 a 15PF= 160. (se incorporan los nuevos requerimientos)
 15PF a 149PF= 134.
 149PF a 186= 37.
 186 a 201= 15.
 201 a 202= 1.
 202 a 212= 10.
 212 a 257= 45.
 257 a 270= 13.
 270 a 285= 15.
 285 a 288= 3.
 25 a 42= 17.

42 a 50= 8.

50 a 59= 9.

59 a 75= 16.

75 a 81= 6.

81 a 85= 4.

85 a 94= 9.

94 a 99= 5.

99 a 99= 0.

99 a 110= 11.

110 a 130= 20.

130 a 133= 3.

Movimientos= $7 + 28 + 160 + 134 + 37 + 15 + 1 + 10 + 45 + 13 + 15 + 3 + 17 + 8 + 9 + 16 + 6 + 4 + 9 + 5 + 0 + 11 + 20 + 3$

Movimientos= 576.

PARTE IV: Administración de Archivos.**Ejercicio 34.**

Dadas las siguientes técnicas de administración de espacio de un archivo:

- *Asignación contigua.*
- *Asignación enlazada.*
- *Asignación indexada.*

(a) *Describir cómo trabaja cada una.*

(b) *Citar ventajas y desventajas de cada una.*

(c) *Indicar, para cada técnica, si la misma puede producir fragmentación interna y/o externa.*

A continuación, se describen las técnicas de administración de espacio de un archivo:

1. Asignación contigua:

- Cada archivo ocupa bloques consecutivos en el disco.
- Se almacena el bloque inicial y el tamaño.
- La lectura es secuencial y, extremadamente, rápida.
- Ventajas: excelente performance en acceso secuencial y aleatorio; fácil de administrar.
- Desventajas: necesita conocer el tamaño del archivo por adelantado; difícil crecimiento del archivo.
- Puede producir fragmentación interna (último bloque puede quedar parcialmente lleno) y fragmentación externa (requiere bloques contiguos).

2. Asignación enlazada:

- El archivo se guarda como una lista enlazada de bloques en cualquier parte del disco.
- Cada bloque contiene un puntero al siguiente bloque.
- No requiere espacio contiguo.
- Ventajas: archivos pueden crecer dinámicamente; no requiere bloques contiguos.
- Desventajas: acceso aleatorio muy lento (hay que recorrer la lista); *overhead* por punteros; si se corrompe un puntero, se pierde parte del archivo.
- Puede producir fragmentación interna (cada bloque puede no usarse por completo).

3. Asignación indexada:

- Se crea un bloque índice (*index block*) para cada archivo.
- El bloque índice contiene una tabla de punteros a los bloques de datos.
- Los bloques pueden estar dispersos como en la asignación enlazada, pero se accede directo mediante el índice.
- Ventajas: permite acceso directo y secuencial; archivos pueden crecer sin necesidad de contigüidad; no depende de la integridad de una cadena de punteros, como en la asignación enlazada.

- Desventajas: ocupa espacio extra para el bloque índice; el tamaño del archivo puede estar limitado por la cantidad de punteros del índice; ligeramente, más compleja de administrar.
- Puede producir fragmentación interna (tamaño del último bloque + tamaño del bloque índice).

Ejercicio 35: Gestión de Espacio Libre.

Dados los siguientes métodos de gestión de espacio libre en un disco:

- *Tabla de bits.*
- *Lista enlazada.*
- *Agrupamiento.*
- *Recuento.*

(a) *Describir cómo trabajan estos métodos.*

(b) *Citar ventajas y desventajas de cada uno.*

A continuación, se describen cómo trabajan estos métodos:

1. Tabla de bits:

- Se mantiene un vector de bits, donde cada bit representa un bloque del disco: 0 es bloque libre y 1 es bloque ocupado.
- El *bitmap* suele mantenerse en memoria para consultas rápidas.
- Ventajas: muy fácil identificar bloques contiguos (útil para asignación contigua); ocupa poco espacio (1 bit por bloque); rápida búsqueda usando operaciones de bits.
- Desventajas: si el disco es grande, el *bitmap* puede ser, relativamente, extenso; puede requerir leer varias palabras de memoria para encontrar un rango libre.

2. Lista enlazada:

- Todos los bloques libres están encadenados uno tras otro formando una lista.
- Cada bloque libre contiene un puntero al siguiente bloque libre.
- El *superblock* mantiene la dirección del primer bloque libre.
- Ventajas: sencilla de implementar; no requiere espacio en memoria más que un puntero al primer bloque.
- Desventajas: ineficiente para buscar bloques contiguos; acceso lento (cada búsqueda requiere leer bloques libres del disco); si se corrompe un puntero, se pierde parte de la lista.

3. Agrupamiento:

- Variante de la lista enlazada.
- En lugar de que cada bloque libre apunte a uno solo, un bloque libre contiene la dirección de los siguientes N bloques libres y un puntero a otro bloque que contiene el siguiente grupo.
- Se reducen accesos al disco.
- Ventajas: búsqueda mucho más rápida que en la lista simple; menos accesos al disco para localizar bloques libres; sigue siendo simple de implementar.
- Desventajas: todavía es difícil encontrar grandes áreas contiguas; depende de la integridad de los bloques de grupo.

4. Recuento:

- Se observa que los bloques libres suelen aparecer en grupos contiguos.
- Se guarda una lista de entradas de la forma: (bloque inicial, cantidad de bloques contiguos libres).
- Permite representar grandes rangos de bloques libres con un solo registro.

- Ventajas: excelente para sistemas en los que los bloques libres están agrupados; permite encontrar áreas contiguas rápidamente; representa muchos bloques libres con pocos registros; muy eficiente en sistemas grandes.
- Desventajas: más complejo de mantener cuando los archivos se crean y borran continuamente; menos eficiente si los bloques libres están muy fragmentados y no contiguos.

Ejercicio 36.

Dado un disco con las características vistas en el Ejercicio 28:

- 7 platos con 2 caras utilizables cada uno.
- 1.100 cilindros, con 300 sectores por pista, donde cada sector es de 512 bytes.
- Seek time de 10 ms.
- 9.000 RPM.
- Velocidad de transferencia de 10 MiB/s (MebiBytes por segundo).

(a) Calcular el tiempo de transferencia real de un archivo de 15 MiB (MebiBytes) almacenado en el disco de manera secuencial (todos sus bloques almacenados de manera consecutiva).

$$\text{Sectores} = \frac{15 \times 1048576 \text{ bytes}}{512 \text{ bytes}}$$

$$\text{Sectores} = \frac{15728640 \text{ bytes}}{512 \text{ bytes}}$$

$$\text{Sectores} = 30720.$$

$$\text{Latencia} = 0,5 \frac{1}{\frac{9000}{60 \times 1000}} \text{ ms}$$

$$\text{Latencia} = 0,5 \frac{1}{\frac{9000}{60000}} \text{ ms}$$

$$\text{Latencia} = 0,5 \frac{1}{0,15} \text{ ms}$$

$$\text{Latencia} = 0,5 * 6, \hat{6} \text{ ms}$$

$$\text{Latencia} = 3, \hat{3} \text{ ms}.$$

$$\text{Tiempo Transferencia} = \frac{512 \text{ bytes}}{\frac{10 \times 1048576 \text{ bytes}}{1000}} \text{ ms}$$

$$\text{Tiempo Transferencia} = \frac{512 \text{ bytes}}{\frac{10485760 \text{ bytes}}{1000}} \text{ ms}$$

$$\text{Tiempo Transferencia} = \frac{512 \text{ bytes}}{1048,56 \text{ bytes}} \text{ ms}$$

$$\text{Tiempo Transferencia} = 0,48828125 \text{ ms}.$$

$$\text{Tiempo Transferencia Real} = 10 \text{ ms} + 3, \hat{3} \text{ ms} + 0,48828125 \text{ ms} * 30720$$

$$\text{Tiempo Transferencia Real} = 10 \text{ ms} + 3, \hat{3} \text{ ms} + 1500 \text{ ms}$$

$$\text{Tiempo Transferencia Real} = 1513, \hat{3} \text{ ms}.$$

Por lo tanto, el tiempo de transferencia real de un archivo de 15 MiB almacenado en el disco de manera secuencial es 1.513,3 ms.

(b) Calcular el tiempo de transferencia real de un archivo de 16 MiB (MebiBytes) almacenado en el disco de manera aleatoria.

$$\text{Sectores} = \frac{16 \times 1048576 \text{ bytes}}{512 \text{ bytes}}$$

$$\text{Sectores} = \frac{16777216 \text{ bytes}}{512 \text{ bytes}}$$
$$\text{Sectores} = 30768.$$

$$\text{Tiempo Transferencia Real} = (10 \text{ ms} + 3,3 \text{ ms} + 0,048828125 \text{ ms}) * 30768$$

$$\text{Tiempo Transferencia Real} = 13,3821614583 \text{ ms} * 30768$$

$$\text{Tiempo Transferencia Real} = 411742,34375 \text{ ms}.$$

Por lo tanto, el tiempo de transferencia real de un archivo de 16 MiB almacenado en el disco de manera aleatoria es 411.724,34375 ms.

Ejercicio 37: Unidad de Asignación.

Las técnicas vistas pueden referenciar a sectores físicos de medio de almacenamiento o clusters (conjunto consecutivo de sectores físicos). Citar ventajas y desventajas de la utilización de cada forma de unidad de asignación.

A continuación, se citan ventajas y desventajas de la utilización de cada forma de unidad de asignación:

1. Sectores físicos de medio de almacenamiento:

- Ventajas: menor fragmentación interna, ya que se desperdicia menos espacio por archivo; asignación más precisa (los archivos usan sólo los sectores exactos que necesitan); mejor para archivos pequeños (no desperdician espacio); control más fino sobre la distribución en disco.
- Desventajas: mayor sobrecarga en las estructuras del sistema de archivos (más entradas por archivo); más operaciones de E/S por archivo (muchos sectores a leer/escribir); mayor tiempo para buscar sectores libres, especialmente si están dispersos; el sistema se vuelve más complejo y más lento en archivos grandes.

2. Clusters:

- Ventajas: menor sobrecarga administrativa; lectura/escritura más rápida (se accede a grandes porciones contiguas); ideal para archivos grandes, ya que mejora el rendimiento; simplifica la búsqueda de espacio libre.
- Desventajas: puede generar fragmentación interna; desperdicio notable en sistemas con muchos archivos pequeños; menor granularidad (asignación menos precisa).

Ejercicio 38.

Si se tiene un disco con 10 platos con 2 caras utilizables cada uno (20 cabezas), 350 cilindros y con 500 sectores por pista, ¿cuántos clusters puede haber si se toma un tamaño de 4 sectores? ¿Y si el cluster es de 8 sectores?

Sectores= sectores/pista * cilindros * cabezas

Sectores= 500 * 350 * 20

Sectores= 3500000.

Con tamaño de 4 sectores:

Clusters= $\frac{3500000}{4}$

Clusters= 875000.

Con tamaño de 8 sectores:

Clusters= $\frac{3500000}{8}$

Clusters= 437500.

Por lo tanto, si se tiene un disco con 10 platos con 2 caras utilizables cada uno (20 cabezas), 350 cilindros y con 500 sectores por pista, si se toma un tamaño de 4 sectores, puede haber 875.000 *clusters* y, si el *cluster* es de 8 sectores, puede haber 437.500 *clusters*.

Ejercicio 39.

Asumiendo un sistema de archivos que utiliza, como unidad de asignación, un cluster de 8 sectores de disco y que el primer cluster de datos (cluster 0) se comienza a almacenar en el sector LBA 120. Determinar el o los valores LBA de los sectores que corresponden a los siguientes números de cluster:

(a) 0.

$$\text{LBA inicial} = 120 + (0 * 8)$$

$$\text{LBA inicial} = 120 + 0$$

$$\text{LBA inicial} = 120.$$

Por lo tanto, los valores LBA de los sectores que corresponden al *cluster* 0 son de 120 a 127.

(b) 123.

$$\text{LBA inicial} = 120 + (123 * 8)$$

$$\text{LBA inicial} = 120 + 984$$

$$\text{LBA inicial} = 1104.$$

Por lo tanto, los valores LBA de los sectores que corresponden al *cluster* 123 son de 1.104 a 1.111.

(c) 299.

$$\text{LBA inicial} = 120 + (299 * 8)$$

$$\text{LBA inicial} = 120 + 2392$$

$$\text{LBA inicial} = 2512.$$

Por lo tanto, los valores LBA de los sectores que corresponden al *cluster* 299 son de 2.512 a 2.519.

(d) 1001.

$$\text{LBA inicial} = 120 + (1001 * 8)$$

$$\text{LBA inicial} = 120 + 8008$$

$$\text{LBA inicial} = 8128.$$

Por lo tanto, los valores LBA de los sectores que corresponden al *cluster* 1.001 son de 8.128 a 8.135.

Ejercicio 40.

Describir la organización del FileSystem de Unix System V. Tener en cuenta la organización del mismo, estructuras de datos, etc.

A continuación, se describe la organización del *File System* de Unix *System V*:

1. Boot Block: Código de arranque.
2. Superblock: Metadatos globales del *File System*.
3. i-list: Tabla fija de i-nodos.
4. Directorios: Mapean nombres a i-nodos.
5. Data Blocks: Contienen datos reales.
6. Free List: Administración de espacio libre.
7. Estructuras en memoria: Tabla de i-nodos, tabla de archivos abiertos, *buffer* caché.

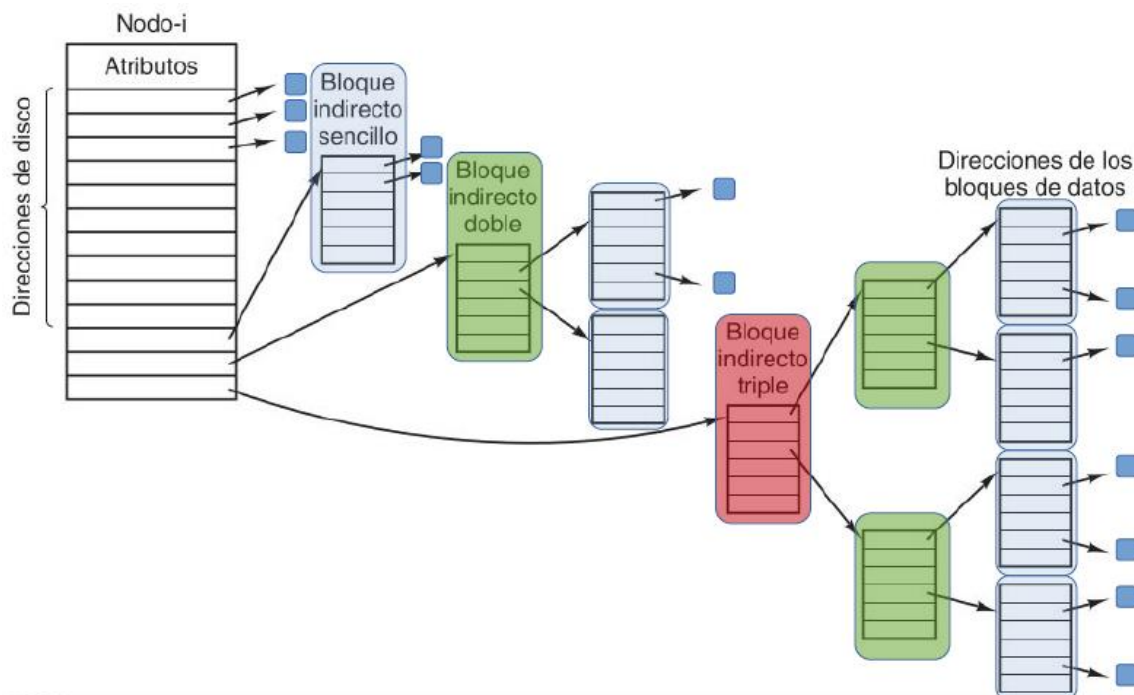
Ejercicio 41: Gestión de Archivos en UNIX System V.

El sistema de archivos de UNIX utiliza una variante del esquema de asignación indexada para la administración de espacio de los archivos. Cada archivo o directorio está representado por una estructura que mantiene, entre otra información, las direcciones de los bloques que contienen los datos del archivo: el I-NODO.

Cada I-NODO contiene 13 campos para direcciones a los bloques de datos, organizadas de la siguiente manera:

- 10 de direccionamiento directo.
- 1 de direccionamiento indirecto simple.
- 1 de direccionamiento indirecto doble.
- 1 de direccionamiento indirecto triple.

(a) Realizar un gráfico que represente la estructura del I-NODO y de los bloques de datos. Cada cluster es de 1 Kib (Kibibits).



(b) Si cada dirección para referenciar un bloque es de 32 bits:

(i) ¿Cuántas referencias (direcciones) a bloque puede contener un cluster?

$$\text{Direcciones} = \frac{1024 \text{ bits}}{32 \text{ bits}}$$

$$\text{Direcciones} = 32.$$

Por lo tanto, un cluster puede contener 32 referencias (direcciones) a bloque.

(ii) *¿Cuál sería el tamaño máximo de un archivo?*

$$\text{Tamaño máximo} = (10 + 32 + 32^2 + 32^3) * \frac{1024}{8} \text{ bytes}$$

$$\text{Tamaño máximo} = (10 + 32 + 1024 + 32768) * 128 \text{ bytes}$$

$$\text{Tamaño máximo} = 33834 * 128 \text{ bytes}$$

$$\text{Tamaño máximo} = 4330752 \text{ bytes.}$$

Por lo tanto, el tamaño máximo de un archivo sería 4.330.752 bytes.

Ejercicio 42.

Dado el FileSystem de UNIX System V (visto en clase), indicar qué cambios se producen en el mismo dadas las siguientes situaciones:

(a) *Se agrega un nuevo archivo “Archivo1” en el directorio “Directorio1”.*

Los cambios que se producen en el *FileSystem* de UNIX System V son:

- Se asigna un nuevo inodo para “Archivo1”.
- Se agrega una entrada (nombre + número de inodo) al directorio.
- Se actualiza el inodo del directorio (*mtime/ctime/size*).
- Se actualiza mapa de inodos.
- Si el archivo agregado no entra en los bloques existentes:
 - Se asigna uno o más bloques nuevos para el directorio.
 - Se actualiza mapa de bloques.

(b) *Se agrega nuevo contenido al archivo “Archivo1”.*

Los cambios que se producen en el *FileSystem* de UNIX System V son:

- Se modifica el inodo de “Archivo1”.
- Se actualiza el inodo del directorio (*mtime*).
- Si el contenido agregado al archivo no entra en los bloques existentes:
 - Se asigna uno o más bloques nuevos para el directorio.
 - Se actualiza mapa de bloques.

(c) *Se modifica el nombre del archivo “Archivo1” a “Archivo2”.*

Los cambios que se producen en el *FileSystem* de UNIX System V son:

- Se modifica la entrada del directorio (nombre de inodo).
- Se actualiza el inodo del directorio (*mtime/ctime*).

(d) *Se hace un link duro a “Archivo2” en el mismo directorio con nombre “LinkArchivo2”.*

Los cambios que se producen en el *FileSystem* de UNIX System V son:

- Se agrega una entrada (nombre + número de inodo) al directorio.
- Se actualiza el inodo del directorio (*mtime/ctime/size*).
- Se actualiza el inodo del archivo original (se incrementa el *link count*).

- Si no hay espacio disponible para una entrada extra:
 - Se asigna un nuevo bloque para el directorio.
 - Se actualiza mapa de bloques.

(e) *Se borra un archivo “Archivo3”.*

Los cambios que se producen en el *FileSystem* de UNIX *System V* son:

- Se elimina la entrada (nombre + número de inodo) del directorio.
- Se actualiza el inodo del directorio (*mtime/ctime/size*).
- Se actualiza el inodo del archivo original (se decrementa el *link count*).
- Si *link count* llega a cero:
 - Se libera uno o más bloques del archivo.
 - Se libera el inodo del archivo.
 - Se actualizan mapa de bloques y mapa de inodos.

Ejercicio 43.

Se tiene un FileSystem con un modelo indexado con niveles de indirección, donde se mantiene un i-nodo con 10 índices de direccionamiento directo, 1 índice para indirecto simple y 2 para indirecto doble. Considerando, además, que se utilizan 32 bits para referenciar direcciones a bloques/clusters.

Suponer que existen los siguientes archivos con sus tamaños lógicos:

- ArchivoA: de 55 KibiBytes.
- ArchivoB: de 12 KibiBytes.
- ArchivoC: de 1.075 KibiBytes.

(a) Si se utiliza, como unidad de asignación, el sector de disco de 512 bytes:

(i) ¿Cuántas direcciones a sectores puede contener un sector de disco?

$$\begin{aligned}\text{Direcciones} &= \frac{512 \text{ bytes}}{\frac{2^{32} \text{ bytes}}{8}} \\ \text{Direcciones} &= \frac{512 \text{ bytes}}{4 \text{ bytes}} \\ \text{Direcciones} &= 128.\end{aligned}$$

Por lo tanto, un sector de disco puede contener 128 direcciones a sectores.

(ii) ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?

$$\begin{aligned}\text{Tamaño máximo} &= (10 + 128 + 2 * 128^2) * 512 \text{ bytes} \\ \text{Tamaño máximo} &= (10 + 128 + 2 * 16384) * 512 \text{ bytes} \\ \text{Tamaño máximo} &= (10 + 128 + 32768) * 512 \text{ bytes} \\ \text{Tamaño máximo} &= 32906 * 512 \text{ bytes} \\ \text{Tamaño máximo} &= 16847872 \text{ bytes}.\end{aligned}$$

Por lo tanto, el tamaño máximo que un archivo podrá alcanzar sería 16.847.872 bytes (16.453 KiB).

(iii) ¿Cuántos sectores se necesitan para almacenar cada uno de los archivos antes enumerados? Indicar el tamaño físico que se requirió para almacenar cada uno.

$$\begin{aligned}\text{Sectores}_A &= \frac{55 * 1024 \text{ bytes}}{512 \text{ bytes}} \\ \text{Sectores}_A &= 55 * 2 \\ \text{Sectores}_A &= 110.\end{aligned}$$

$$\text{Sectores}_B = \frac{12 * 1024 \text{ bytes}}{512 \text{ bytes}}$$

$$Sectores_B = 12 * 2$$

$$Sectores_B = 24.$$

$$Sectores_C = \frac{1075 * 1024 \text{ bytes}}{512 \text{ bytes}}$$

$$Sectores_C = 1075 * 2$$

$$Sectores_C = 2150.$$

$$TF_A = 110 * 512 \text{ bytes}$$

$$TF_A = 56320 \text{ bytes}$$

$$TF_A = 55 \text{ KiB.}$$

$$TF_B = 24 * 512 \text{ bytes}$$

$$TF_B = 12288 \text{ bytes}$$

$$TF_B = 12 \text{ KiB.}$$

$$TF_C = 2150 * 512 \text{ bytes}$$

$$TF_C = 1100800 \text{ bytes}$$

$$TF_C = 1075 \text{ KiB.}$$

Por lo tanto, para almacenar los archivos ArchivoA, ArchivoB y ArchivoC, se necesitan 110, 24 y 2.150 sectores, respectivamente.

(iv) Indicar la fragmentación interna que se produce para cada uno de los archivos antes enumerados.

$$FI_A = TF_A - TL_A$$

$$FI_A = 55 \text{ KiB} - 55 \text{ KiB}$$

$$FI_A = 0 \text{ KiB.}$$

$$FI_B = TF_B - TL_B$$

$$FI_B = 12 \text{ KiB} - 12 \text{ KiB}$$

$$FI_B = 0 \text{ KiB.}$$

$$FI_C = TF_C - TL_C$$

$$FI_C = 1075 \text{ KiB} - 1075 \text{ KiB}$$

$$FI_C = 0 \text{ KiB.}$$

(b) Si se utiliza, como unidad de asignación, el cluster de disco de 2 KibiBytes:

(i) ¿Cuántas direcciones a clusters puede contener un cluster de disco?

$$\text{Direcciones} = \frac{2 * 1024 \text{ bytes}}{\frac{32}{8} \text{ bytes}}$$

$$\text{Direcciones} = \frac{2048 \text{ bytes}}{4 \text{ bytes}}$$

$$\text{Direcciones} = 512.$$

Por lo tanto, un *cluster* de disco puede contener 512 direcciones a *clusters*.

(ii) ¿Cuál sería el tamaño máximo que un archivo podría alcanzar?

$$\text{Tamaño máximo} = (10 + 512 + 2 * 512^2) * 2 * 1024 \text{ bytes}$$

$$\text{Tamaño máximo} = (10 + 512 + 2 * 262144) * 2048 \text{ bytes}$$

$$\text{Tamaño máximo} = (10 + 512 + 524288) * 2048 \text{ bytes}$$

$$\text{Tamaño máximo} = 524810 * 2048 \text{ bytes}$$

$$\text{Tamaño máximo} = 1074810880 \text{ bytes.}$$

Por lo tanto, el tamaño máximo que un archivo podrá alcanzar sería 1.074.810.880 bytes (1.049.620 KiB).

(iii) ¿Cuántos *clusters* se necesitan para almacenar cada uno de los archivos antes enumerados? Indicar el tamaño físico que se requirió para almacenar cada uno.

$$\text{Clusters}_A = \frac{55 * 1024 \text{ bytes}}{2048 \text{ bytes}}$$

$$\text{Clusters}_A = 55 * 0,5$$

$$\text{Clusters}_A = 27,5$$

$$\text{Clusters}_A = 28.$$

$$\text{Clusters}_B = \frac{12 * 1024 \text{ bytes}}{2048 \text{ bytes}}$$

$$\text{Clusters}_B = 12 * 0,5$$

$$\text{Clusters}_B = 6.$$

$$\text{Clusters}_C = \frac{1075 * 1024 \text{ bytes}}{2048 \text{ bytes}}$$

$$\text{Clusters}_C = 1075 * 0,5$$

$$\text{Clusters}_C = 537,5$$

$$\text{Clusters}_C = 538.$$

$$TF_A = 28 * 2048 \text{ bytes}$$

$$TF_A = 57344 \text{ bytes}$$

$$TF_A = 56 \text{ KiB.}$$

$$TF_B = 6 * 2048 \text{ bytes}$$

$$TF_B = 12288 \text{ bytes}$$

$$TF_B = 12 \text{ KiB.}$$

$$TF_C = 538 * 2048 \text{ bytes}$$

$$TF_C = 1101824 \text{ bytes}$$

$$TF_C = 1076 \text{ KiB.}$$

Por lo tanto, para almacenar los archivos ArchivoA, ArchivoB y ArchivoC, se necesitan 28, 6 y 538 *clusters*, respectivamente.

(iv) Indicar la fragmentación interna que se produce para cada uno de los archivos antes enumerados.

$$FI_A = TF_A - TL_A$$

$$FI_A = 56 \text{ KiB} - 55 \text{ KiB}$$

$$FI_A = 1 \text{ KiB.}$$

$$FI_B = TF_B - TL_B$$

$$FI_B = 12 \text{ KiB} - 12 \text{ KiB}$$

$$FI_B = 0 \text{ KiB.}$$

$$FI_C = TF_C - TL_C$$

$$FI_C = 1076 \text{ KiB} - 1075 \text{ KiB}$$

$$FI_C = 1 \text{ KiB.}$$

Ejercicio 44.

Para los incisos (a) y (b) el ejercicio anterior y considerando que el FileSystem se utiliza en un disco con las siguientes características:

- 6 platos con 2 caras útiles cada uno.
- 2.500 pistas por cara y 100 sectores por pista de 512 bytes cada uno.
- 7.200 RPM.
- Seek time de 10,5 ms.
- Velocidad de transferencia de 146 MiB/s (MebiBytes por segundo).

Calcular el tiempo de transferencia de los 3 archivos (ArchivoA, ArchivoB y ArchivoC) del ejercicio previo.

(a)

$$\text{Velocidad de transferencia} = \frac{146 \cdot 1024}{1000} \text{ KiB/ms}$$

$$\text{Velocidad de transferencia} = \frac{149504 \text{ KiB}}{1000} \text{ KiB/ms}$$

$$\text{Velocidad de transferencia} = 149,504 \text{ KiB/ms.}$$

$$\text{Tiempo Transferencia}_A = \frac{55 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms}$$

$$\text{Tiempo Transferencia}_A = 0,368 \text{ ms.}$$

$$\text{Tiempo Transferencia}_B = \frac{12 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms}$$

$$\text{Tiempo Transferencia}_B = 0,08 \text{ ms.}$$

$$\text{Tiempo Transferencia}_C = \frac{1075 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms}$$

$$\text{Tiempo Transferencia}_C = 7,19 \text{ ms.}$$

Por lo tanto, el tiempo de transferencia de los archivos ArchivoA, ArchivoB y ArchivoC es 0,368 ms, 0,08 ms y 7,19 ms, respectivamente.

(b)

$$\text{Velocidad de transferencia} = \frac{146 \cdot 1024}{1000} \text{ KiB/ms}$$

$$\text{Velocidad de transferencia} = \frac{149504 \text{ KiB}}{1000} \text{ KiB/ms}$$

$$\text{Velocidad de transferencia} = 149,504 \text{ KiB/ms.}$$

$$\text{Tiempo Transferencia}_A = \frac{56 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms}$$

$$\text{Tiempo Transferencia}_A = 0,375 \text{ ms.}$$

$$\text{Tiempo Transferencia}_A = \frac{12 \text{ KiB}}{149,504 \text{ KiB}}$$

Tiempo Transferencia_B = 0,08 ms.

$$\textit{Tiempo Transferencia}_C = \frac{1076 \text{ KiB}}{149,504 \text{ KiB}} \text{ ms}$$

Tiempo Transferencia_C = 7,197 ms.

Por lo tanto, el tiempo de transferencia de los archivos ArchivoA, ArchivoB y ArchivoC es 0,375 ms, 0,08 ms y 7,197 ms, respectivamente.

Ejercicio 45.

Analizar las siguientes fórmulas necesarias para localizar un I-NODO en la tabla de inodos de un FileSystem similar a System V (visto en la teoría):

- *Número de bloque del inodo* = $[(\text{nro. de inodo} - 1) / \text{nro. de inodos por bloque}] + \text{bloque de comienzo de la lista de inodos.}$
- *Desplazamiento en el bloque del inodo* = $[(\text{nro. de inodo} - 1) \bmod (\text{número de inodos por bloque})] * \text{medida de inodo del disco.}$

(a) Según la primera fórmula, asumiendo que, en el bloque 2, está en el comienzo de la lista de inodos y que hay 8 inodos por bloque, calcular dónde se encuentra el inodo 8 y el 9. ¿Dónde estarían para un bloque de disco con 16 inodos?

Con 8 inodos por bloque:

$$\text{Número bloque}_{\text{inodo } 8} = \frac{8-1}{8} + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = \frac{7}{8} + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = 0 + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = 2.$$

$$\text{Número bloque}_{\text{inodo } 9} = \frac{9-1}{8} + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = \frac{8}{8} + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = 1 + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = 3.$$

Con 16 inodos por bloque:

$$\text{Número bloque}_{\text{inodo } 8} = \frac{8-1}{16} + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = \frac{7}{16} + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = 0 + 2$$

$$\text{Número bloque}_{\text{inodo } 8} = 2.$$

$$\text{Número bloque}_{\text{inodo } 9} = \frac{9-1}{16} + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = \frac{8}{16} + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = 0 + 2$$

$$\text{Número bloque}_{\text{inodo } 9} = 2.$$

Por lo tanto, si hay 8 inodos por bloque, los inodos 8 y 9 se encuentran en los bloques 2 y 3, respectivamente, mientras que, para un bloque de disco con 16 inodos, estarían en los bloques 2 y 2, respectivamente.

(b) De acuerdo a la segunda fórmula, si cada inodo del disco ocupa 64 bytes y hay 8 inodos por bloque de disco, el inodo 8 comienza en el desplazamiento 448 del bloque de

disco. ¿Dónde empieza el 6? Si fueran inodos de 128 bytes y 24 inodos por bloque, ¿dónde empezaría el inodo 8?

Desplazamiento bloque_{inodo 6} = $[(6 - 1) \text{ MOD } 8] * 64 \text{ bytes}$

Desplazamiento bloque_{inodo 6} = $(5 \text{ MOD } 8) * 64 \text{ bytes}$

Desplazamiento bloque_{inodo 6} = $5 * 64 \text{ bytes}$

Desplazamiento bloque_{inodo 6} = 320 bytes.

Desplazamiento bloque_{inodo 8} = $[(8 - 1) \text{ MOD } 24] * 128 \text{ bytes}$

Desplazamiento bloque_{inodo 8} = $(7 \text{ MOD } 24) * 128 \text{ bytes}$

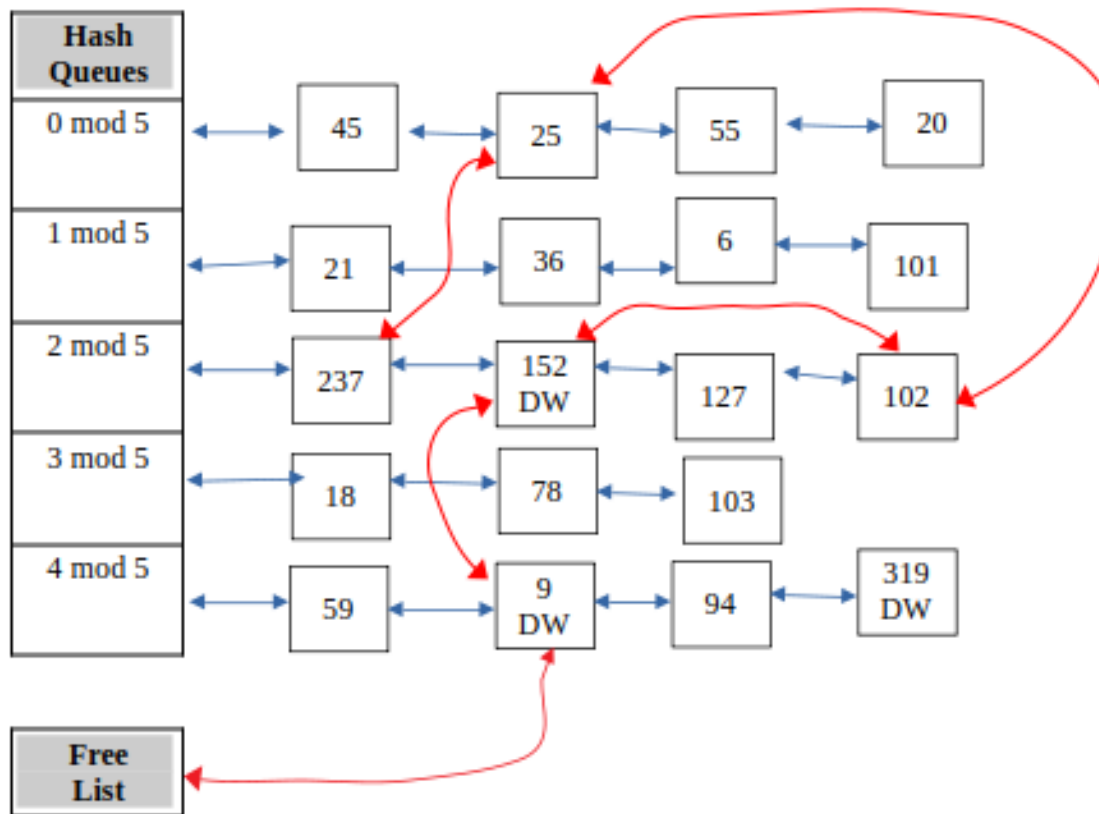
Desplazamiento bloque_{inodo 8} = $7 * 128 \text{ bytes}$

Desplazamiento bloque_{inodo 8} = 896 bytes.

Por lo tanto, si cada inodo del disco ocupa 64 bytes y hay 8 inodos por bloque de disco, el inodo 6 empieza en el desplazamiento 320 bytes del bloque de disco y, si fueran inodos de 128 bytes y 24 inodos por bloque, el inodo 8 empezaría en el desplazamiento 896 bytes del bloque de disco.

PARTE V: Caché de Disco.**Ejercicio 46.**

Dado el siguiente gráfico que representa el estado de Buffer Caché en Unix System V (visto en clase):



Responder a los siguientes incisos (considerar cambios en los headers, las hash queue, free list, operaciones de E/S, etc.):

(a) ¿Qué sucedería si un proceso P1 requiere el bloque 45?

Escenario 5: El Kernel encuentra el bloque en la hash queue, pero está busy.

Si un proceso P1 solicita el bloque 45, lo que sucedería es que el Kernel encontraría este bloque en la hash queue, pero el buffer asociado a este bloque está marcado como busy. El proceso P1 quedaría bloqueado en espera a que el buffer se desbloquee. Cuando el proceso que lo tiene lo libere, P1 y otros procesos en espera serían despertados, y el proceso P1 deberá buscar, nuevamente, en la hash queue y en la free list.

(b) ¿Qué sucedería si un proceso P1 solicita el bloque 113?

Escenario 2: El *Kernel* no encuentra el bloque en la *hash queue* y utiliza un *buffer* libre.

Si un proceso P1 solicita el bloque 113, lo que sucedería es que el *Kernel* no encontraría este bloque en la *hash queue* y se tomaría el primer *buffer* de la *free list* (el menos recientemente usado). El *Kernel* removería el *buffer* de la *free list* y cambiaría su estado a *busy*. El *Kernel* realizaría una operación de lectura desde el disco para cargar el bloque 113 en el *buffer* obtenido. El proceso P1 quedaría bloqueado en espera a que finalice dicha operación de E/S. Una vez finalizada esta operación de E/S, el *Kernel* asignaría el *buffer* al proceso P1, insertaría el *header* del *buffer* en la *hash queue* correspondiente (sólo se cambian punteros, no se intercambian ubicaciones de memoria) y, finalmente, reacomodaría los punteros de la *free list* (ya que uno de sus elementos fue retirado).

(c) ¿Qué sucedería si un proceso P1 solicita el bloque 102?

Escenario 1: El *Kernel* encuentra el bloque en la *hash queue* y el *buffer* está libre (en la *free list*).

Si un proceso P1 requiere el bloque 102, lo que sucedería es que el *Kernel* encontraría este bloque en la *hash queue* y su *buffer* estaría disponible en la *free list*. El *Kernel* removería el *buffer* asociado al bloque 102 de la *free list*, cambiaría su estado a *busy*, lo asignaría al proceso P1 y, finalmente, reacomodaría los punteros de la *free list* (ya que uno de sus elementos fue retirado).

(d) ¿Qué sucedería si un proceso P1 solicita el bloque 9?

Escenario 1?: El *Kernel* encuentra el bloque en la *hash queue* y el *buffer* está libre (en la *free list*), pero está marcado como DW (*Delayed Write*).

Si un proceso P1 requiere el bloque 9, lo que sucedería es que el *Kernel* encontraría este bloque en la *hash queue* y su *buffer* estaría disponible en la *free list*, pero está marcado como DW. El *Kernel* realizaría una operación de escritura del contenido del *buffer* al disco. El proceso P1 quedaría bloqueado en espera a que finalice dicha operación de E/S. Una vez finalizada esta operación de E/S, el *Kernel* eliminaría la marca DW, removería el *buffer* asociado al bloque 9 de la *free list*, cambiaría su estado a *busy*, lo asignaría al proceso P1 y, finalmente, reacomodaría los punteros de la *free list* (ya que uno de sus elementos fue retirado).